# 情報とプログラミング

—歴史と数学からの入門—

snow00two

2026 年 10 月 25 日

# イントロダクション

この本は理系全般，特に数学に興味のあるの大学生・社会人に向けて情報学とプログラミングの入門となることをまとめた．特に情報学とプログラミングの歴史に関係したこととそこに表れる数学に関連したこととから説き起こし，すでに知っていると思われる高校の情報 I のレベルより高いレベルへの入門を目指した．

このような本をまとめた背景を述べたい．学生に接していると CLI=Command Line Interface を触ったことがなく，開発環境をローカルに構築したことがない，VSCode すら触ったことの無い学生が多数いることがわかる．またコロナ禍以降，スマートフォン依存もよく見かけるようになった．スマートフォンは時間を意味なく消費し，目や耳や首や肩などを始めとして体の不調を引き起こし，脳の発達を邪魔し，アルゴリズムが世界観をも壊している．更に，これからは生成 AI に依存・代替される人間が増えることが予想されている．

これらに対応するため，これからの学生・社会人には

- スマートフォンとインターネットと生成 AI に適切な距離を置ける
- CLI を使いこなす (生成 AI は既に使いこなしているので，逆に生成 AI を使いこなすためには必須である)
- 必要に応じて自力で開発環境を構築し，ツールがつかえるようになる
- 幾つかのプログラミング言語でプログラミングができる
- 生成 AI に Markdown などを上手く使って適切に構造化した指示が出来る
- 生成 AI により拡散された偽情報を見抜ける

ことが大切になると思われる．

そこでこの本では CLI の演習を行い，デジタルデータの仕組みを述べ，取り扱いを説明する．次に論理回路やチューリングマシンなど計算論の基礎への説明を与え，インターネットや WEB の仕組みやセキュリティーについて説明する．最後にはデジタルデトックスについても論じている．付録には具体的で身近なプログラミング言語 (例えば Excel/VBA, LaTeX, markdown, JavaScript, Shell Script, git/GitHub) のプログラミング演習となるような資料を幾つか上げたので，なるべく多くのものにチャレンジし，技術を深掘りしてほしい．また，演習問題も多数あげたので自分の理解の確認に使って貰えると有り難い．授業のレポートで友達のものやネットに上がっているものを単にコピペしても理解

も技術も深まらないように，生成 AI の答えを貼り付けても理解も技術も深まらない．それらを参考にしても構わないが，大凡のアイデアを理解したら，コードが読みやすいようにソースを適切にまとめ，分かり易い変数の名前を選ぶとか，適切なコメントを入れるとかしてほしい．自分なりにプログラミングの本質を理解し，アイデアを検討しなおし，適当な関数にまとめるとか，クラスにまとめるとかしてほしい．そうすれば理解も技術も深めて自分のものにすることができ，全体を合わせると，上に上げた項目に対処できる力が得られるはずである．

なおこの本では MacOS の英語表示を使って説明をしている．日本語表示でもほぼ同様である．Windows でも大体は同じことが成り立つと思うが，慣れないと対応が分かりづらい部分もあると思う．ネット等で調べたり生成 AI に聞いて各自対応してほしい．

この本の図のソースファイルや関連した動画やアプリを私のサイトに用意したのでそれらを動かしながら理解を深めてもらいたい．

# 目次


イントロダクション ・・・

4.2.3 Web のサーバー・クライアントの演習 ・・・・・・・・・・・・・・・・・・・ 60
第 5 章 セキュリティ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
5.1 暗号の数理・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
5.1.1 $\mathbf{Z}$ の剰余類 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
5.1.2 公開鍵暗号—RSA・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 70
5.1.3 デジタル署名 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 72
5.1.4 認証局・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 73
5.1.5 暗号を実装するプロトコル ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 74
5.2 セキュリティマネージメント・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 74
5.2.1 セキュリティの基本概念・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 74
5.2.2 技術的脅威 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 77
5.2.3 技術的脅威への対応 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 79
5.2.4 人間的・物理的脅威と対応 ・・・・・・・・・・・・・・・・・・・・・・・・・・ 82
第 6 章 情報・社会・AI・健康・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 87
6.1 情報とは ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 87
6.2 情報社会と法律 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 87
6.3 AI の発達の歴史 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 88
6.4 情報社会と心と体の健康 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 88
6.4.1 データ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 89
6.4.2 対策の前提となる考え方・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 89
6.4.3 デジタルデトックス ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 91
付録 A Excel の関数と VBA ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 93
A.1 Excel の関数と VBA ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 93
A.1.1 Excel の関数でのプログラミング・・・・・・・・・・・・・・・・・・・・・・・ 93
A.1.2 VBA でのプログラミング ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 94
A.1.3 関数によるデータ分析 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 99
A.1.4 歴史と参考文献 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 100
付録 B TeX と Markdown・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 103
B.1 TEX ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 103
B.1.1 TEX のインストール ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 103
B.1.2 TEX のタイプセット ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 104
B.1.3 TEX の図式作成ソフト・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 108
B.1.4 TEX の描画ソフト・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 108
B.2 Markdown ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 115

付録 C JavaScript と p5.js ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 119
C.1 p5.js の 2D アプリ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 119
C.1.1 p5.js ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 119
C.1.2 一般の JavaScript の動かし方 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 120
C.1.3 JavaScript の言語仕様 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 121
C.1.4 2D アプリのソース ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 122
C.1.5 p5.js の 2D モードの説明とソースファイルの解説 ・・・・・・・・・・・・・・・・ 126
C.2 p5.js の 3D-アプリ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 127
C.2.1 3D アプリのソース ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 127
C.2.2 p5.js の 3D モードの説明とソースファイルの解説 ・・・・・・・・・・・・・・・・ 130
付録 D Shell と Shell Script 入門・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 133
D.1 標準入出力とパイプ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 133
D.2 Shell Script・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 134
付録 E Git と GitHub・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 139
E.1 バージョン管理の必要性 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 139
E.2 バージョン管理ツールの歴史・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 140
E.3 Git のインストール・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 140
E.4 Git の最低限の基本操作・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 141
E.5 Git のやや詳しい操作 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 143
E.6 GitHub ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 145
E.7 VSCode と Git/GitHub ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 148
E.8 GitHub Pages ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 148
E.9 この先に ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 150

# 第 1 章　CLI 入門

## 1.1　コンピューターの歴史

コンピューターの歴史を簡単に振り反ってみる：

- 19 世紀半ばにバベッジ (Babbage) による階差機関・解析機関の構想があった．計画は現代のコンピューターに近かったが，開発には成功しなかった．近年実現された階差機関
- 19 世紀後半にはアナログコンピューターが登場した．例へば 1930 年頃のバネバー・ブッシュ (Vannevar Bush) の微分解析機がある．
- 19 世紀末にホレリス[1)] (Hollerith) により開発されたパンチカードを使った集計で合衆国の国勢調査が行われた．
- 1936 年チューリング (A. Turing) がチューリングマシンの概念を発表し，計算可能性について考察を行った ([68])．
- 最初の電子計算機 ENIAC がモークリー (J.Mauchly) とエッカート (J.P.Eckert) により 1943〜1946 に開発された．
- 1951 年フォン・ノイマン (von Neumann) による史上初のストアドプログラム (stored program) 方式のコンピューター EDVAC が開発された．現在のコンピューターは全てこのノイマン型の子孫である．そのころの入力法は紙テープやパンチカードだった．
- 最初は機械語でプログラムされていたが．グレース・ホッパー (Grace Hopper) によりコンパイラーが開発され，アセンブラでコーディングされるようになり，更に FORTRAN や COBOL などの高級言語が生まれた[2)]．
- Operating system の発達：最初はオペレーターと呼ばれる人が順番にまとめてプログラムを読み込んで実行するバッチ処理 (batch processing) の形で行われていた．その後 1964 年にオペレーターの代わりをプログラム自身で行う Time sharing system の CTSS(Compatible Time Sharing System) が生まれ，テレタイプやキーボードからモニ

1)　のちの IBM( International Business Machines ) の創業者の一人

2)　FORTRAN(formula translator) は IBM, John Warner Backus により 1957, COBOL(Common Business Oriented Language) は G. Hopper, CODASYL(データシステムズ言語協議会) により 1959 に開発された ([71]). 共に現役で使われている言語である．

ターを見ながらコンピューターに命令するというインタープリターを使った使用が盛んになり，そこからそのプログラムは OS (Operating System) と呼ばれるようになった．数々の商用 OS(例 IBM System360) が発売され Multics の失敗ののち，特に

UNIX[3] と C 言語と shell の誕生：

- 1969[4] AT&T の Bell 研究所のケン・トンプソン (Ken Thompson) により PDP-7 上でアセンブラで UNIX の開発が始まった
- 1970 PDP-11 に移植 (図 1.1)
- 1973 デニス・リッチー (Dennis Ritchie) はケン・トンプソンの開発した B 言語をもとに C 言語を開発し，ケン・トンプソンと共に C 言語で Unix を書き直した．始めての高級言語による OS の開発だった．これにより OS の移植性が格段に高まった．
- 1991 リーナス・トーバルズ (Linus B. Torvalds) が Linux を公開し，今でも開発が続いている．皆さんが使っているアンドロイド携帯やクラウドのコンピューターの多くは Linux ベースである[5]．

**図 1.1　Ken Thompson (sitting) and Dennis Ritchie working together at a PDP-11, タイプライターのようなものがテレタイプ (Wikipedia より)**

3)　UNIX は X/Open Company Ltd. の登録商標．ただし，カーニハンは Unix と綴ることを勧めている ([33]).

4)　この年インターネットの最初の実験とアポロ 11 号による最初の月着陸が行われた．

5)　Linux の重要な判断の最終決定はトーバルズが行っている．そのためトーバルズに何かがあったとき，コンクラーベが行われることになっている

- 1970 年代 GUI(Graphical User Interface) が開発された (§1.2).
- 個人的には嫌だが，BCI(ブレイン・コンピューター・インタフェース）の時代が来るかも知れない.

## 1.2 GUI の歴史

- 第二次大戦後の 1945 年バネバー・ブッシュは “As we may think” で MEMEX の構想を述べた.
- 1963 年サザーランドがスケッチパッドを開発
- 1960 年代中期にエンゲルバートがマウスを開発
- 1974 年 Xerox の PARC (Plo Alto Research Center) で ALTO が作られ，アラン・ケイ (Alan Kay) が 1977 年にダイナブックの構想を発表した．最初の オブジェクト指向言語の一つの SmallTalk や Multi-Window システムも開発された.
- PARC の訪問に刺激をうけたスティーブ・ジョブス (Steven Jobs) による Lisa 開発と 1983 年発売，失敗
- Lisa 開発から外れたスティーブ・ジョブスによる Macintosh が 1984 年[6] 発売された.
- 1981 年に IBM は IBM PC を発売したが，その OS を MicroSoft に発注し，Bill Gate の率いる MicroSoft は 86-DOS を元に MS-DOS = MicroSoft Disc Operating System を開発した．1984 年の Macintosh の発売の後，1985 年に MS-DOS の上に GUI として Windows 1.0 を開発・発売し，1992 年に Windows 3.1, 1995 年に Windows 95 が発売され，現在に繋がる.

コンピューターの発展の詳しい歴史については [23], [17], [9], [27], [30], [57] などを参照してほしい．YouTube などで検索すると興味深い動画も多数見つかる．インターネットの発達と生成 AI の発達については後日解説する.

## 1.3 CLI

現在では GUI で色々なことができるようになり，実際ほとんどの人が GUI を使っていると思うが，Command Line Interface(CLI, CUI =Character User Interface と呼ばれることもある) でないと出来ないことも多い．何が出来て何が出来ないかは CLI を使ってみたことがないと分からないと思うが，生成 AI も CLI を使いこなしているので生成 AI が何を

6) 「1984 年」は現代社会を予言したジョージ・オーウェルのディストピア小説 [53]．このリンク先はリドリー・スコット監督が 1983 年に発表した Macintosh のコマーシャルである [23].

しているのか分かるぐらいまでは CLI を使いこなしてほしい．

ここでは CLI の OS の例として Unix を取り上げ (MacOS は Unix の兄弟 FreeBSD を基に作られていて，Linux も Unix の親戚に当たる)，まず Unix のファイルシステムについて紹介する．shell の使い方については次のセクションで説明する．

- ディレクトリー：

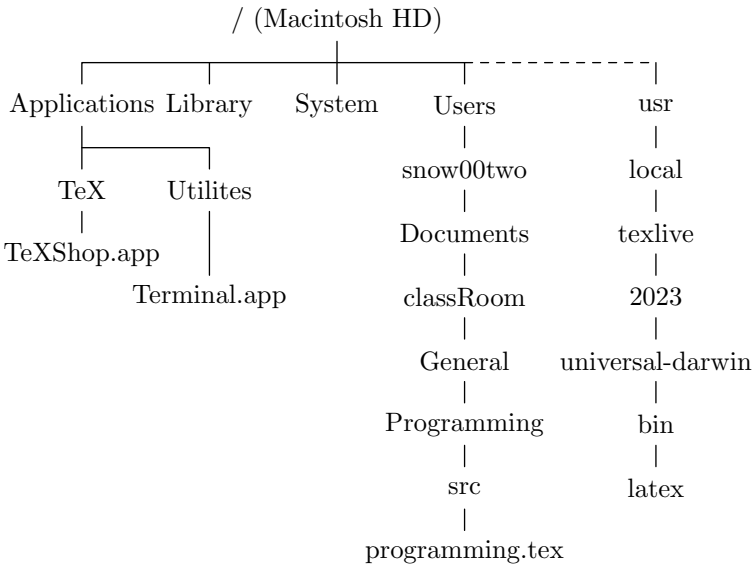



**図 1.2　ファイルシステムのツーリー構造**

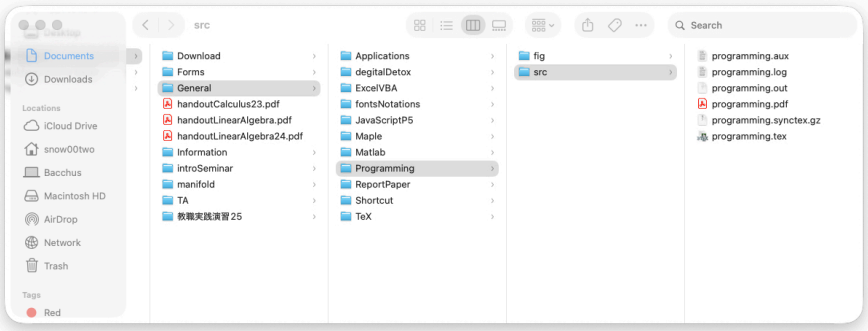



**図 1.3　Column 表示でのツーリー構造の確認**

ディレクトリーやファイルは図 1.2 のようなツリー構造に配置されている．特にパソコンのハードディスクの一番上の部分をルートといい / と表す．全てのディレクトリーやファイルはルートから一意的にたどっていくことが出来る．ファイルやディレクトリが/ から見てどこにあるかを示すのを絶対 path という．例えば図 1.2 の programming.tex は

/Users/sonw00two/Documents/classRoom/General/
Programming/src/programming.tex

のように表される．ただし，Windows ではパスを

```
C:\Windows\
```

のようにバックスラッシュ\を使って表示する．また，C というのは MSDOS では A, B がフロッピーディスクドライブ (3.5 インチフロッピーディスク (図 1.4) とフロッピーディスクドライブ (図 1.5)) として使われていたので C を起動用ハードディスクとして使った名残である．事情があってこのようにパスの表示が Uuix 系と異なることになったが，コンピュータにおける負の遺産の一つと言える．

**図 1.4　私の引出の奥から見つかった 3.5 インチフロッピーディスク．いくつかのアプリではファイル保存のアイコンとしても生き延びている．**

**図 1.5　Macintosh のモニターの右下にあるのがフロッピーディスクドライブ Marcin Wichary from San Francisco, CC BY**

後で出てくる .. は一つ上のディレクトリーをあらわす相対 path の例

- 不可視ファイル：ディレクトリ /usr は普通には見えない設定になっているので知らないとアクセスできない．Finder のメニューバーの Go のメニューから Go to Folder を選ぶと四角いウィンドウが開くので /usr と入力し，go をクリックで移動できる．Finder で cmd shift G とタイプしてもよい．
- 拡張子：ファイル名には hoge.tex の .tex のような拡張子 (extension) があるが，多くの場合アイコンのみ表示され見えないようになっている．開発のときは様々なファイルを扱うので，拡張子を表示しないと効率的なファイルの管理が難しい．また，メールの添付ファイルのアイコンは偽造できるが拡張子により直ぐに峻別できるので，セキュリティーの観点からも表示するのが望ましい．
- 関係したファイルは一つのディレクトリーに，関連していないファイルは異なるディレクトリーにいれ，関連のある，あるいは類似のディレクトリーは纏めて一つの大き

なデレクトリーに入れるなど入れ子構造にして（それがツリー構造の定義である）管理するのが基本である．また，簡単にそのディレクトリーを説明するテキストファイル ReadMe.txt, README.md をそのディレクトリーに置くことも良い習慣である．

## 1.4 CLI 実践演習

オバマ元大統領からのメッセージ

以下のアプリのインストール・色々なサービスへ登録してもらい，コマンドラインの使い方を説明する．ただし，CLI では破滅的なことも出来るので，バックアップは自分でこまめに取っておくようにして下さい．プログラミングの演習で生じた被害への補償は一切行えません．この免責事項を認めた人だけがこの本を読み続けることが出来ます．

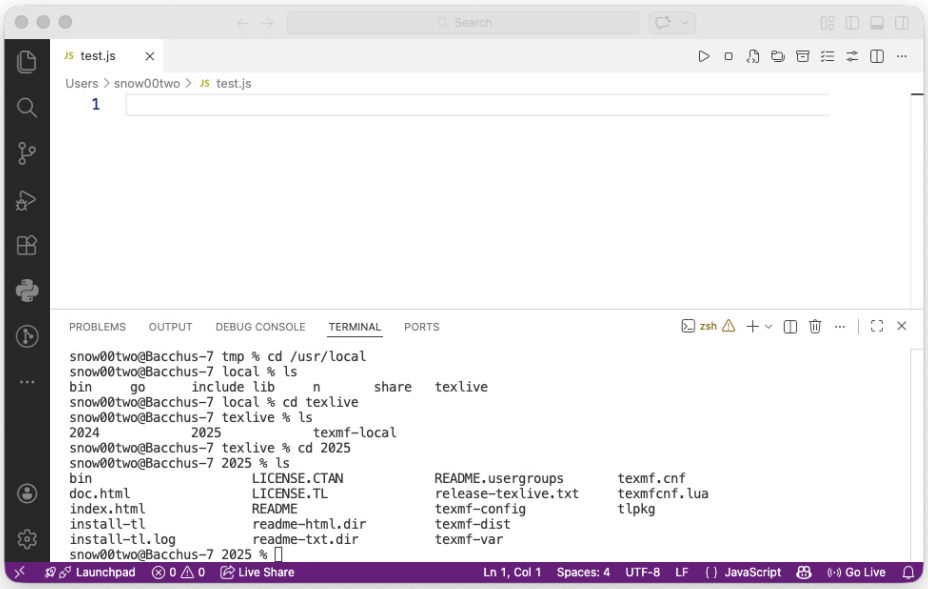



**図 1.6　VSCode，下が terminal**

**演習 1**　1. VSCode のインストール：

フリーのエディタというより統合開発環境 (IDE = Integrated Development Environment)Visual Studio Code （略称 VSCode, ここからダウンロード可能，図 1.6 参)．様々なコンピューター言語のための（自動補完，構文解析のための）インテリセンス，リンター，スニペット，ユニットテスト[7]等の機能拡張がある．マイクロソ

7)　詳しい説明はネット等で調べてください．

フトの Visual Studio シリーズのエディタのみをフリーで公開したもので，他にも似たものが色々あったが，最近はデファクトスタンダード．JavaScript で作られたアプリの一つで Chromium+JavaScript+Node.js+Electron により作られている (図 1.7).

2. terminal /shell:

   Mac や Linux 等では既にインストールされている．Windows では Powershell あるいは WSL(Windows Subsystem for Linux) をインストロールする必要があるだろう．

   これから必要になる tool 等も CLI と上でも述べた tool などを使ってインストールする必要のあるものが多い．Mac の標準の terminal は

   /Applications/Utilities/Terminal.app

   にある．Mac なら (Home)brew もインストールしておくと都合が良い．ここからダウンロード

   不可視なディレクトリやファイルも沢山あるので適宜コマンドで確認する必要がある．VSCode の下の部分にも terminal が付いている（図 1.6）.

3. version 管理の基本ソフト git とネットサーバー GitHub. 多人数で開発を行うにはこれを使いこなさないといけないが，個人開発でも有用．GitHub で VSCode の設定ファイルの同期も出来る．大学等で Microsoft365 に契約していると，そのアドレスで GitHub に登録し Student Developer Pack を使うように設定できるかもしれない.
4. 大学等で Microsoft365 に契約していると，ログインすると Copilot（chatGPT を office suite 向きにアレンジしたもの）が使用料無しで使えるようになっているかもしれない．GitHub の Student Developer Pack に入っていると，VScode の拡張機能タブで「GitHub Copilot」を検索してインストールすると，使用料無しで VScode から GitHub Copilot pro が使えるようになるかもしれない.
5. JavaScript は Chrome などの Browser で既に使えるようになっているが，セキュリティのためディスクへのアクセスなどは制限されているので，強力な実行環境となる node.js とその管理ツール npm も必要になる．ここからダウンロード. これからは TypeScript にも慣れる必要があるだろう.
6. Python3 とその管理ツール pip や (Ana)conda も必要ここからダウンロード

**演習 2**　拡張子を表示するようにパソコンを設定して下さい.

**演習 3**　VSCode の設定を，必要なら検索か Copilot に聞いて，GitHub で同期するように設定して下さい.

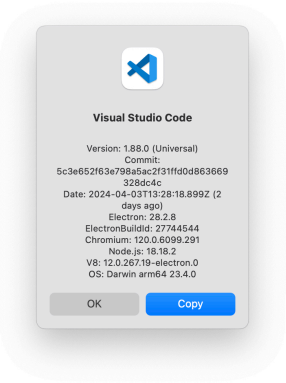



**図 1.7　VSCode のクレジット**

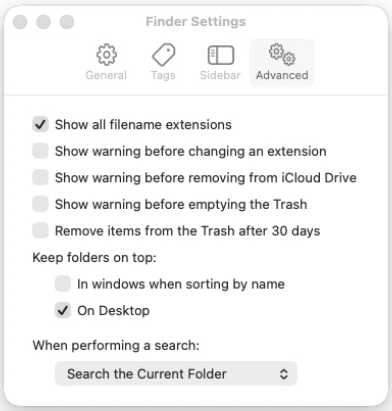



**図 1.8　Mac でのエクステンションの表示**

**演習 4**　VSCode で GitHub Copilot を立ち上げ，（もし答えを既によく理解していれば簡単だが，その分野の知識がないと難しい）以下のような質問をいくつか行って下さい．

1. 大学の代数の数学者として以下の質問に数学科の学生に分かるように答えて下さい．標数 $p$ の有限体が素体 $\mathbf{Z}/(p)$ のガロア拡大であることを示し，そのガロア群がどうなるか教えて下さい．
2. 大学の幾何の数学者として以下の質問に数学科の学生に分かるように答えて下さい．実射影平面 $\mathbf{RP}^2$(2 次元の実射影空間) の定義を述べ，それが閉多様体であることを示し，その $\mathbf{Z}$ 係数ホモロジーがどうなるか教えて下さい．
3. 大学の情報の専門家として command line interface に不慣れな学生に，git, GitHub の command line interface による管理の仕方を説明して下さい．
4. 最近 LLM や生成 AI を勉強していて，それ自体はよく出来ていると思いますが，この方向で AGI があと数年で出来るとも思えません．しかし生成 AI を開発している会社の CEO 達は「あと数年で AGI が出来そうだ」などと言っています．人工知能と経営学のそれぞれの専門家の立場から，なぜそのような言動を繰り返すのか答えてください．
5. 大学の経済学の専門家としてトランプ大統領にも分かるように，輸入した物に関税をかけると物価が上がることを説明して下さい．

その生成 AI の回答を読んで，（自分の知識の無い分野のその答の）内容が理解できたか確認して下さい．

コマンドラインインターフェースを使いこなさないとプログラミングは難しい．以下に

簡単なコマンドの使用例をあげる．terminal(端末) を立ち上げるとプロンプトが表示される (図 1.9)．私の端末では · · · % となっているのがプロンプトで，そのままキーボードからコマンド（文字）を打ち，リターンキーを押すとそのコマンドが実行され，次の行に結果が表示される．これがコマンドラインインタープリター shell である．以下では % で

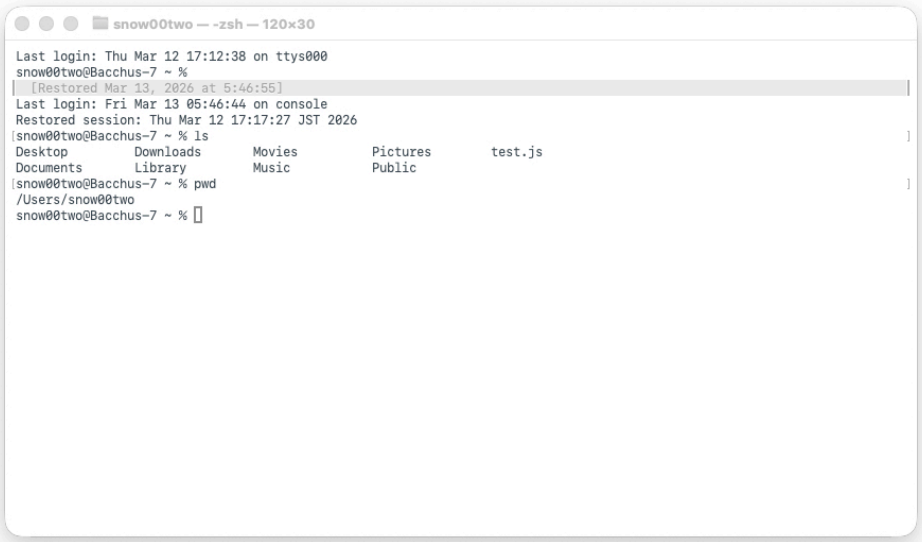



**図 1.9　Terminal**

プロンプトを表す．

以下# 以降はコメント

% cd # ホームディレクトリへ移動

% cd .. # 一つ上のディレクトリへ移動

% pwd # 現在のディレクトリの名前の表示

/Users

% ls # 現在のディレクトリにあるディレクトリやファイル表示

Shared sonw00two

% ls −a # 不可視なものも含めたディレクトリやファイル表示

localized Shared sonw00two

% cd sonw00two # このディレクトリのディレクトリ sonw00two へ移動

% mkdir tmp # ディレクトリ tmp の作成

% cd tmp # ディレクトリ tmp へ移動

% ps aux >ps.txt # 全てのプロセスを ps.txt に書く

% more ps.txt # ps.txt を表示. q で表示を閉じる

% mv ps.txt ps1.txt # ps.txt の名前をps1.txt へ

% cp ps1.txt ps2.txt # ps1.txt を ps2.txt にコピー

% rm * # このディレクトリの任意のファイルを削除. * はこのディレクトリの任意のファイル名を表すいわゆるワイルドカード. また, ファイルを全部消さないとディレクトリは削除できない.

% cd ..

% rmdir tmp # ディレクトリ tmp の削除 % rm –R tmp で中のファイルを消さなくても削除可能ではあるが慣れるまでは確認してから削除を勧める.

shell には全てをキーインしなくても, ファイル名などをある程度打ってタブキーを打つとあとの部分を補完してくれる機能がある. 他の開発環境でも異なるキーかもしれないが大抵補間機能がついている.

**演習 5** コマンドラインを使い, ディレクトリーを移動したり, ディレクトリーやファイルの作成や削除を行って下さい.

# 第 2 章　デジタルデータ

## 2.1　二進法

- 辞書を引くと「digit とは指, あるいはアラビア数字の 0〜9 のこと」とあり，その形容詞，あるいは (鍵盤楽器の) キー が digital で 対立語が analog とある．このことから digital は 10 進法と関係しているように見える．しかし，
- 0, 1 を中心に置いた 2 進法 (binary system) がデジタルの最小単位である．ブール代数では $x^2 = x$ をみたす $x$ が大事で，どの体でもこのような数は 0, 1 しかないことから 2 進法が表れるという説明を受けたことがある[1)]．電源ボタンも 0 と 1 を並べてデザインされたもの（図 2.1）である[2)]．

**図 2.1　私の室のプリンタの電源ボタン**

2 進法では

$$1011 = 2^3 + 0 \cdot 2^2 + 2^1 + 2^0 = 11$$

のよう数字が表される．0, 1 をビット bit(小片, または，binary digit の略語) という．$N$ ビットで表せる数は $2^N$ 個である．代数的には剰余環 $\mathbf{Z}/2^N\mathbf{Z}$ と見なすことが自然だが，これについては暗号の回に解説する．

また，負の整数 (補数) や小数も 2 進法で表され，コンピューター上では特殊な方法で処理される．入試などではよく出されるようだが今回は省略する．

- 16 進法 hexadecimal system では 0〜15 を

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}$$

1)　Bool[1] のコメントによる
2)　| は電線が繋がった様子を模式化したものという説明もある．

で表す．16 進法を使っていることを明記するため 0x1D (0x1d のように小文字を使う場合もある) のように書くことがある．

- 8 ビットを 1 バイト Byte という．$2^8 = 256 = 16 \times 16$ なので 16 進法の二つの数で表される．例えば

$$0x3A = 3 \times 16 + 10 = 58$$

のようになる．ただし，歴史的には Byte が 8 ビットに決まるまでは紆余曲折があったそうである：なぜ Byte は 8 ビットに決まったのか？

上のようなコンピューターの特性からコンピューター上でデータを表すにはそのデータを二進法で表す（近似する）必要がある．これを符号化という．以下データ毎にどのように符号化されているかを解説したい． 大きく二つに分けて，文字のならびからなるデータをテキスト text といい，その他のデータをまとめてバイナリー binary という．もちろん，数自体も大事なデータであるが，ここではテキストの一部と見なす．

## 2.2 テキストデータ

- ASCII (American national Standard-Code for Information Interchange) 米国情報交換用標準コード, 33 個の制御文字とスペースを含むキーボードの特殊文字・数字・アルファベットの大文字小文字の文字を 0 から 127 までの $2^7$ 個の整数で表したもの．1963 年に標準化された．例えば 0x7A は z を表す．二進数では 01111010 となり一番左の桁は常に 0 に成っていることも後で重要になる．また，制御文字の中で黒のものは現在では使われていない．

  ASCII の文字が写真のキーボーボードにすべて表れている (ただし，Tab, Control, Shift, Opt(Alt), Cmd, Fn などの修飾キーと Return は除く) ことに注意．

**図 2.2　keyboard(HHKB)–QWERTY 配列**

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | NUL | DLE | SP | 0 | @ | P | ‘ | p |
| 1 | SOH | DC1 | ! | 1 | A | Q | a | q |
| 2 | STX | DC2 | ” | 2 | B | R | b | r |
| 3 | ETX | DC3 | # | 3 | C | S | c | s |
| 4 | EOT | DC4 | $ | 4 | D | T | d | t |
| 5 | ENQ | NAK | % | 5 | E | U | e | u |
| 6 | ACK | SYN | & | 6 | F | V | f | v |
| 7 | BEL | ETB | ’ | 7 | G | W | g | w |
| 8 | BS | CAN | ( | 8 | H | X | h | x |
| 9 | HT | EM | ) | 9 | I | Y | i | y |
| A | LF | SUB | * | : | J | Z | j | z |
| B | VT | ESC | + | ; | K | [ | k | { |
| C | FF | FS | , | < | L | \ | l | \| |
| D | CR | GS | - | = | M | ] | m | } |
| E | SO | RS | . | > | N | ^ | n | ~ |
| F | SI | US | / | ? | O | _ | o | DEL |

**表 2.1　ASCII—上の行の数字＋列の 16 進数**

TEX でお馴染みかも知れないが \ と ASCII に含まれない ¥が文字化けする問題がある．Mac でもこれが困った問題を引き起こすことがあるので注意が必要 (Windows のパスの区切りがバックスラッシュ\であることも問題を複雑にしている).

ここに載せたキーボードは左上のアルファベットの並びが QWERTY であることから QWERTY 配列と呼ばれている．

tab （インデント）ついて：tab には機械タブ　　(これではよく分からないので\t と書くことがある ) と空白何個かで作るタブ　　の二種類がある．プログラミンではコードのツリー構造を人間に分かり易くし可読性を高めるためタブを入れることが勧められるが，機械タブと空白何個かで作るタブを区別するもの（コンパイラー）があるので注意が必要．美的にも二つのタブが混ざると綺麗でない．

キーボードが出てきたのでついでに：一生ものの外付けキーボード (定義より特殊文字の位置は固定されている) を手に入れて (図 2.2 は私の HHKB キーボード，確か二万五千円ほどした．ただし，テンキーは別途購入する必要がある) 使い込むこと：

1. touch-type[3] をマスターし，コマンドラインインターフェースに慣れる．当たり前だが練習しないとマスターできない．YouTube などには沢山の練習動画がアップされている．
   (a) F と J のキーの上には突起があり，人差し指の触覚だけで位置が確認できるようになっている．
   (b) ラップトップパソコンだとついキーが見えてしまうので，タッチタイピングの練習のためにも外付けキーボードが必要である (私はやったことが無いが，見えないように手の上に布を置くという方法もある)．
   (c) ラップトップのキーが外付けキーボードと異なっているときは，私は Karabiner-Elements という機能拡張で，HHKB となるべく同じになるように設定している (図 2.3)．タッチタイピングで一番重要なことは，日常使う全てのキーボード環境でショートカットなどの修飾キーと特殊文字のコンビネーションを目で見ずに正確にタイプできるようになることである．

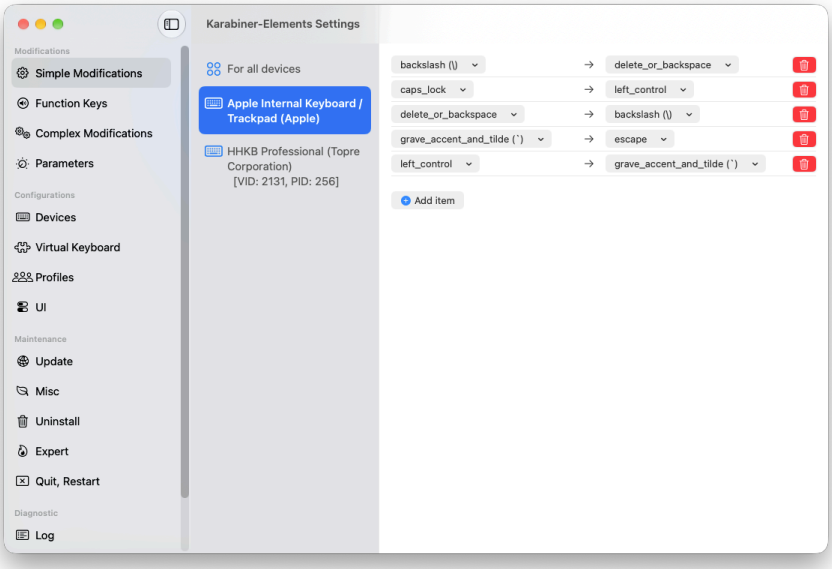



**図 2.3　karabiner-Element の設定**

2. 一生ものの外付けキーボードは一生付き合うものなので，キーキャップリムーバーを手に入れキーボードの洗浄[4] を定期的 (例えば一年に一回の大掃除のとき) に行なって衛生的にしてほしい．

3)　以前はブラインドタッチと呼ばれていたが差別的な響きがあったことからこう呼ばれることが増えた．
4)　全部のキーキャップをはずし，キーボードの下の部分の汚れを綿棒などでとり，容器に水と次亜塩素酸ナトリウムを入れたものにキーキャップを暫く浸けて水洗いし乾燥させて再装填．

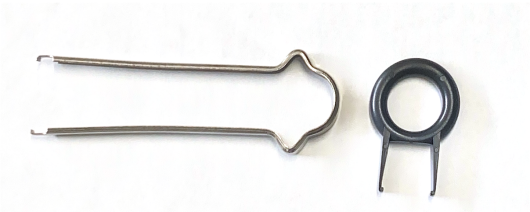


**図 2.4　Keycap Remover**

ことをお奨めする．

### 演習 6

```
% echo ABCDEFGHIJK > test
% od -c -t x1 test
0000000    A   B   C   D   E   F   G   H   I   J   K  \n
  41  42  43  44  45  46  47  48  49  4a  4b  0a
0000014
```

最初の行の最初の% は prompt, 次の > は redirect, >> にするとファイルがあるときはそれに追加される．od は octal dump の略で - 以下は option

経験者からのアドバイス：演習で解答し（他でもコンピューターの知識，あるいは，コンピューターに限らず何でも調べ）たら，メモ帳などに説明を含む簡単なメモを残すようにして下さい．結局は時間の節約になります．

**図 2.5　私の Mac のメモ (Notes)**

**宿題 1**　タッチタイピングを練習して下さい．

- 日本語の文字コードについてはひらがな・カタカナ・漢字などの文字の種類が多いことから 2Byte($= 2^{16} \approx 64,000$) の数字を使う二バイト文字を使う必要があった．しかもその方法に JIS, SJIS, EUC など異なったものがあり文字化けが問題であった．更に，ネットの発達などにより世界中の文字への対応が必要となり，Unicode [5)]が制定された．
  - Unicode は数字に過ぎない．最近では約 11 万字以上を表すことができる．U+ の後に 16 進数で表し，U+000000〜U+10FFFF だけ有るが冗長性が大きいので，表現の体系が幾つかある．最近のスタンダードは UTF-8 である，他にも UTF-16, UTF-32 などがある．ただし，U+は入力しなくていい．
  - 例えば U+0041 は A を表し，ASCII を含んだ拡張になっていることが分かる．更に UTF-8 では例えば U+0041 を 0x41 の 1byte で表すような，1byte の最初の 2 進数が 0 の文字を ASCII と判定するアルゴリズムを用いている．つまり，ASCII は ASCII のままで使用できるようになっており既存のコンピューター資源をそのまま使うことが出来る．これが UTF-8 が普及した理由である [73].
  - 改行の示し方も UNIX, Mac では LF (line feed character, line finder 改行文字) で，Windows では CR+LF である．そこで改行の違う UTF-8 が存在するので注意が必要である．

**演習 7**　CLI で文字化け対応

```
% brew install nkf
# nkf がインストールされていないとき
# WSL でも多分 % sudo apt install nkf で OK
% nkf -s letter.tex > test.tex
% nkf -w test.tex > test2.tex
```

これらを普通のエディターで文字化けしないように開いて下さい．

**演習 8**　Unicode のローカル（自分の PC）での入力法について検索して（あるいは生成 AI に聞いて），ファイルに直接 UTF-8 で文字入力出来るようになってください．

例えば事務から送られてくるメールでよく文字化けしているの Unicode を調べて文字化けなく出力出来るようになって下さい．

5)　このサイトを訪れるとその壮大さに圧倒される．

**演習 9**　Windows などで未だに SJIS を使っている人は早めに UTF-8 に移行することをお奨めします．やり方を調べて実行してください．

- 文字のならびからなる文を文字列といい，プログラミングの中では変数や数や関数と区別するため’ や " で囲んで表すことが多い．例えば `"This is a statement."` のように表す．

**演習 10**　文字の数字の真の数字への変換：例えば shell で 1+2 を計算すると

```
% 1+2
zsh: command not found: 1+2 #当たり前か？
% echo 1+2 >text.txt
% od -c -t x1 text.txt
0000000    1   +   2  \n
         31  2b  32  0a    #少し考えると 1+2 は文字列
0000004
% echo $((1+2)) #調べるとこうすれば良いことが分かる
3
% python #もっとも今だとこのあたりが普通か？
>>> 1+2
3
>>> # control D で終了
```

自分の使っている (興味のある) 言語で文字の数字の数字への変換，逆の変換のやり方を調べてください．

## 2.3　画像データ

バイナリーデータは画像データや音声データ，プログラムの実行ファイルなどの区別がある．まずは画像データ：

- まず画像の物理的特性．テレビやパソコンのディスプレーで画像を作る単位を pixel(画素) という (図 2.8[6])．例えば私のモニターが 1200 × 1920pixel の表示のときは

6)　WikiPedia より By Ravedavel

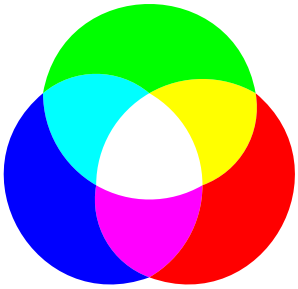


図 2.6　加法混色

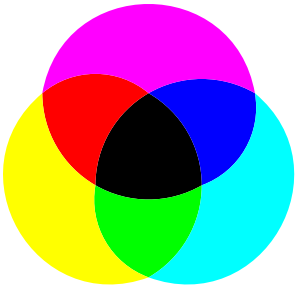


図 2.7　減法混色

$2,304,000 \approx 2.3 \cdot 10^6$ 個の pixel を使っていることになる．各 pixel の色は RGB, つまり，Red Green Blue の略で加法混色 (図 2.6) することで色々な色を表現できることが分かっている．これらは $[0,1]$ の中の実数の三つ組みで表せるが，実際は離散化（量子化とも言う）し，それぞれの色を 1 バイト ($2^8 = 256$) に分けて近似する．全体で 3 バイトで

$$2^8 \cdot 2^8 \cdot 2^8 = 2^{24} \approx 16 \cdot 10^6$$

種類の色を表すことができる（この仕様をフルカラーという）．それらが pixel の数だけあるのでコンピュータの一画面を表すのに必要なデータは

$$16 \cdot 10^6 \times 2.3 \cdot 10^6 \approx 36.8 \cdot 10^{12} \approx 40 \text{ 兆個}$$

あることになる．

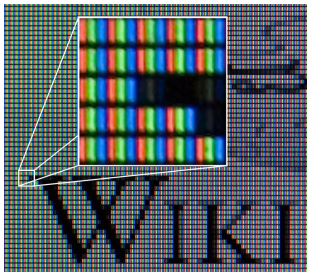


図 2.8　pixel

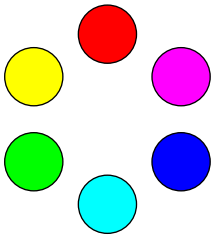


図 2.9　色相環

色を表すのには直接 0〜255 を三つ並べて RGB を

$$[125, 45, 255] \quad \text{or} \quad (125, 45, 255) \quad \text{or} \quad \text{color}(125, 45, 255)$$

のように書いたり，カラーコードといって#の後に 16 進法の数字を 6 つ (省略形で 3 つ) 並べて

#a43aff

のように表すことがある．また，red, blue, mazenta などように既に定義された名前をつかえることがある（どのように表すかは使っている言語による）．

**演習 11**　VSCode ではカラーコードを入力するとサンプルが表示される機能がある (図 2.10) ので実際に確かめてください．

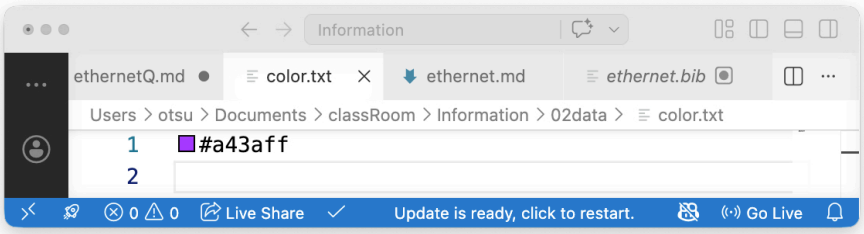



**図 2.10　VSCode でカラーコード表示**

印刷用には CMYK(Cyan Magenta Yellow blacK) という減法混色が用いられる (図 2.7)．

- 色については美術や心理学の観点から，
  - 色相 (hue)：赤や青などの色の違い (色相環 図 2.9 参)．
  - 彩度 (saturation)：色の鮮やかさ
  - 明度 (luminosity, brightness)：色の明るさ

  により説明される．更に
  - 補色（図 2.9 の反対側にある色）
  - 暖色・寒色
  - 有彩色・無彩色
  - トーン
  - ベースカラー，メインカラー，アクセントカラーの比率

  などの概念を知って，それを使いこなす練習をすれば実際のデザイン・プログラミングにも役立つだろう (例えば [58], [64], [70], [71], 参照)．
- 画像を動かすことで動画が得られる．個々の画像を frame という．テレビ放送では 30fps (=frames per second) つまり，1 秒間に 30 frame 表示することになっている．したがって 1 秒間に 1200 兆個のデータを使っていることになる．
- 画像データをコンピュータで表すには点と直線や曲線の組で表すベクタ表現，図形を画素の集合として表すラスタ表現の二種類の方法がある．ベクタ表現は拡大縮小が自由であるがラスタ表現は拡大すると見づらくなる．

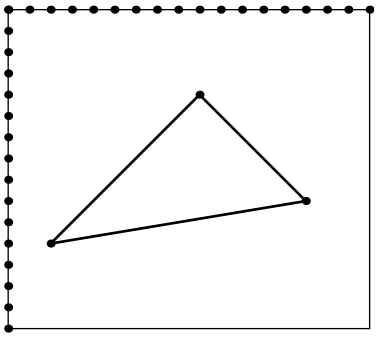


図 2.11　ベクタ表現

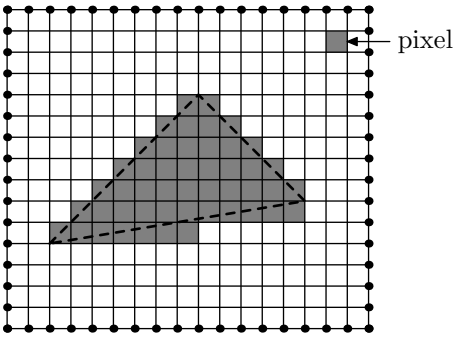



図 2.12　ラスタ表現

- 文字符号を具体的に画像データとして表示した物がフォントである．フォントにはベクタ表示のアウトラインフォントとラスタ表示のビットマップフォントがある．近年ではアウトラインフォントが主流となっている．現代では様々な書体のフォントが使用可能である．

  ただし，□（豆腐）という形で，フォントがない文字が表されることがある（いたずら好きの友達がヒエログリフで送ってきた可能性もないわけではないが，pdf で文字の埋め込みに失敗した可能性の方が高い）．これを起こさないために Google による NoTo (=No Tofu) いうもプロジェクトが進んでいる．

## 2.4　音声データ

音は物理的には空気中の圧力の差が秒速 340m ぐらいで伝播していく縦波であり，それを静止あるいは比較的低速で移動する動物が耳で捉えたものである．よって，音楽データは時間 $t$ についての圧力の変化 $f(t)$ によって表せる．まず $f(t)$ は単なる連続関数で (場合によっては不連続点のあるものも扱うので) そのままではコンピュータでは扱えないことを注意する．

ヒトの聴覚は 20Hz(ヘルツ) から 2 万 Hz までといわれているのである程度高い音は聞こえないと考え，フィルターで除外する．その周波数を $W$ とし，そのため生じる最小周期を $T = \frac{2\pi}{W}$ と置く．得られた関数をもう一度 $f(t)$ と置き，$n$ を整数として

$$a_n = f(nT)$$

を作る (図 2.13)．$W$ をサンプリング周波数，$\{a_n\}$ を作ることを サンプリングという．

$$f_n(t) = \frac{\sin \pi(n - \frac{t}{T})}{\pi(n - \frac{t}{T})}$$

を使って

$$\bar{f}(t) := \sum_{n=-\infty}^{\infty} a_n f_n(t)$$

を作ると，$f$ が $\bar{f}$ と一致するというのがシャノン-染矢のサンプリング定理である．

現実には $a_n$ の値は二進数に量子化され，そのビットを量子化ビット数という．音は高周波数成分を含むことと量子化によりある程度の誤差は避けられない．

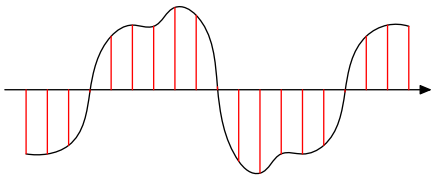


図 2.13　音波と標本化

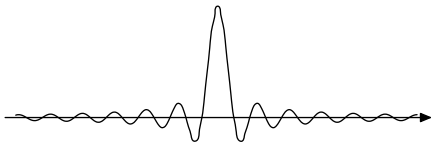


図 2.14　$f_0$(t)

具体的に CD の場合の規格を見てみると，サンプリング周波数は 44.1kHz で量子化ビット数は 16bit である．したがって，1 秒間で必要なデータ量は

$$64000 \times 44100 \approx 2.8 \times 10^{11}$$

ステレオだとこの 2 倍である．

DTM(=Desk Top Music) のためには音楽の理論や心理学についても必要になると思うがここでは取り上げない．このセクションの名前は音声データであるが，人の声は意味を伝える特別な音であるので後にもう一度取り上げる．

## 2.5　実行ファイル

コンピュータのバイナリーの中で重要なものの一つはプログラム (の実行ファイル) そのもののである．C 言語のファイル

```
#include <stdio.h>


int main(){
 printf("Hello, World!\n");
}
```

を hello.c という名前で保存．冒頭の<stdio.h>は標準入出力を使うために必要なライブラリーであり，それを最初に読み込んでいる．そのディレクトリで

```
% ls
```

```
hello.c          README.md
% gcc hello.c
% ls
a.out            hello.c          README.md
% ./a.out
Hello, World!
% od -c -t x1 a.out | more
```

最後の more ではスペースキーでページが更新され，B で前のページに移り，Q で終了する．gcc は gnu[7] c compiler の略である．

**演習 12**　上の作業を自分で行い，Hello, World! が何処にあるかを探してください．

現在では (システム開発者以外は)C 言語を知らなくてもそれほど困らないと思うが，参考文献としてはやはり [29] を挙げておく．また，これからは Rust や Go などを学んだ方が役に立つかもしれない[8]．

Shell Script について：

```
#!/bin/sh


echo "Hello World!"
```

を hello という名前で保存．#!/bin/sh は Shell Script の書き出しの決まり文句である[9]．そのディレクトリで

```
%ls -l
total 8
-rw-r--r--  1 snow00two  staff  33 Apr 30 13:28 hello
# このままだと実行可能でない
% chmod +x hello    # 実行可能に変換
% ./hello
Hello World!
```

hello はテキストファイルであるが，実行ファイルでもあるので，これもバイナリーと呼ばれることがある．

---

7)　GNU は GUN's Not Unix の頭文字でグニューと読む．R. Stallman が始めた．
8)　プログラミング言語の将来予測は極めて難しいことが知られている．
9)　私が昔読んだ [28] ではここが必要とは書れていないが，[74] では書くのは当然と書いてある．

**演習 13**　上の作業を自分で行ってください．

## 2.6　データの取扱い

次にデータの取り扱いについて述べる：

- データの授受の時よく使われるテキストの形式のことを構造化データという．いずれも基本 ASCII ファイルであり，(文明が崩壊しない限り) 未来永劫どのようなエディタでも開き編集することが出来るものである．
  1. CSV =Comma Separated Values：Excel のデータを文字列で書き出し ,(Comma) で区切ったテキスト (図 2.15)．, を tab に代えた (TSV =Tab Separated Values) もあるが，.csv の拡張子なのに実は tab で区切られたものも多い．

     また，Windows の csv ファイルにはファイルの先頭には，BOM (=Byte Order Mark, ぼむ) という 3 バイトのデータが着いていることがあり，Mac・UNIX 環境では悪さをすることがある．BOM の取り除き方については検索して探してほしい．

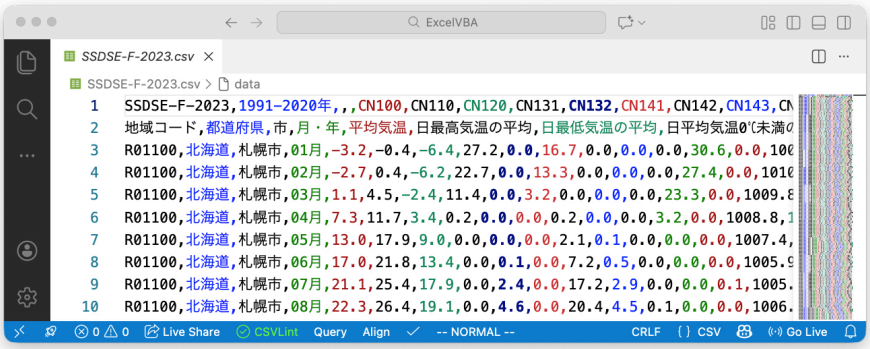



**図 2.15　CSV:SSDSE-F-2023**

  2. JSON = JavaScript Object Notation：元々は JavaScript の連想配列から作ったもの[10]だが最近色々な所で使われる (図 2.16)．npm では package.json というソフトウェアの依存性を記述したファイルがよく使われている．
  3. YAML = YAML Ain't Markup Language：Python に似た構造化データ．JSON よりコンパクトに記述できる (図 2.17)．ソフトの設定などに使われる．

10)　JSON は 2001 年クロックフォードにより JavaScript の中から「発見」されたといわれている．

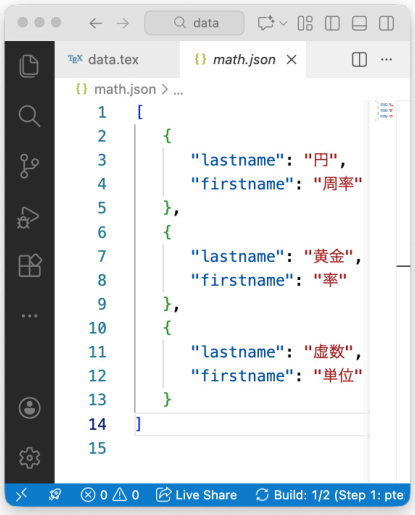



図 2.16　JSON

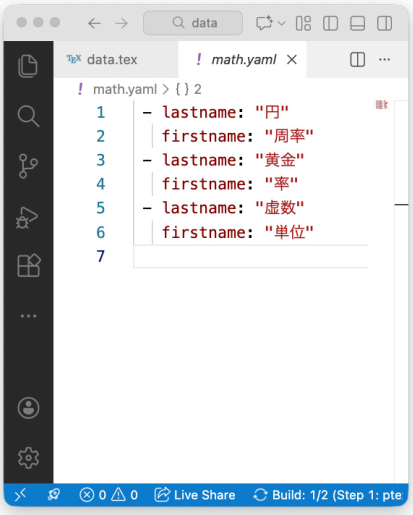



図 2.17　YAML

4. XML = eXtendable Markup Language：HTML に似たデータ記述言語で一時期はやったが，最近はあまり使われない．しかし，パソコンの内部では今でもよく使われている．

これらのアルゴリズムはコマンドラインのコマンドの形や，プログラミング言語のライブラリーの形で提供されている．

**演習 14**　yaml, json, xml 間の変換：yq を使う

```
% cat math.json
[
  {
    "lastname": "円",
    "firstname": "周率"
  },
  {
    "lastname": "黄金",
    "firstname": "率"
  },
  {
    "lastname": "虚数",
    "firstname": "単位"
```

```
    }
]
    % brew install yq #yqがインストールされていないとき
    % yq -P -oy math.json > math.yml
    % cat math.yml
- lastname: 円
  firstname: 周率
- lastname: 黄金
  firstname: 率
- lastname: 虚数
  firstname: 単位
    % yq -o json math.yml  > math2.json
```

- Base64：メールや Http のヘッダなどでバイナリーをテキストに変換して送る方式. バイナリーを 6 ビットずつに分け，それを 0〜63 の数字で表すとき 0 から 25 に A〜Z, 26 から 51 に a〜z, 52 から 61 に 0〜9, 62, 63 に- _ を当てはめることで（最後の二つは異なるやり方もある）文字列に変換するやりかたである.

  このアルゴリズムもコマンドラインのコマンドの形やプログラミング言語のライブラリーの形で提供されている.

**演習 15**

```
    % echo "abc" > test
    % tar -czf test.tar.gz test #適当なバイナリーを作成
    % base64 -i test.tar.gz
    H4sIAIj+uGkAA+3PMQ6AIAwFUGZPOSOOWOA8aGQ1Ebi/IMaRDY1J3/
    LTtMNv2mJSgyGiZYaazporUbf5xkCGjHGacbaApImcAhxdrMox+aNU2
    VPMvbtyFkJn3x6BJ3/CL+v0dQchhBDvOwEZLljpAAgAAA==
```

# 画面に入らないので適当に改行

- 任意の長さのデータから固定長のデータを取り出す関数をハッシュ関数 (hash funcition) といい，その値をハッシュ値 (hash value) という．特に一方向ハッシュ関数が重要である.

ただこのままでは何を言っているのかさっぱり分からないと思うので，一方向ハッシュ関数に期待されてる性質をまとめておく．

- どのような長さのデータでも出力は同じ長さ
- 計算は易しく，大して時間が掛からない．
- 耐衝突性：異なる入力には異なる出力を出す．もっともディリクレの原理から出力のビット数を超える個数の入力の中では必ず衝突が起こるので，少ない個数の異なる入力には異なる出力を返すという意味である．
- 一方向性：ハッシュ値から入力の逆算が難しい．

ハッシュ値は暗号とは異なることに注意する．また，そのアルゴリズムの設計も難しそうであることも分かる．

ハッシュ値の使い方について：ハッシュ値はコンピュータ・セキュリティの分野で広く日常的に使われている．幾つか例を挙げるが，これから先も関係した話題が出てくるだろう：

- パスワードの保存：パスワード自体をそのままで保存するのは危険なので，パスワードのハッシュ値を保存して，パスワードを打つ毎にそのハッシュ値を比較してパスワードが正しいことを毎回確認することが行われている．

  ただし，ありそうなパスワードのハッシュ値を事前に計算していおいて，そのパスワードのハッシュ値のリスト (レインボーテーブルという) と比較してパスワードを推測する辞書攻撃があるので，ユーザー毎に疑似乱数 (ソルトという) を決めておき，パスワードにソルトを加えたデータのハッシュ値を使う方法も行われている．
- ソフトウエアを配布するとき，配布元のサイトのミラーサイトを使うことはよく行われているが，それが本物であることを保証するため，そのソフトウエアのハッシュ値を計算して配布元のサイトに置いておき，ミラーサイトからソフトウエア手に入れた入手者がそのハッシュ値を計算し，配布元のサイトのハッシュ値と比較してそれが本物であることを確認することが行われている．
- 疑似乱数：先程もでてきたが一方向ハッシュ関数をつかうことで (ランダムに近く見える) 疑似乱数の生成が行われる．

現在使われている (た) 一方向ハッシュ関数を挙げておく．ただし，コメントに書かれた危殆化とは既に 不十分であることが分かっていることを意味していて，互換性のために未だサポートされているが，これからの積極的な使用は推薦されていない

| 一方向ハッシュ関数 | 長さ | 発表年 | コメント |
|---|---|---|---|
| MD5 | 128bit | 1992 | 危殆化 |
| SHA-1 | 160bit | 1995 | 危殆化 |
| SHA-256 | 256bit | 2001 | |
| SHA-512 | 512bit | 2001 | |
| SHA-3 | 可変 | 2015 | [73] に詳しい説明 |

**表 2.2　使われている (た) 一方向ハッシュ関数**

ことを意味している．

これらのアルゴリズムもコマンドラインのコマンドの形やプログラミング言語のライブラリーの形で提供されている．

**演習 16**

```
% echo "abc" > test
% sha256 test
```

SHA256 (test) = edeaaff3f1774ad2888673770c6d64097e391bc362d7d6fb34982ddf0efd18cb
# 画面に入らないので小さくした．

- 圧縮：データを保存したり送ったりするとき内容を変えないように圧縮して送れると保存量・通信量を減らせて都合が良い．元のデータ量と圧縮したデータ量の比を圧縮率という．

$$\text{圧縮率} = \frac{\text{圧縮した後のデータ量}}{\text{圧縮する前のデータ量}} \times 100\ \%$$

圧縮をといたとき元のデータが完全に復元できるのが可逆的圧縮，データが完全に復元できず欠損が起こるのを不可逆圧縮という．とくに音声・映像データは不可逆圧縮で十分 (で遅れがない方が大事) なことが多い．いくつか圧縮法の例を挙げる：

  - ランレングス法：この紙（あるいはモニター画面）のような白と黒などの繰り返しの多いデータの時，連続する同じデータの数を数えて示すのがランレングス法である．例えば

$$AAAAABBAAAAAAAABBBBBBBBAAAAA$$

のようなデータを $A5B2A8B7A5$ と圧縮する．

– ホフマン法：例えば

おきゃがれこぼし　おきゃがれこぼし

ゆんべもこぼして　またこぼした

という文 (歌舞伎「外郎売り」より, [26] には先代の市川團十郎の舞台の動画もある．) を（空白を除いて）適切なかたまりに分割すると次のような頻度になる．

| 部分 | こぼし | おきゃがれ | ゆんべも | て | また | た |
|---|---|---|---|---|---|---|
| 頻度 | 4 | 2 | 1 | 1 | 1 | 1 |
| コード | 00 | 01 | 100 | 101 | 110 | 111 |

**表 2.3　ホフマン法の例**

6 個の部分があるので 3bit 必要だが，頻度が多い 2 つの部分に 0 から始まる短い 2 進数 00, 01 を与え，残りの 4 つに 1 ではじまる 3bit の 2 進数を表のように与えると

010001001010010111000111

と置き換えることが出来る．文字の区切りがちゃんと判定できることを確かめてほしい．このように頻度に応じて短い数を当てるのがホフマン法である．

データの特徴を上手く使った方法を選ぶ必要がある．

**宿題 2**　発声練習：「外郎売り」を練習して下さい．

# 第 3 章　コンピュータの原理

アルゴリズム algorithm の語源はアラビアの数学者フワーリズミであるといわれる．彼の数学書「インド人たちの数」が 12 世紀にラテン語に翻訳されたとき「アル・フワーリズミは言った」から文が始まっていたが，その後名前であることは忘れられインド・アラビア式の計算の方法[1)]をアルゴリズムと呼ぶようになった ([44])．

現在ではアルゴリズムは算法という意味で使われるが，1930 年代に数理論理学において論理の限界と計算可能性と関連して論じられるようになった．フォン・ノイマンはこれらの研究のただ中にいたが，それらをもとに 1951 年に世界初のノイマン型コンピュータ EDVAC を開発を成し遂げた．

ここでは論理学と電子回路から分かるコンピュータの仕組み，更に時計付き回路とメモリーについて解説する．参考文献として [6], [43] をあげておく．

## 3.1　論理回路

- 1 を真 0 を偽として表す．コンピュータの内部ではある一定の正の電圧 $V$ がかかった状態が 1 電圧がかかっていない状態が 0 である．
- $A$, $B$ を 0, 1 を取る変数とするとき, $A$ の否定を NOT $A$，$A$ かつ $B$ を $A$ AND $B$, $A$ あるいは $B$ を $A$ OR $B$ と表す．真理表で表すと

| $A$ | NOT $A$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

**表 3.1**　NOT

| $A$ | $B$ | $A$ AND $B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**表 3.2**　AND

1)　インド・アラビア式の計算の歴史的な位置づけ：古代インドに起源を持つ 0 を含む数字と位取り記法が，アラビア数字に変わりながらイベリア半島を通じてヨーロッパに入ってきたのも 10 世紀頃であった．また，紙の生産は古代中国で始まったが，その製造法も 12 世紀頃にアラビアを通じてヨーロッパに入ってきた．それらは簿記などに使われ商業・金融の発展に繋がった [65]．その後活版印刷が発明され，科学や工学や文化の発達，宗教の対立と戦争の拡大を招いた．

- コンピュータの回路としては AND の否定の NAND, OR の否定の NOR, OR で 2 つが共に 1 のときを 0 に代えた XOR(排他的 OR) を考えると都合が良い．それらの真理表を書くと

| $A$ | $B$ | $A$ OR $B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**表 3.3**　OR

| $A$ | $B$ | $A$ NAND $B$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**表 3.4**　NAND

| $A$ | $B$ | $A$ NOR $B$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**表 3.5**　NOR

| $A$ | $B$ | $A$ XOR $B$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**表 3.6**　XOR

- 図式で考えた方が分かり易い場合があるので記号を準備する：

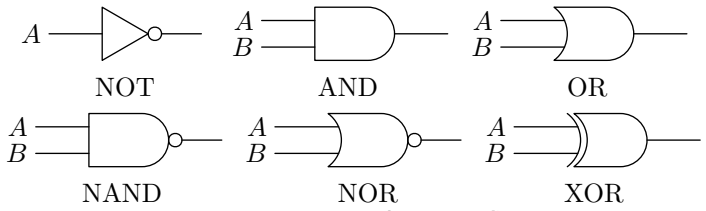


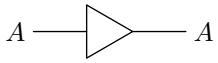

**図 3.1**　論理の図式

- 数の 0, 群での単位元，カテゴリー論での恒等射のように何もしない論理素子 ID も考えると都合が良い．それを以下のように表す．
- 一桁の二進数 (0, 1 のみからなる数) の和を計算し，二進数で表す論理回路を設計してみる．$A$, $B$ が与えられた一桁の二進数とし，和の一桁目を $S$ と表し，位上がりを $C$ と表すと真理表は表 3.8 のようになる．これから $S$ は XOR で $C$ は AND になることがわかる．よって図 3.3 ようにすれば良い：

**演習 17**　NAND のみで NOT, AND, OR, NOR, XOR を表せ．

| $A$ | ID $A$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

**表 3.7**　ID




ID

**図 3.2**

| $A$ | $B$ | $S$ | $C$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

**表 3.8**　**二進数の和**

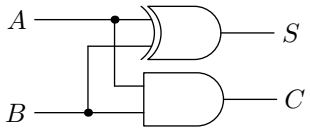



**図 3.3**　**一桁の二進数の和**

**宿題 3**　二桁の二進数の二桁目の計算をして（一桁目の位上がりがあることに注意）二桁目と三桁目を二進数で表す論理回路を設計せよ．それを使って $n$ 桁の二進数の和を計算する論理回路を設計せよ．

- 論理回路の FET(Field Effect Transistor) などを使った物理的実現（論理ゲート）の例を挙げる：

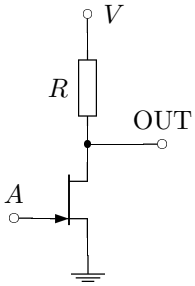



**図 3.4**　NOT

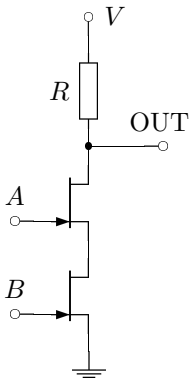



**図 3.5**　NAND

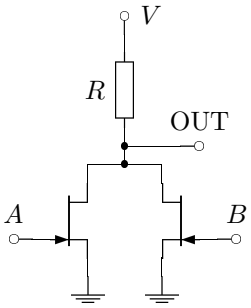



**図 3.6**　NOR

ただし，細い四角は抵抗，II みたいなものが FET である．

- 例えば AND ゲートは NAND ゲートと NOT ゲートの結合で実現される．NAND ゲートから NOT ゲートを作るのも簡単である．演習で見たように他の論理演算も NAND のみで表されるので集積回路は NAND ゲートのみから構成することが出来る．
- デコーダ：真理変数の組（これは二進数と同一視される）のデータから情報を取り出すには図 3.7 (a) ような回路を使えば良い．ただし，図の黒丸は (b) のような AND を使った回路とする．

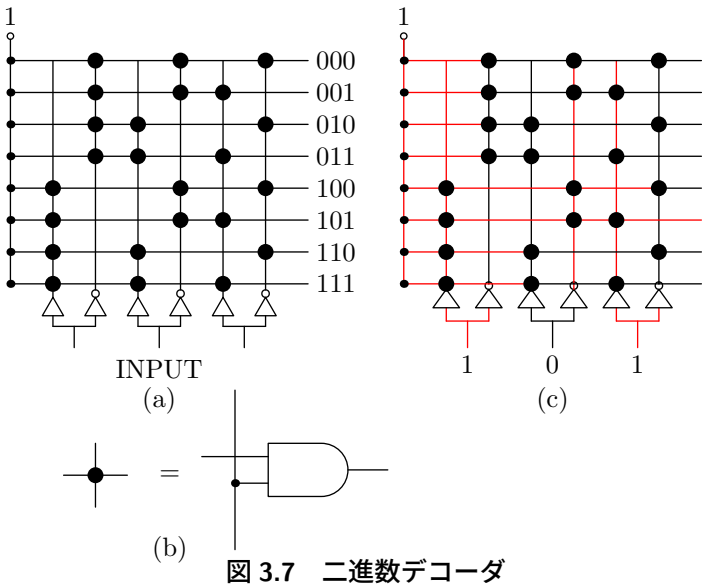



**図 3.7　二進数デコーダ**

例として (c) のように 101 のデータを入力すると赤の縦線に電圧がかかる．元々 横線には電圧がかかっているので 6 番目の線にのみ電圧がかかる．

- エンコーダ：逆に選ばれたワイヤーのデータを真理変数の組（二進数）に変換するには，図 3.8 のような一本のワイヤーを選ぶ装置を使えば良い．ただし，白抜きの四角は (b) のように OR を使った回路とする．例として (c) のように 7 番目の線に電圧をかけると 110 を取り出すことが出来る．

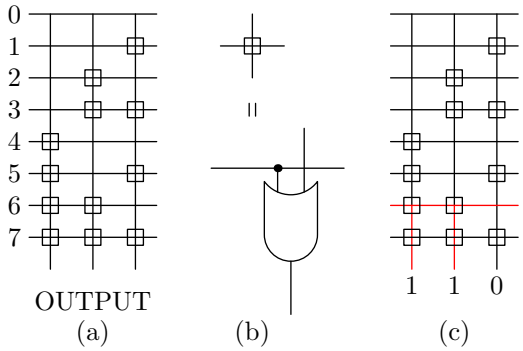



**図 3.8　二進数エンコーダ**

- 一般の論理回路：一般の論理回路は, $0, 1$ をとる $m$ 個の変数 $X_i$ $(i = 1, \ldots, m)$ に $0, 1$ をとる $n$ 個の変数 $Y_j$ $(j = 1, \ldots, n)$ を返す関数の組と考えられるので

$$Y_j = F_j(X_1, \cdots, X_i, \cdots, X_m) \quad j = 1, \ldots, n$$

と表すことが出来る．$\mathbf{X} = (X_1, \cdots, X_m)$, $\mathbf{Y} = (Y_1, \cdots, Y_n)$, $\mathbf{F} = (F_1, \cdots, F_m)$ と置く．$V^m = \{0,1\}^m$ とすると，$\mathbf{X} \in V^m$, $\mathbf{Y} \in V^n$ なので，$\mathbf{F}$ は写像

$$\mathbf{F} : V^m \to V^n$$

とみなせる．これは $m$ バイトの二進数の集合とも見なせる（し，$m$ 次元超立方体 $[0,1] \times \cdots \times [0,1]$ の頂点の集合とも見なせる (図 3.9 )）．つまり，$\mathbf{F}$ は $m$ バイトの二進数を $n$ バイトの二進数に対応させる任意の写像とみなせる．

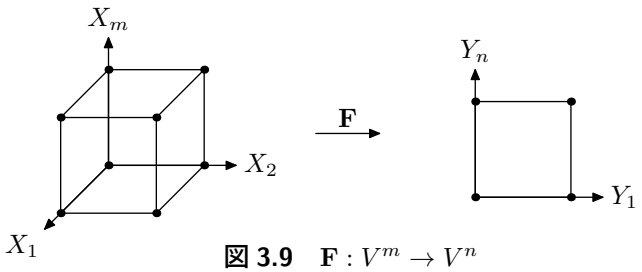



**図 3.9　$\mathbf{F} : V^m \to V^n$**

この視点に立つと，二進数デコーダと二進数エンコーダを使ってこの写像 $\mathbf{F}$ を図 3.10 のように構成できることがわかる．特に AND と OR, NOT だけで構成されることがわかる．ただし，これは $\mathbf{F}$ を実現する論理回路の存在を示したに過ぎず，実際の回路の設計ではいかに効率よく資源を配分するかが大切になる．

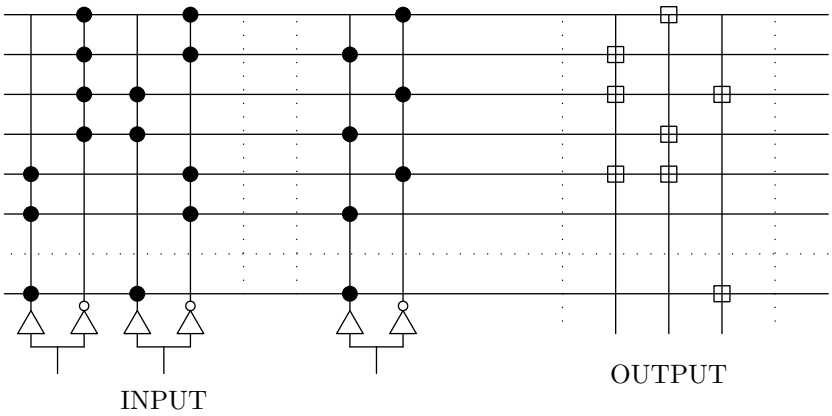



**図 3.10　任意の論理回路の実現**

## 3.2　フリップフロップ回路とメモリ

- フリップフロップ回路（双安定バイブレーターとも言う）とは図 3.11 のような回路である．$\bar{Q}$ は $Q$ の否定，$S$ はセット，$R$ はリセットで同時に真になることはない変数とする．

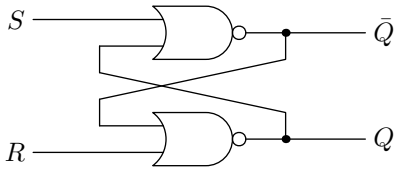



**図 3.11　フリップフロップ**

| $S$ | $R$ | $Q$ | $\bar{Q}$ |
|---|---|---|---|
| 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |

**表 3.9　フリップフロップ**

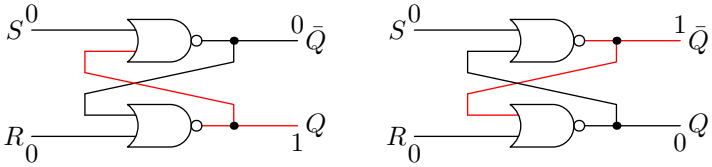



**図 3.12　$S = 0, R = 0$ の安定性**

- この回路の応答を調べる．図 3.12 から分かるように $R = S = 0$ のとき $Q, \bar{Q}$ の状態を保つ，ここでは左右二つの図を描いたが上下を入れ換えれば同じ図なので以後一方のみ書くことにする．図 3.13 から分かるように, 右の図では回路は安定だが，$S = 1$ のときは左の図の状態は不安定で安定な右の図の $Q = 1$ に遷移することが分かる．
- 状態が移るとき時間がかかるので，それを考慮して図 3.14 のような時計パルス波 $\phi$ で同期することを考える，ただし $t$ は時間である．$\phi$ に NOT したものを $\bar{\phi}$ と呼ぶ．このパルス波を生成するには図 3.15 のような回路を考えれば良いことが知られている．抵抗 $R$ キャパシティ $C$ のとき $0.7CR$ 秒間の周期でパルス波を生成する．この回路も双 (非安定) バイブレーターと呼ばれている．

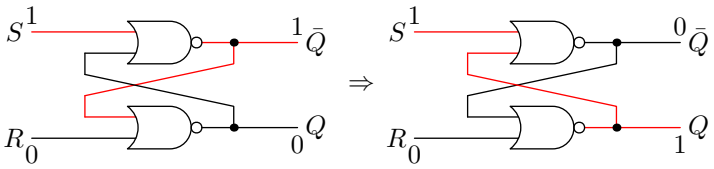



**図 3.13**　$S = 1 \Rightarrow Q = 1$

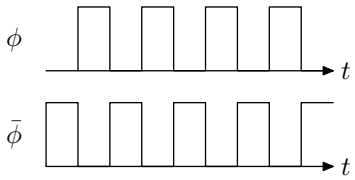



**図 3.14**　時計パルス

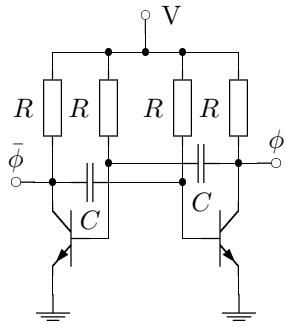



**図 3.15**　パルス波生成回路

- 図 3.16 のように AND 回路を使ってフリップフロップ回路にパルス波を加えると時計付きフリップフロップ回路が得られる．以下右の略記号で表す．

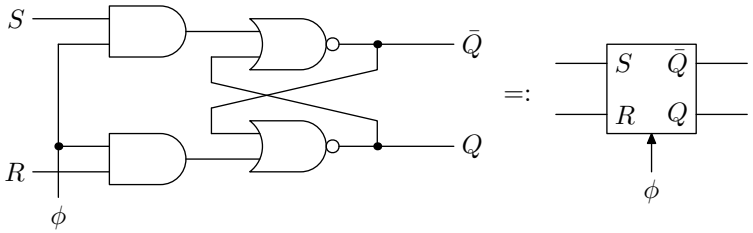



**図 3.16**　時計付きフリップフロップ回路

- この回路の応答を図 3.17 で調べる.

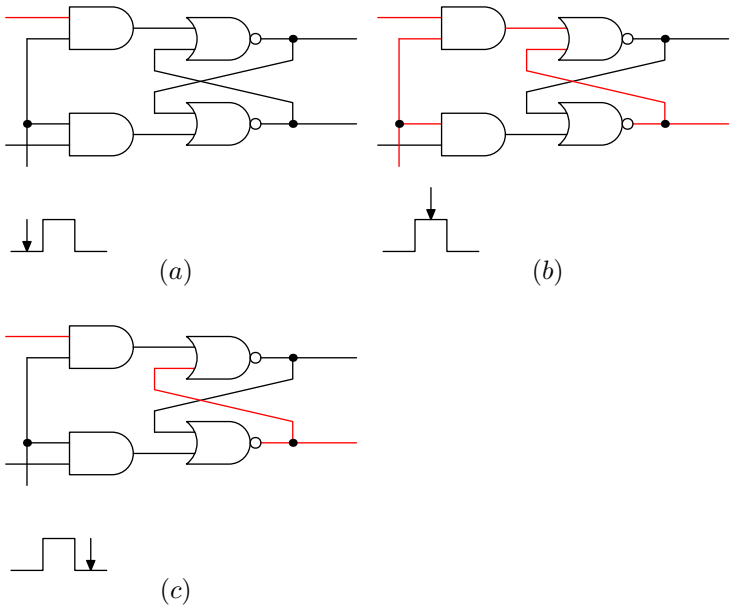



**図 3.17　時計と回路の動き**

- 時計付きフリップフロップ回路を二つ並べてパルス波で同期を取ると D フリップフロップ（DFF）が得られる.

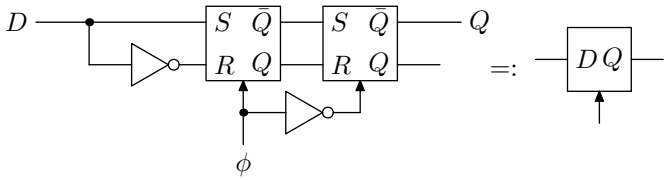



**図 3.18　D フリップフロップ**

- この動作を調べて見ると D の意味もわかる

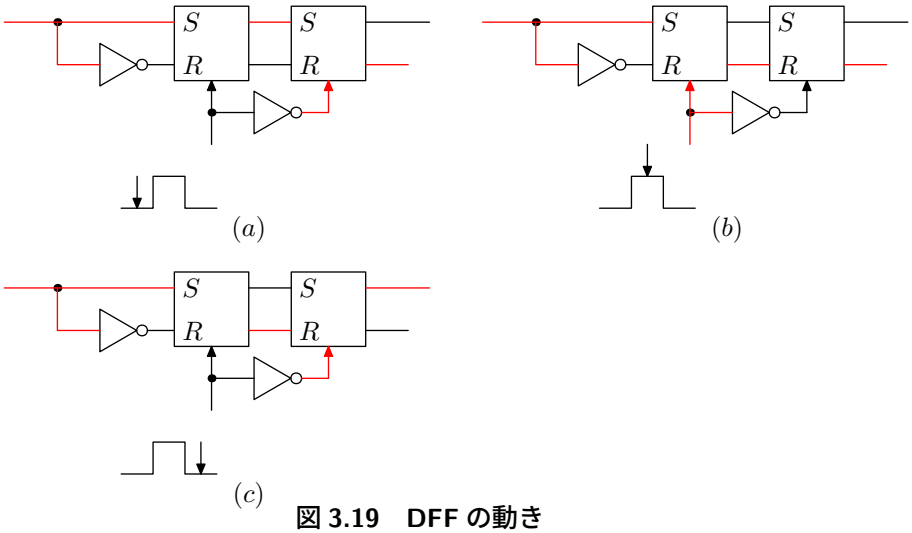



**図 3.19　DFF の動き**

- 次に DFF を幾つか並べるとメモリを構成できる．

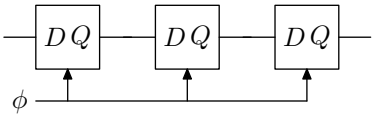



**図 3.20　DFF によるメモリ**

- この DFF メモリにデータを格納することをシュミレートしてみる．まず，全てが 0 になるように初期化する．次に入力する二進数を 101 とする．

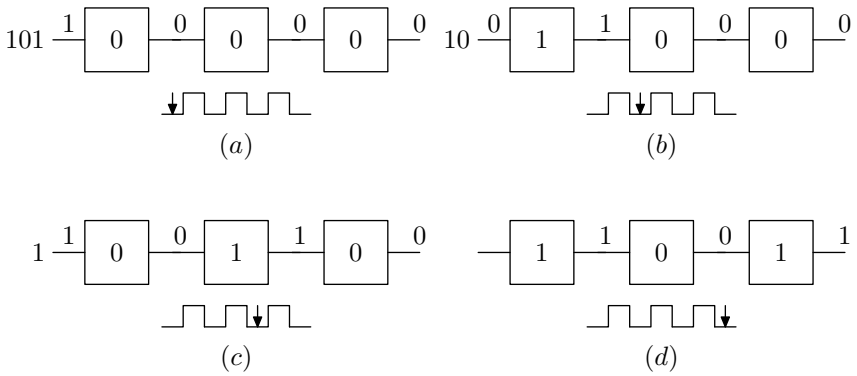



**図 3.21　D-FF によるメモリ格納**

**演習 18**　以下の回路でもフリップフロップ回路が実現できることを真偽表を書いて示せ．

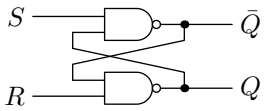



**図 3.22　フリップフロップ回路**

## 3.3　有限オートマトン

前回説明した論理回路を抽象化したのがオートマトンである (歴史的には多分オートマトンが先で論理回路が後であろう).

- まず例：0, 1 の列からなる入力の総和が偶数か奇数かを判定し順次そこまでが偶数か奇数を書き出すオートマトンは以下のように表される：

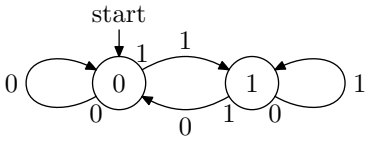



**図 3.23　偶奇判定機械**

○で囲まれているのを状態という．この場合 0 と 1 が状態で，それぞれその時点での偶数と奇数の状態を表している．start は入力を受け付ける前の状態でこの場合は 0 から始まることを表してる．矢印は入力が根元の数字のとき状態がどう移るかを表し，矢印の真ん中の部分の数字を出力するとする．この場合は 011010 のような入力を順次受け付けると，0 ならば状態を変えず元の状態を書き出し，1 ならば状態を替え異なる状態を書き出すことを表している．

- 次の例：二進法の入力にその 2 個前の入力を出力するオートマトンを設計してみる．前 2 個の入力を 00, 01, 10, 11 とする状態を考える．最初は入力が無いので 00 であると見なす．0 と 1 が入力される度に左にそれが付け加わり，右側の 0 か 1 を出力すれば良い．従って，以下のようなものを考えれば良い．

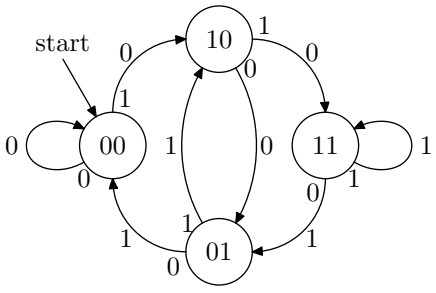



**図 3.24　2 遅延機械**

**演習 19**　二進法の入力にその 1 個前の入力を出力するオートマトンを設計せよ.

- オートマトンを数式で表してみる．$t = t_i = i \in \mathbf{N}$ を時間のパラメータとし，$Q = \{q_1, \ldots, q_L\}$ を状態の集合, $S = \{s_1, \ldots, s_M\}$ を入力の集合 $U = \{u_1, \ldots, u_N\}$ を出力の集合とする．すると入力は

$$s : \mathbf{N} \to S$$

と言う任意の関数で, 状態と出力の時間による変化は

$$q : \mathbf{N} \to Q, \quad u : \mathbf{N} \to U$$

という関数で表される.

$$\begin{aligned} F : Q \times S \to Q &\quad \text{：状態の遷移の規則を表す関数} \\ G : Q \times S \to U &\quad \text{：出力の規則を示す関数} \\ q(1) \in Q &\quad \text{：初期条件} \end{aligned}$$

がオートマトンであり，時系列

$$q(t_{i+1}) = F(q(t_i), s(t_i)), \quad u(t_{i+1}) = G(q(t_i), s(t_i))$$

が出力と状態遷移であることが分かる.

- このように定式化すると，この $F, G$ を持つオートマトンが，前回紹介したクロック周波数で同期したメモリ（これは上の演習の 1 遅延機械でもよい）と論理回路により表現できることが分かる：まず，$S, U, Q$ を二進法で表現しておき，$k$-ビット, $l$-ビット, $m$-ビットとし，その数のワイヤーでその値を表わしているとする．以下のようなクロック周波数 $\phi$ と $\bar{\phi}$ で同期された 2 つのメモリと $F, G$ を実現する論理回路を組み合わせることでオートマトンが実現される.

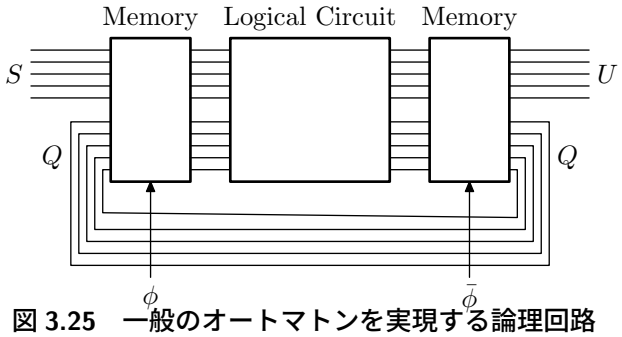



**図 3.25　一般のオートマトンを実現する論理回路**

## 3.4　チューリングマシン

- チューリングマシンとは，箱状のセルを一列に刻んだテープのある一個のセルにオートマトンが付いたもの (図 3.26) である．このオートマトンをヘッドと言ったりする．テープの各セル上には最初に有限個の記号が書かれていて，空白を 0 と表すとある所から右，ある所から左はすべて 0 になっているものである．時間は離散的な時計に従ってすすみ，最初に指し示すセルとヘッドの状態は予め定まっている．ある時刻のオートマトンの状態とそれの指しているセルに書かれている記号によってオートマトンは次の状態に移り，その箱に何かの記号を書き込み，右か左に一つだけ動き時間が進む．ある決まった状態に移ると機械は停止する．

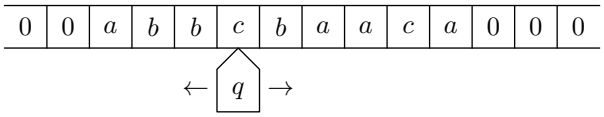



**図 3.26　チューリングマシン**

- チューリングマシンの例：単なるオートマトンとは異なることを先程と同様に例をもって示したい．問題は次のような ( と ) の有限個の列が与えられたとき，それが ( と ) の対応として整合性があるかないかを判定することである．もちろん整合性があれば ( と ) の数は等しくなるので，( と ) の数が等しくなければ整合性はないことがわかるがそれでは十分ではない．また，任意個数の ( と ) が表れるので有限状態のみでは判定できない．

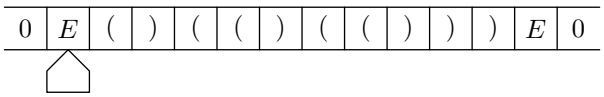



**図 3.27　初期状態**

まず，テープの初期状態は調べたい ( と ) の組み合わせを一個一個セルに入れて両端を $E$ ではさんでいるとする．そしてヘッドは左側の $E$ を指しているとする．図に書いた $R$ は右に，$L$ は左にヘッドが動くことを表する．左から ペアの最も近い ( と ) を見つけまず ) を関係ない記号 $X$ に書き換え左に動き ( を $X$ に書き換え右に動くことを実現するオートマトンは図 3.28 のようになる．この入力だとまず一つ右に動き ( は無視してもう一つ右に動くと ) があるのでそれを $X$ に書き換え左に動き ( があるのでこれを $X$ に書き換え右に動く．

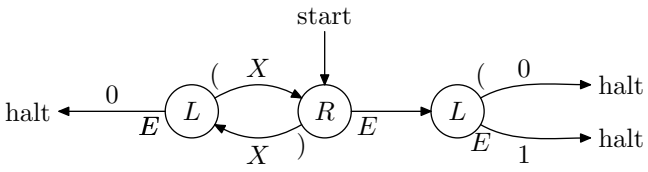



**図 3.28　括弧整合性判定機械のヘッド**

終了の仕方について：まず ) を $X$ に書き換えた後左にすすんで何も見つからず $E$ に到達した場合，整合性がないので 0 を出力して停止する．そうでなければ，対になる ( が全て見つかり，これを続けると右の $E$ に到達することになる．そのあと左に移動して ( が残ってれば整合性がないので 0 を出力して停止する．( が残っていなければ $E$ に到達し整合性が確認され 1 を出力して停止する．

- オートマトンとリューリングマシンの違いはオートマトンの所で述べた $F : Q \times S \to Q, G : Q \times S \to U$ の他にテープの移動方向を与える関数

$$D : Q \times S \to \{0, 1\}$$

を指定することである．0 を左，1 を右と決めておく．

- インデックスの付いたファイルを検索して，その内容を適当な場所にコピーするチューリングマシンを作成する．

まず，テープのデータの格納状況を説明する．簡単のためインデックスとファイルの内容は共に 3 ビットの 2 進数で $X$ と $X$(または $Y$) の間の前半 3 ビットがインデックスで，後半 3 ビットがファイルの内容である (図 3.29)．左の $Y$ がファイルの先頭で右の $Y$ がファイルの終を表す．左側の端の $Y$ と最も左の $X$ との間の 3 ビットが探したいファイルのインデックスであり，オートマトンは最初に最も左の $X$ を指した状態にあるとする．

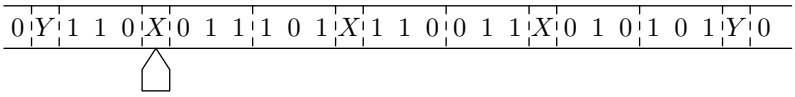



**図 3.29　初期状態**

オートマトンは次の図 3.30 のように設計される：
まずヘッドは左に進みながら数字の 0 を $A$ へ 1 を $B$ へ書き換え，$Y$ に到達すると右に進み始める．まず最初の $A$ を 0 へ $B$ を 1 へ書き換え，次の $A, B$ や $X$ を無視して最初の数字まで右に進む．元の数字が 0 のときその数字が 1，元の数字が 1 のときその数字が 0 ならばインデクスは一致していないので，次の右の $X$ まで進み，そこから左に進みながら 0 を $A$ へ 1 を $B$ へ書き換え $Y$ まで進み，次のインデックスの比較

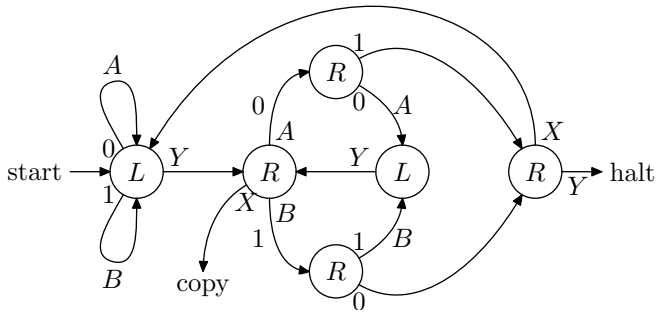



**図 3.30　ファイル探索機械**

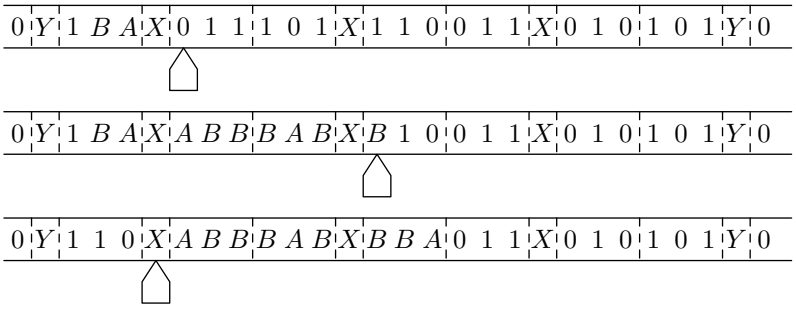



**図 3.31　探索中のテープの変遷**

に移る．元の数字が 0 のとき最初の数字が 0，元の数字が 1 のときその数字が 1 ならば，そこまではインデクスが一致しているので，その 0 を $A$ へ 1 を $B$ へ書き換えもう一度 $Y$ まで左に戻って，つぎの桁の数字の比較を行う．これを続け，完全に一致したとき，ヘッドは最も左の $X$ まで動く．もし途中で一致しない場合は次の右の $X$ まで進み，そこから左に進みながら 0 を $A$ へ 1 を $B$ へ書き換え $Y$ まで進み，次の (上の $X$ の左の) インデックスとの比較に移る．

次にそのインデックスのデータを $Y$ と最も左の $X$ の間の部分にコピーする以下のようなオートマトンを考える：

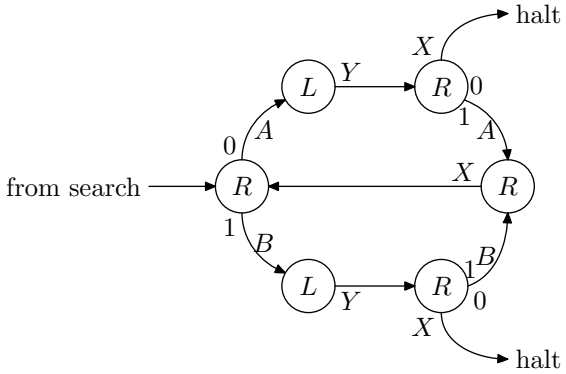



**図 3.32　コピー機械**

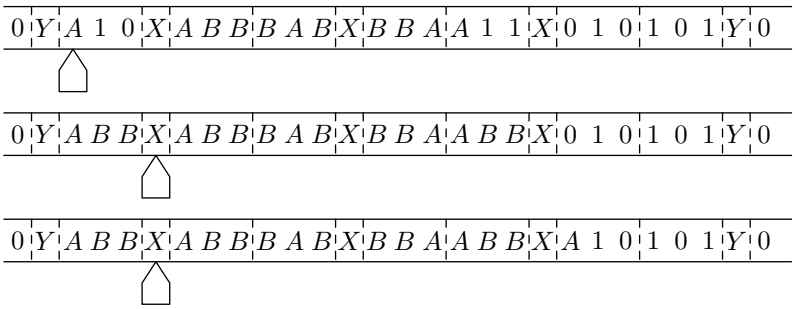



**図 3.33　コピー中のテープの変遷**

検索が終わった後最も左の $X$ とデータの間は全て $A, B$ に置き換わっているので，右側の最初の数がデータの数字である．そこで $X$ から右側に進み最初の数が 0 のとき $A$ へ 1 のとき $B$ へ書き換え左へ進み $Y$ に戻る．$Y$ から右に進み上の最初の数を元の数が 0 のとき $A$ へ 1 のとき $B$ へ書き換え，最も左の $X$ まで右に進み，次の桁の数まで右に進み，次の桁の数をコピーしてそれを繰り返し終るまで続ける．ただし，このチューリングマシンでは最後は意味の無い次のファイルの最初の数をコピーして，$Y$ と $X$ の間にもうコピーする場所が無いので最も左の $X$ で停止することになる．

## 3.5　ユニバーサルチューリングマシン

全てのチューリングマシンの動作を完全にシミュレートできるチューリングマシンをユニバーサルチューリングマシンという．$T$ がシミュレートされるチューリングマシンとし，$U$ をユニバーサルチューリングマシンとする．

簡単のため $T$ の状態は 2 ビットとし，$T$ のテープが $Y$ の左に再現されている．そこのセルには $0, 1$ のみが書かれてるとするが現在作業中のセルに $M$ という記号が書かれているとする．

最も左の $X$ の後にあるのは遷移状態のリストである：状態とセルの値との組で 3 ビット，次の状態の 2 ビットと書き込む値が 1 ビット，ヘッドの移動向きを示す 1 ビットの組であり，それら全てを最も左の $X$ から左に 7 ビットずつ $X$ で分けて書いておき，最後に右の $Y$ で終わるようにする．左側の端の $Y$ と最も左の $X$ との間の 3 ビットが $T$ の状態とテープに書き込む数字を表している．

図 3.35 にこのユニバーサルチューリングマシンのヘッドの構造を示す ([43])．上の部分はファイル探索機械と同じであるが，これはインデクスを $T$ の状態とテープから読み込まれた数の組として，次の遷移状態，つまり，$T$ の次の状態とテープへ書き込む数とヘッドの動く方向を探していることになる．次の右の中部の部分はコピー機械と同じである．

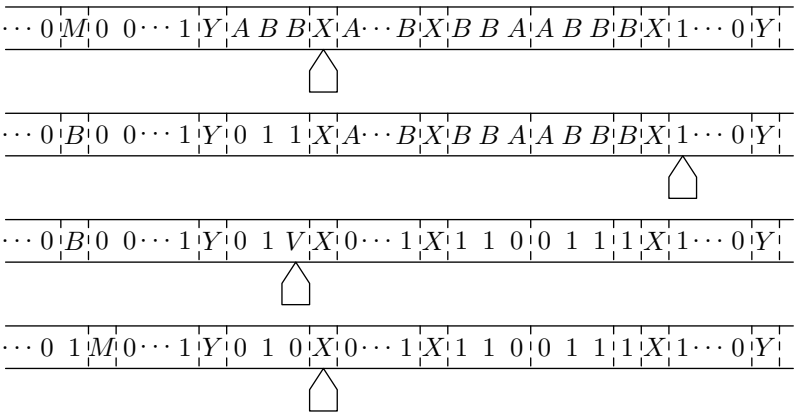



**図 3.34**　$T$ のセルへの値の書き込みと移動と読み込み

ただし最後にコピーした数字はヘッドの動く方向を示すビットでこの場合は右を表している．

右の中部の下の部分では $X$ から $M$ まで左に進み，まずヘッドの移動方向をそこに書き込む．この場合は $B$ である．次に右に進み $A$ を $0$, $B$ を $1$ に置き換えながら最も左の $X$ まで進む．

その $X$ の左の $A$ と $B$ をもう一度 $0, 1$ に戻すために，最も右の数まで進み $A$ を $0$ に $B$ を $1$ に書き換えながら最も左の $X$ まで左に進む．もう一つ左にすすみ最初の数字を読み込み, 記号 $V$ に置き換え，$T$ のテープの $M$ の有ったところまで左に進む．もし，そこに $A$ があれば，そこに先程読み込んだ数字を書き込み左に動く．$B$ があれば，そこに先程読み込んだ数字を書き込み右に動く．そしてヘッドの動いた先の数字を読み込み，最も左の $X$ まで進み一つ左に進み $V$ の書かれたセルに読み込んだ数字を書き込み，最後に右に一つ $X$ まで戻る．

これで一サイクルが終わり次のサイクルが始まる．$T$ が停止するまで続いていく．

このオートマトンは記号 $8$ 個，状態 $24$ 個からなっている．

**演習 20**　このユニバーサルチューリングマシンがちゃんと動くか確認して下さい．

最後にチューリングについては伝記とその映画化 ([67]) がある．数学も凄いが人生も凄い人なので興味のある人には鑑賞してもらいたい．

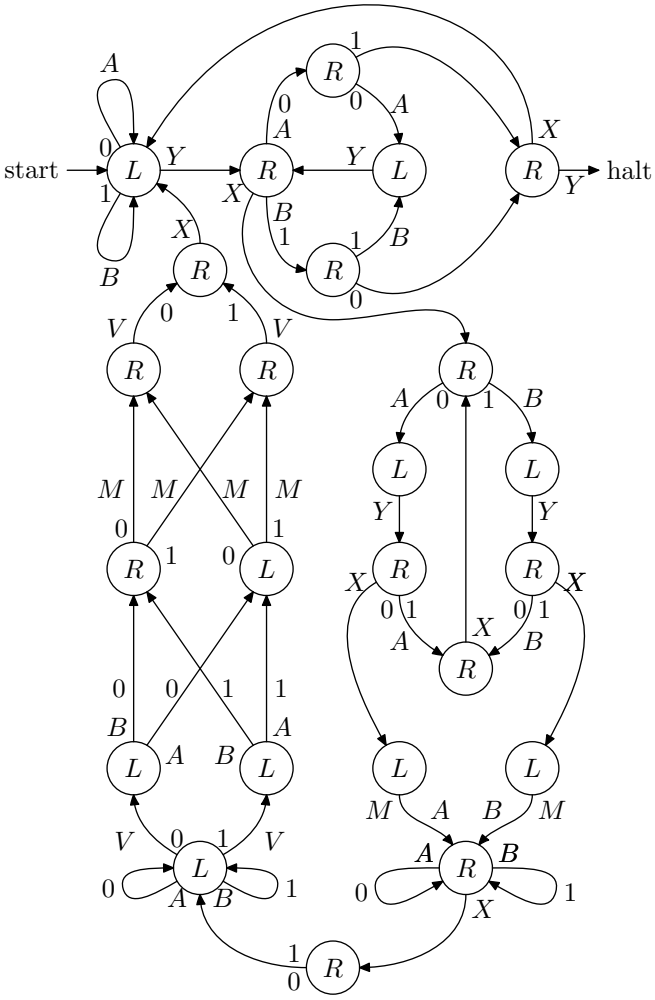



**図 3.35　Minsky のユニバーサルチューリングマシン**

# 第 4 章　インターネット

ネット net とは網 (あみ) のことであり，network とは網全体，例えば，駅と線路と列車の全体，テレビ局の全国放送網のことをさすが，インターネット The Internet はコンピュータの全世界的な network のことである．

## 4.1　インターネットの仕組み

### 4.1.1　コミュニケーションとメディアの歴史

昔から狼煙・太鼓を使ったり，手紙によるコミュニケーションや徒歩や馬による移動などが行われていた．しかし，近代になって重要になった人の移動や人口の変化，貨幣の循環を含むコミュニケーションやメディアの進展には以下のようなものがある (世界史の復習に一冊あげるなら [75])：

- 13〜15 世紀のモンゴル帝国がユーラシア大陸に広がりと交易が発展した．
- それがきっかけとなり 14 世紀に黒死病 (ペスト) がヨーロッパに広がり，ヨーロッパでは 3 人に 1 人が死んだと言われている．
- 東ローマ帝国が 1453 年にオスマン帝国に滅ぼされ多くの文化人が西洋に亡命し，その人達の活動とイスラムとの接触を通してギリシャ・ローマの文化が復活しルネッサンスが始まるきっかけになった．
- 1450 年代グーテンベルグによる活版印刷の開始．
- モンゴル帝国などから伝わった航海術が十字軍などを通して発展し，それを使った 1492 年[1)]のコロンブスによる新大陸の（ヨーロッパ人から見た）発見．それに続くヨーロッパからの南北アメリカ大陸への移民の流入．
- これ以降ヨーロッパから持ち込まれた伝染病で新大陸の多くの現地人[2)]が死亡した．それによるプランテーションでの労働力不足のために，17〜19 世紀に奴隷貿易が行われ，アフリカの多くの黒人が奴隷としてアメリカ大陸に移住させられた．
- 1545 年の南米のポトシ銀山の発見のあとヨーロッパへ銀が大量に流入し貨幣供給量

1)　この年がスペインによるレコンキスタの完成の年であるのは偶然では無い．
2)　一説には 10 人中 9 人

が大幅に増加した．これが価格革命・商業革命をもたらした．
- 会社組織・簿記・決算書・銀行の発展．株式・債券・外国為替など様々な市場を含めた資本主義の発展．
- 新聞・雑誌等の発行によるマスコミの発達．
- 石炭を使った (第一次) 産業革命．
- 蒸気機関車・自動車・道路・飛行機などの交通の発展．
- 写真・レコードの発明．
- 石油やガスの使用の増加[3]．電 (磁) 気が発見され，十九世紀末には送電網が整備された (第二次産業革命)．以下の発明も電気の基盤があってのものである．
- モールス S. Morse による電信の発明．
- ベル G. Bell による電話の発明．
- マルコーニ G. Marconi による無線の発明．
- 映画・ラジオ・テレビ放送の発展．
- コンピュータの発明と普及．
- パソコンの普及．
- 携帯電話 (スマートホン) の発展．
- インターネットの発展．

### 4.1.2　インターネットの歴史

1960 年代に米国国防総省の ARPA(=Advanced Reseach Projects Agency, 国防高等研究計画局) で ARPANET の計画が始まったが，次の二つの見方がある．

1. 1960 年代後半に，米国が核攻撃を受けてもグラフ状のネットの壊れた部分をパケット が自動的に迂回して繋がるような仕組みの ARPANET の計画が始まった．
2. 色々な種類のネット技術が分散している現実の世界でも，お互いに問題なく繋げられるネットワークを作りたいという理想に燃えた当時の技術者が internet の技術を開発した．

ここではこれについての明言は避け[4]，1969 年に Berkley 大学のコンピュータから Stanford 大学のコンピュータへ送信されたのがその最初の実験であったことのみ言及しておく．その後 1990 年代に商用に広がりインターネット (the Internet) と呼ばれるようになった．

---

3)　今回のイラン戦争で化学肥料も石油に依存していることが明らかになった．
4)　私のポートフォリオに AI の見解や資料がある．

ただし，インターネットというネットが一つあるわけではなく，家や会社や大学などの LAN =Local Area Network と大きな領域を繋ぐ WAN=Wide Area Network の緩やかな塊がインターネットである．物理的なネットの仕組みは色々バラエティーがあるが，共通の決まりでパケットを送合い，受けとるようになっている．

1 秒間に何れだけの情報が送れるかというのを計る単位として bps =bit per second が使われる．通信容量ともいいい通信の性能を測るため後で出てくる．

### 4.1.3 LAN や携帯電話

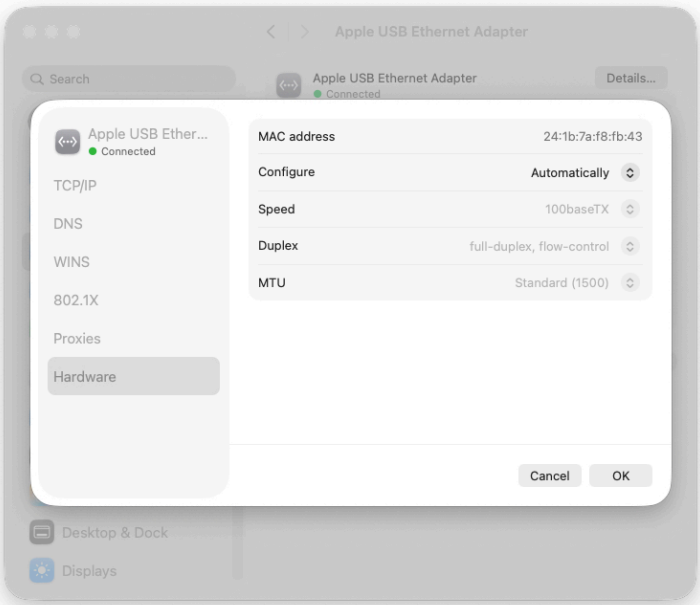



図 4.1　MAC アドレス

図 4.2　ハブ

- LAN の具体的なものとして Ethernet[5)]がある．まず，全ての Ethernet ボードに MAC アドレス (Media Access Control address) という一意的な 12 ケタの 16 進数が与えられ

5)　Ether とは相対性理論で否定されたエーテルのことである．もっとも否定されたのは古典論に従う Ether であって相対性理論的 Ether は存在していても問題は無い．これも私のポートフォリオに AI の見解や資料がある．

ている．私の mac の外付け Ether ボードだと (図 4.1)

24:1b:7a:f8:fb:43

になっている．

あるエリアのパソコンやデバイスはイーサーケーブルとハブで結びついていて，エリア内のパソコンたちはそれを使って通信する．そのエリア内からルーターを通して外の WAN に繋がっている．基本的な通信の基本はパケット通信で模式で表すと図 4.3 のようなビット列である．ここでのアドレスとは MAC アドレスのことである．初期のイーサネットの容量は 3Mbps 程度だったが最近では 100Gbps を越えるものも多い．

| 送信元アドレス | 送信先アドレス | データ長 | データ | エラーチェック |
|---|---|---|---|---|

**図 4.3　イーサーネットのパケットの構造**

- LAN を無線で繋がるようにした無線 LAN を使うことも多い．IEEE 802.11 のような規格で呼ばれるが，最近は Wi-Fi ということが多い．2.4〜2.5GHz, 5GHz 以上の周波数で動作する．無線 LAN は誰でも盗聴できるので WPA2/WPA3 のような暗号化を用いることが多い．

**演習 21**　自分の使っている Ether 器機 (例えば Wi-Fi) の Mac アドレスを調べて下さい．近くの友達と比べて確かに異なることを確かめて下さい．

- BlueTooth なども 2.4GHz の周波数でデバイスとパソコンの通信を行っている．
- 携帯電話は英語で Cellular Phone というが，Cell というのは平面上にある基地局を中心に平面を分割したものの一つ一つのことである (図 4.4)．使用可能な周波数帯を隣りの局で混信しないように配置するが，ある程度遠くの局では混信しないので同じ周波数帯を再利用していい．更にその周波数帯を細かく分割して全ての携帯電話と基地局は連絡を取り合っている．携帯電話には国際移動体装置識別番号という 15 桁の識別番号が割り当てられていて，電源が入ると ID がブロードキャストされ携帯電話がいるセルにある最寄りの基地局でその情報を把握して，電話会社が把握し，必要に応じて電話回線やインターネットと繋ぐように運用されている．携帯が動くに伴い基地局も変更される．無線 LAN がある所では安い方を使うようになっている．

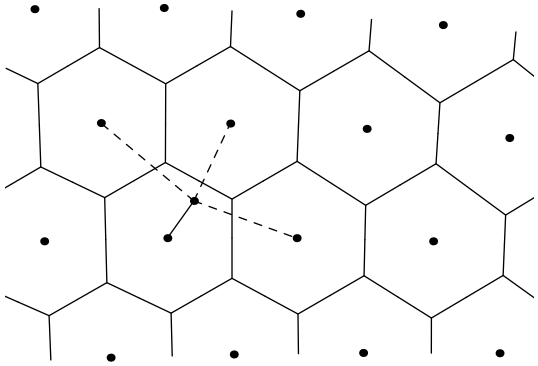


図 4.4　携帯電話基地と Cell と携帯電話

| 世代 | コメント | 速度 | 年 |
|---|---|---|---|
| 1G | アナログ通信 | | 1980 年代 終了 |
| 2G | デジタル通信 | 最大 64kbps | 1990 年代 終了 |
| 3G | 国際的な規格 | 最大 3.6Mbps | 2000 年代 最近終了 |
| 4G | 現在主流 | 最大 1Gbps | 2010 年代 |
| 5G | 次世代 | 最大 10Gbps | 現在 |

表 4.1　携帯電話の通信方法の変遷

### 4.1.4　TCP/IP

- プロトコル (protocol) とはもともと外交における議定書のことであるが，コンピュータのネットワークでは全体で守る規約のことである．特にインターネットでは TCP/IP という規約で代表されるものである．これもパケット転送技術である．その構造を図で表すと図 4.5 のようになる．

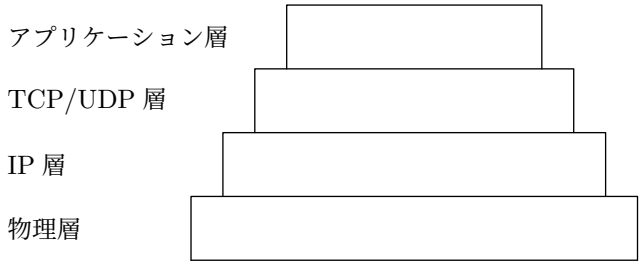



図 4.5　パケットの階層

物理層とは前のセクションで紹介したような LAN などの通信の最も下層のレーヤーのことである．上の層が下の層のデータの中に埋め込まれているイメージで捉えてほしい．

- IP address(Internet Protocol address) は数字の並びで表した各コンピュータの名前である．IPV4 では . で区切られた 8bit の 10 進数 (0〜255)4 個で表されている．よって，$2^{32} \approx 43$ 億ほどのアドレスしかない．私のマックだと (図 4.6)

  192.168.1.4

  になっている．

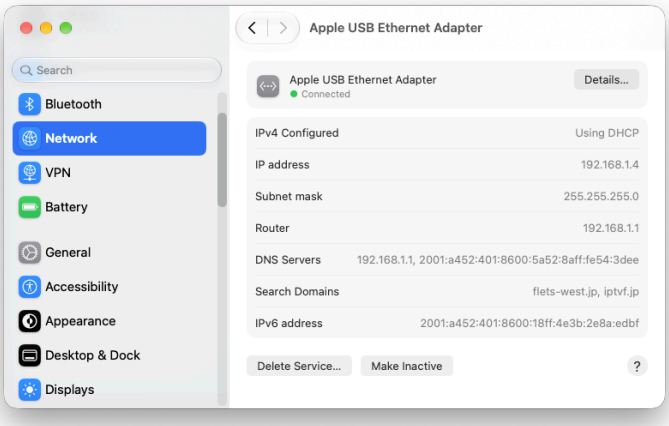



**図 4.6　IP アドレス**

当初はコンピューター一台毎に Global な IP アドレスを配布する予定であったが，私が使っているのは Global な IP アドレスではなく，プライベートアドレスで我々のローカルエリアでのみ使える (正確には同じアドレスが使い回されているが，そのローカルエリア以外からはアクセスできない) アドレスになっている．しかも，DHCP(Dynamic Host Configuration Protocol ) という方法でアドレスを配っていて毎回使うアドレスは異なっている．通信のためには自分の IP アドレスと送りたいコンピュータの IP アドレスの組が必要になることにも注意する．

IPV4 を拡張した IPV6 では 16 バイトの 16 進数を 2 バイトづつ: で区切って書く．例えば

2001:0db8:bd05:01d2:288a:1fc0:0001:10ee

これだと $2^{128}$ 個 (日本語での十進法での単位が不明なので書かない) で十分な数がある．

いずれにしろ人間は大きい数字を扱うのは得意でないので youtube.com のような名前でアドレスを表すことが多い．これをドメインネームという．ドメインネームを

IP アドレスに変換するサービスを行うのが DNS(Domain Name System) である．実際に仕事を行っているサーバーを DNS Server という．

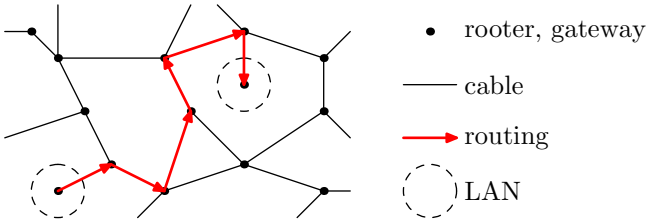



**図 4.7　ルーティング**

ルーター・ゲートウェイを通じてパケットは受け渡されてゆき最終的なターゲットのコンピューターに渡される．その順番がルーティングである．ルーターやゲートウェイは頻繁に他のルーターやゲートウェイを通るとどれ位時間が掛かるかのリストのようなものを常に更新している．それを元に次に何処に送れば最適かを判断してパケットを転送している．

- IP パケットは

| 基本情報 | TTL | 送信元 IP アドレス | 送信先 IP アドレス | エラーチェック | データ |
|---|---|---|---|---|---|

**図 4.8　IP のパケットの構造**

のようなものである．基本情報には version とかヘッダや全体の長さの情報が入っている．TTL は Time to live(生存時間) の略で 1byte の大きさを持ち，初期条件が 40 程度に設定されていて gateway を通過する度に 1 ずつ減っていき 0 になるとそのパケットは廃棄される．

- TCP(Transition Control Protocol) は図 4.5 に示したように IP の上に存在するプロトコルである．各コンピュータ毎にポート (Port) と呼ばれる 2byte, つまり $= 2^{16} \approx 64,000$ 程の数字 (port number) を指定できる．Port は日本語だと港[6)]のことだが，会社の電話番号が IP アドレスで Port は会社の内線番号のような役割をはたしている．つまり一台のサーバーがこれを使って 6.4 万程度のコンピューターと同時に接続可能である．

  通信には送り手の IP アドレスとポートナンバーと受取り手の IP アドレスとポートナンバーの組が必要になることに注意してほしい．また，送りたいパケットが長いときは適宜分割して，その分割の何番目かを示すシークエンス番号が付いている．

6)　語学に興味のある人へ：port は港から派生して昔の戦艦の砲門のふたことを表すようになった．それから工学では外部器機やケーブルを接続するための穴のことを意味するようになった．

| 送信元ポート | 送信先ポート | シーケンスナンバー | データ | エラーチェック |
|---|---|---|---|---|

**図 4.9　TCP のパケットの構造**

UDP(User Datagram Protocol) という TCP と同じレベルのプロトコルもあり，双方向のストリームを必要としないデータ交換に使われている．

- 図 4.5 に示したように TCP/UDP の上にのるのがアプリケーション層 (Application layer) である．FTP, telnet, ssh, SMTP, POP3, HTTP, HTTPS などのようにインターネットで使うアプリケーションに対応したプロトコルである．例えばメールだと SMTP, POP3 のようなプロトコルを使っている．FTP はデータを転送するためのプロトコルであり，HTTP はウェブを実現するプロトコルである (HTTP については次回詳しく解説する)．

特に最初の通信を確立するために TCP/UDP のポートの規定値として使うのが Wellknown port である．その表を以下に挙げる：

| Port No. | | プロトコル |
|---|---|---|
| 20 | TCP | FTP |
| 22 | TCP/UDP | ssh |
| 23 | TCP | telnet |
| 25 | TCP/UDP | SMTP |
| 52 | TCP/UDP | DNS |
| 80 | TCP/UDP | HTTP |
| 110 | TCP | POP3 |
| 443 | TCP/UDP | HTTPS |

**表 4.2　Wellknown port の例**

通常あるプロトコルの通信が確保されると well-known port 以外の適当なポートを指定し直して，それを使って以後の通信が行われる．

### 4.1.5　Utilities

UNIX の端末には TCP/IP 用の様々なツールが用意されている：

- netstat
- ifconfig

- ping
- traceroute
- port scanner: 標準ではインストールされていないが Nmap などが使える．

## 4.2 Web の仕組み

### 4.2.1 Web の始まり

現在インターネットで最も使われているのが WWW(World Wide Web), 略して W3 とか Web と呼ばれるプロトコルである．1980 年に CERN (Centre Européen de Recherche Nucléaire) のティム・バーナーズ=リー ([41]) が個人的な開発を始め, 1989 年に「Information Management: A Proposal」という提案書をまとめた．そこに書かれているのが WWW である．

Web はもともと URL・HTTP・HTML というプロトコルの三つ組みから成っていた．

- URL は Uniquely Resource Locater の略で情報の位置を示すもので基本構造は以下のようである：

  https://info.cern.ch/hypertext/WWW/TheProject.html

  最初の https がスキームという接続方式を示す所で，次の info.cern.ch[7] がサーバー名，残りがパスである．因みに上の URL は CERN に公表されている最初の WWW のページのアーカイブの URL(へのリンク) である．
- HTTP は Hyper Text Transfer Protocol の略で WWW の情報の受け渡し方法を決めている．後でもう少し詳しく述べる．スキームは (この時点では https は存在せず) http のほか ftp, file なども柔軟に選ぶことが出来た．
- HTML は Hyper Text Markup Language の略で情報の表現方法を決めている．SGML という既に存在したマークアップ言語 (mark up とは印刷の用語で，原稿の余白に印刷に必要なフォントやその大きさ，割り付けなどを書き込むこと) を拡張したページの記述のための言語である．ハイパーテキスト (Hyper Text) の実装を含んでいた．

  Hyper Text 自体の公表はテッドネルソンのザナドゥなどが先であったがインターネットでの実装は HTML のほうが早かった．

WWW の実装にはインターネット上に情報を持っているサーバー (Server) とそれをローカルで表示するクライアント (Client) としてのブラウザー (Browser) が必要だっ

7) ch は Switzerland のドメイン名

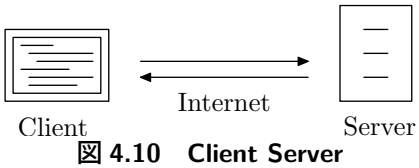



**図 4.10　Client Server**

たが，ティム・バーナーズ=リーはそのようなサーバー (CERN httpd) とブラウザー (WorldWideWeb) も公表した．また初期でも画像も Hyper Text を用いて表示することが出来た．最後に，彼は Web の著作権を全て放棄しているが，それも WWW の成功の一因であると言われている．

### 4.2.2　Web の発展

手元にある短い index.html という html のソースファイルを見てみる：

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" width=device-width,
       initial-scale=1.0, maximum-scale=1.0, user-scalable=0>
    <style> body {padding: 0; margin: 0;} </style>
    <script src="p5.js"></script>
    <!-- <script src="p5.sound.min.js"></script> -->
    <script src="sketch.js"></script>
  </head>
  <body>
    <a href="https://p5js.org/">p5.js</a>
  </body>
</html>
```

基本<hoge> と</hoge>で要素を囲む形で構造を示しているのが分かる．このようなものをタグ (tag) という．最初の行がこのファイルが html であることを示し，<html>と</html>で囲まれた部分が html である．それが<head>と</head>に囲まれたヘッダ部分と，<body>と</body>に囲まれた本文に分かれている．ヘッダ部分には文字コード，スタイルや<script>と</script>に囲まれた部分に JavaScript の (そのディレクトリーに置かれた) ファイルが指定されている．また本文の<a>と</a>には (href の後に) リンクが指定されている，ただし，a は anchor の略である．ツリー構造が分かるように適当にインデン

トが入れてあることも分かると思う．また，index.html というのはそのディレクトリにアクセスするとき最初に読み込まれるファイルの標準的な名前である．

**演習 22**　以前紹介した markdown のファイル IntroToMarkdown.md のあるディレクトリーで

```
% pandoc -f markdown -t html -s -o IntroMD.html
      IntroToMarkdown.md
% open IntroMD.html
```

とすると html に変換されたページがブラウザーに表示されるので，確かめてください (300 行ほどのファイルでそのうち 200 行以上が後で紹介する CSS の部分である)．ただし，% はプロンプト．

**演習 23**　上の CERN へのリンクをクリックして最初の WWW のページを訪れてください．Chrome だと View のメニュー中の Developer のサブメニューのなかの View source を選ぶとソースファイルを見ることが出来るのでやってみてください．他のサブメニューのアイテムも選んでみて何が出来るかためしてみてください．

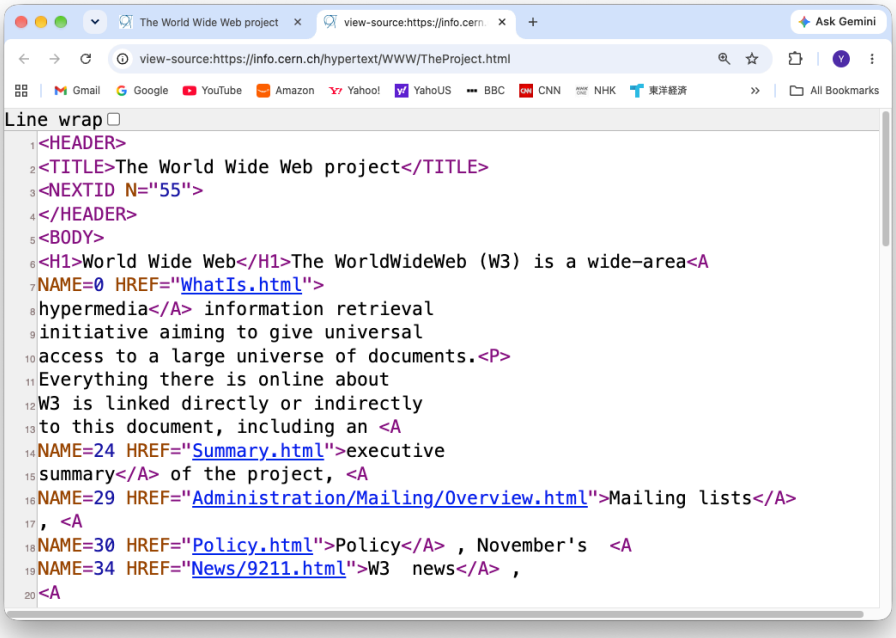



**図 4.11　世界初の WWW のホームページのソース**

**演習 24**　一般のサイト, 例えば yahoo のホームページのソースファイルを上と同じ方法で確認してください．見たら分かると思いますが，最近の Web サイトのソースファイルはサーバーのコンピュータが自動生成したものが多くなっています．

WWW は広く受け入れられていったが，直ぐに様々な拡張が成されていった．

- 初期の頃 HTML には見た目に関する部分も含まれていたが，その部分を独立して分離したのが CSS(Cascaded Style Sheet) である．美術的なセンスが必要なところなので難しい所もあるが，簡易的に使えるものも色々出回っている．
- HTML に動きを与える目的で作られたプログラミング言語が JavaScript である．ネットスケープコミュニケーションズの Brendan Eich によって 1995 年に僅か 10 日間で開発された．1995 年は Windows95 が発売された年で，そこに IE=Internet Explorer がバンドルされて発売されたので，ネットスケープとしては IE と差異化することを狙って開発された．最初は Mocha, 次に LiveScript と呼ばれたが，当時業務提携していたサンマイクロシステムズの当時注目されていたプログラミング言語 Java にあやかって JavaScript と名前を替えた．ただし，Java とは直接関係のない言語であり，現在でも誤解を生み続けているプログラミング言語での負の遺産の一つである．
- HTTPS は暗号化など HTTP にセキュリティの強化を図ったプロトコルである．

**図 4.12**　クエリと返答

- クエリ (query)：amazon で「国債」を検索すると図 4.12 のような画面になる．このと

きブラウザーのアドレスバーに表示されている URL を抜き出すと

```
https://www.amazon.co.jp/s?k=%E5%9B%BD%E5%82%B5&__mk_ja_JP=
  %E3%82%AB%E3%82%BF%E3%82%AB%E3%83%8A&crid=2TUMVIKOHEXS5&
  sprefix=%E5%9B%BD%E5%82%B5%2Caps%2C215&ref=nb_sb_noss_1
```

この？以下に表れる部分がクエリである．このような URL を使って質問を送り，サーバーが質問に対する答えのホームページを返している．% で始まる 16 進数二つの並びが UFT-8 であり，パーセントエンコーディングという．確認してみると

```
% echo -n "国債"|od -An -tx1
e5  9b  bd  e5  82  b5
```

で = 以下の部分と一致していることがわかる．ただし，ブラウザーでの表示は図 4.12 のように人間が読みやすくなるように文字に変換されている．同様の方法で様々なデータのやりとりを行っている．

- クッキー (Cookie): Web のログイン情報や使用に関する情報を Web Browser の中に保存したもの．Chrome だと View のメニュー中の Developer のサブメニューのなかの Developer tools を選び，Application のタブのペインの中の Storage の Cookie を選ぶと保存された Cookie を確認出来る．

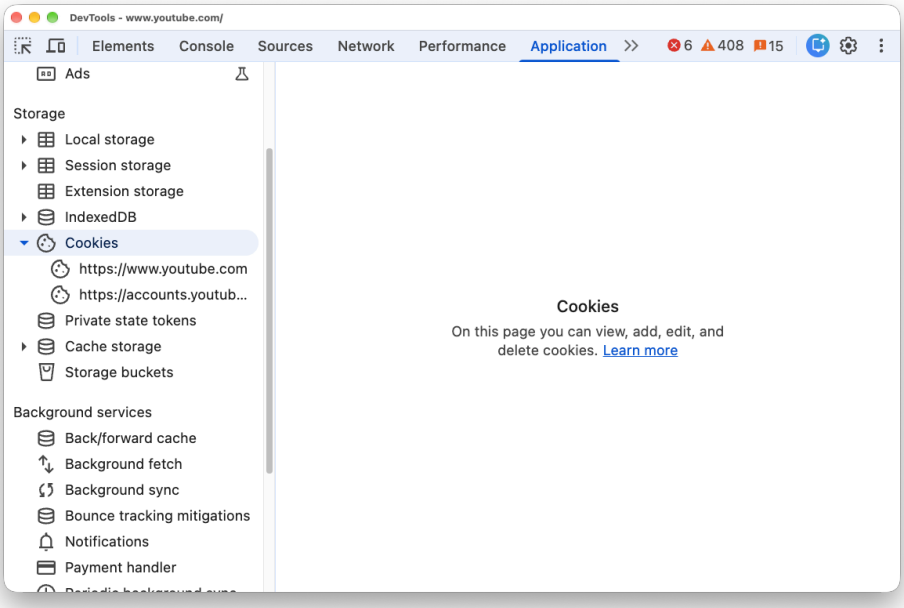



**図 4.13　https://www.youtube.com/の Cookie**

あとでセキュリティーの部分でもコメントするが，Cookie には重要な情報も保存されていて，それがネット上で売買されているので定期的に消去することをお奨めする．

- 文書の構造をツリーで表したものを DOM(Document Object Model) という．図 4.14 に後に使う hello.html の DOM を示す．これを使って JavaScript は様々なページの内容にアクセスすることが出来る．

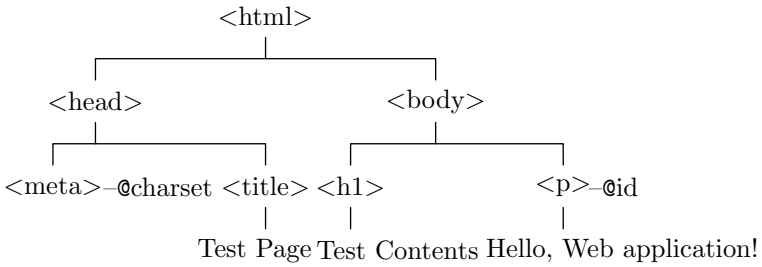



**図 4.14　hello.html の DOM**

**演習 25**　クエリの実験を自分のパソコンで再現してください．

**演習 26**　Cookie の実験を自分のパソコンで再現してください．

### 4.2.3　Web のサーバー・クライアントの演習

ここでは WEB のクライアント・サーバーのコマンドラインでの演習を行う: 特に nc = net cat と curl = client for URL を使用して，WEB のクライアント・サーバーを体験してもらいたい ([32] 参)．

- まず，サーバーを立ち上げる：作業ディレクトリーに移動しサーバー側の端末で

```
% nc -l 8080 #return #Ubntu では >nc -q 1 -l 8080
```

とする．8080 が TCP のポートナンバーである．
クライアント側の別の端末を立ち上げ

```
% curl http://localhost:8080/ #return
```

とすると，サーバ側の端末に次のような表示が表れる

```
% nc -l 8080
```

```
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.1.2
Accept: */*
```

そこで，サーバー側で手で

```
HTTP/1.1 200 OK #return
#return
Hello, Web application! #return
```

と入力すると，クライアント側で

```
% curl http://localhost:8080/
Hello, Web application!
```

と表示されるはずである．もし，HTTP/1.1 200 OK も表示したいときは

```
% curl  -v  http://localhost:8080/
```

とすればいい．やってみると以下のような出力が得られた：

```
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.1.2
> Accept: */*
>
< HTTP/1.1 200 OK
* no chunk, no close, no size. Assume close to signal end
<
Hello, Web application!
* Closing connection 0
```

- 毎回手入力するのは大変なので，1 回 ctrol D で nc を終了して，もう一度

```
% {echo 'HTTP/1.1 200 OK';echo ;
```

```
    echo 'Hello, Web application!';}|nc -l 8080
    # echo の前の改行不要
```

とすると，自動で先程の応答を送ることが出来る．

- 次に返事するファイルを html に変更する．まず作業ディレクトリーが tmp のとき，その中に server/ と client/ ディレクトリーをつくる

```
% mkdir server client
```

server/に以下のような hello.html を作成する

```
<!DOCTYPE html>
<html lang="ja">
    <head>
        <meta charset="UTF-8">
        <title>Test Page</title>
    </head>
    <body>
        <h1>Test Contents</h1>
        <p id="greed">Hello, Web application!</p>
    </body>
</html>
```

サーバー側で

```
% {echo 'HTTP/1.1 200 OK';echo ;cat hello.html} |
nc -l 8080  # nc の前の改行不要
```

クライアント側で

```
% curl -s -o hello.html http://localhost:8080/
% open hello.html
```

とするとブラウザーに表示される．

- 最後にブラウザーで直接表示するためには，(nc が既に終了しているので) サーバー側で同じことをして

```
% !!
```

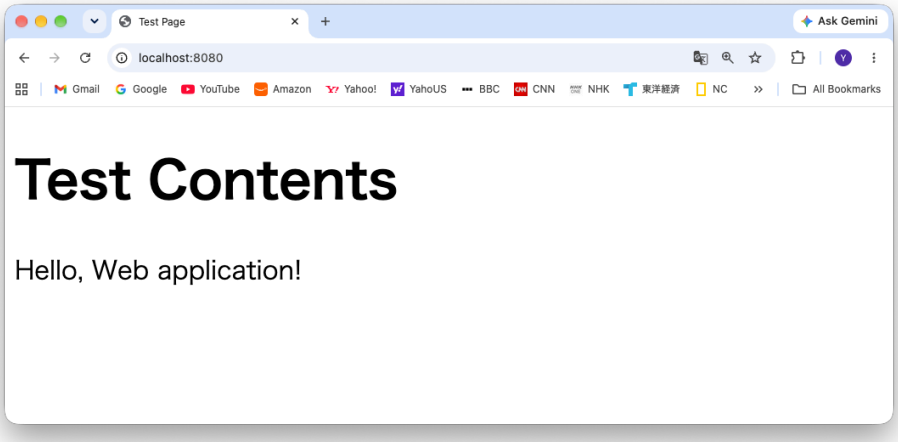



**図 4.15　Test の Browser での表示**

ブラウザーのアドレスバーに http://localhost:8080/と入れると図 4.15 のように表示されるであろう．このときサーバー側には以下のようなログが表示される．

```
GET / HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua: "Chromium";v="148", "Google Chrome";v="148",
 "Not/A)Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "macOS"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
148.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;
q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,
application/
signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
```

```
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9
```

これらが HTTP プロトコルの応答になる.

**演習 27**　この実験を自分のパソコンで再現してください．最後の実験でブラウザーでソースファイルを開き，hello.html と同じものであることを確認してください.

# 第 5 章　セキュリティ

セキュリティには数学的・技術的側面と人間・社会・経営的側面があるが，まず数学的・技術的側面をとりあげ，その後人間・社会・経営的側面を取り上げる．

## 5.1　暗号の数理

### 5.1.1　Z の剰余類

まず，暗号や認証で使われる初等整数論の復習を行う．どうしても代数でよく使われる言葉が出てきてしまうので [10], [62], [32] などを参考にしながら読み解いてもらいたい．この解説が代数へのイントロとなれば望外の喜びである．

$\mathbf{N}$ を（1 以上の）自然数全体のなす集合, $\mathbf{Z}$ を整数全体のなす集合とする．$\mathbf{Z}$ は加法 $+$ に関して群[1)]になっている．$\mathbf{Z}$ には掛け算 $\cdot$ ( $\times$ と書くこともある) もあるが 0 を除いた $\mathbf{Z} \setminus \{0\}$ では割り算ができない (任意の $n \neq \pm 1$ は $\mathbf{Z}$ 内で逆元が無い) ので群ではない．$\mathbf{Z}$ は $+$ と $\cdot$ に関して環[2)]であるが，体[3)]ではない．$m+n = n+m$ なので可換である，$m \in \mathbf{N}$ を固定し，$x, x' \in \mathbf{Z}$

$$x \equiv x' \mod m \Leftrightarrow \exists q \in \mathbf{Z} \text{ such that } x - x' = qm$$

と置くとこれは同値関係である．$x \in \mathbf{Z}$ を代表元とする同値類を $\bar{x}$ と置く．

$$x \equiv x' \mod m \text{ and } y \equiv y' \mod m$$
$$\Rightarrow \quad x+y \equiv x'+y' \mod m, \quad xy \equiv x'y' \mod m$$

---

1)　集合 $G$ が群であるとは任意の $a, b \in G$ に対して積 $ab \in G$ が定まり，次の三条件を満たすことである：

$$(ab)c = a(bc) \quad \forall a, b, c \in G$$

単位元 $e \in G$ が存在し，

$$ea = ae = a \quad \forall a \in G$$

任意の $a \in G$ に対して逆元 $a^{-1}$ が存在して

$$aa^{-1} = a^{-1}a = e$$

$ab = ba$ をみたすとき可換群といい，$+$ で表すことが多い．

2)　集合 $R$ が環であるとは演算 $+$ と $\cdot$ をもち $+$ に関しては可換群で，$\cdot$ については結合律をみたし可換であり，$\cdot$ と $+$ については分配則をみたすこと．$\mathbf{Z}$ や多項式環 $\mathbf{Z}[X]$ が典型例

3)　環 $K$ が体とは $K$ の 0 以外の元が $\cdot$ についての逆元を持つこと．$\mathbf{R}$ や $\mathbf{C}$ が例．

なので

$$\bar{x}+\bar{y}=\overline{x+y},\quad \bar{x}\cdot\bar{y}=\overline{xy}$$

と定め暫く $G$ と置く．$(G,+)$ は $\bar{0}$ を単位元とする可換群である．

$$f:\mathbf{Z}\ni x\mapsto \bar{x}\in G$$

とすると $f(x+y)=f(x)+f(y)$[4]であり，

$$\{x\in\mathbf{Z}\mid f(x)=\bar{0}\}=m\mathbf{Z}$$

見なせるので

$$G\sim\mathbf{Z}/m\mathbf{Z}$$

となる．巡回群とは一つの元から生成される群のことであり，$\mathbf{Z}/m\mathbf{Z}$ は $+$ については $\bar{1}$ で生成されているので有限巡回群である．これを $C_m$ と表すことがある．

$\mathbf{Z}/m\mathbf{Z}$ は $+$ と $\cdot$ について $\mathbf{Z}$ の計算と同じルールで計算していい．例えば

$$\bar{x}(\bar{y}+\bar{z})=\overline{xy+xz}=\bar{x}\bar{y}+\bar{x}\bar{z}$$

になる[5]．以下，$\mathbf{Z}/m\mathbf{Z}$ の積 $\cdot$ について詳しい構造を調べる．最初にいくつか計算してみる．

例 1. $\mathbf{Z}/4\mathbf{Z}$ の足し算と掛け算の演算表を作ってみる．足し算については巡回群 $C_4$ なのでそのままであるが，積に関しては $\bar{2}$ が零因子[6]となっているので群ではない．

| $+$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ |
|---|---|---|---|---|
| $\bar{0}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ |
| $\bar{1}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{0}$ |
| $\bar{2}$ | $\bar{2}$ | $\bar{3}$ | $\bar{0}$ | $\bar{1}$ |
| $\bar{3}$ | $\bar{3}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ |

**表 5.1** $+$ mod 4

| $\cdot$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ |
|---|---|---|---|---|
| $\bar{0}$ | $\bar{0}$ | $\bar{0}$ | $\bar{0}$ | $\bar{0}$ |
| $\bar{1}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ |
| $\bar{2}$ | $\bar{0}$ | $\bar{2}$ | $\bar{0}$ | $\bar{2}$ |
| $\bar{3}$ | $\bar{0}$ | $\bar{3}$ | $\bar{2}$ | $\bar{1}$ |

**表 5.2** $\cdot$ mod 4

4) このようなものを $+$ についての群の準同形という．
5) $f(x+y)=f(x)+f(y)$, $f(xy)=f(x)f(y)$ をみたすものを環の準同形という．
6) $a\neq 0$, $b\neq 0$ が $ab=0$ となるとき零因子という．

例 2. $\mathbf{Z}/5\mathbf{Z}$ の足し算と掛け算の演算表を作ってみる．掛け算では零因子がないので $\bar{0}$ を除く $(\mathbf{Z}/5\mathbf{Z})^\times = (\mathbf{Z}/5\mathbf{Z}) \setminus \{\bar{0}\}$ は群となる．これは巡回群 $C_4$ と同型である (表 5.1 と異なって見えるかもしれないが，$\bar{1}_4, \bar{2}_5$ のように表し区別し，$f : (\mathbf{Z}/5\mathbf{Z})^\times \to \mathbf{Z}/4\mathbf{Z}$ を

$$f(\bar{1}_5) = \bar{0}_4, \quad f(\bar{2}_5) = \bar{1}_4, \quad f(\bar{3}_5) = \bar{3}_4, \quad f(\bar{4}_5) = \bar{2}_4$$

とすると $(\mathbf{Z}/5\mathbf{Z})^\times$ は掛け算，$\mathbf{Z}/4\mathbf{Z}$ は足し算として同型[7]になることが分かる)．

| $+$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ |
|---|---|---|---|---|---|
| $\bar{0}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ |
| $\bar{1}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ | $\bar{0}$ |
| $\bar{2}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ | $\bar{0}$ | $\bar{1}$ |
| $\bar{3}$ | $\bar{3}$ | $\bar{4}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ |
| $\bar{4}$ | $\bar{4}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{2}$ |

**表 5.3** $+$ mod 5

| $\cdot$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ |
|---|---|---|---|---|---|
| $\bar{0}$ | $\bar{0}$ | $\bar{0}$ | $\bar{0}$ | $\bar{0}$ | $\bar{0}$ |
| $\bar{1}$ | $\bar{0}$ | $\bar{1}$ | $\bar{2}$ | $\bar{3}$ | $\bar{4}$ |
| $\bar{2}$ | $\bar{0}$ | $\bar{2}$ | $\bar{4}$ | $\bar{1}$ | $\bar{3}$ |
| $\bar{3}$ | $\bar{0}$ | $\bar{3}$ | $\bar{1}$ | $\bar{4}$ | $\bar{2}$ |
| $\bar{4}$ | $\bar{0}$ | $\bar{4}$ | $\bar{3}$ | $\bar{2}$ | $\bar{1}$ |

**表 5.4** $\cdot$ mod 5

$m, n \in \mathbf{Z}$ に対して $(m, n)$ で $m$ と $n$ の最大公約数を表す．

$$(m, n) = 1 \quad \Leftrightarrow \quad \exists a, b \in \mathbf{Z} \text{ such that } am + bn = 1$$

がユークリッドの互除法から分かる．このことから

$$(m, n) = 1 \quad \Leftrightarrow \quad \exists \bar{b} \in \mathbf{Z}/m\mathbf{Z} \text{ such that } \bar{b}\bar{n} = \bar{1}$$

$(m, n) = 1$ のとき $m$ と $n$ は互いに素という．$m \in \mathbf{N}$ を固定し $(m, n) = 1$ なる $\bar{n} \in \mathbf{Z}/m\mathbf{Z}$ を既約剰余類という．

既約剰余類 $\bar{a} \in \mathbf{Z}/m\mathbf{Z}$ を固定すると $\bar{b}\bar{a} = \bar{1}$ なる $\bar{b} \in \mathbf{Z}/m\mathbf{Z}$ が見つかるので

$$f : \mathbf{Z}/m\mathbf{Z} \ni \bar{n} \mapsto \bar{a} \cdot \bar{n} \in \mathbf{Z}/m\mathbf{Z}$$

を考えると $f$ は $+$ に関して準同形写像で，$\bar{b}$ を掛けることで同形写像であることが分かる．

$$\varphi(m) = |\{n \mid n = 1, \ldots, m-1 \text{ such that } (m, n) = 1\}|$$

と置くと $\mathbf{Z}/m\mathbf{Z}$ には $\varphi(m)$ 個の既約剰余類が存在することが分かる．$\varphi$ はオイラー関数と呼ばれる．$\mathbf{Z}/m\mathbf{Z}$ には $\varphi(m)$ 個の生成元が存在し，$\mathbf{Z}/m\mathbf{Z}$ はどの生成元をとっても加群としては同型な巡回群となる．

7) $f$ が (この場合は群の) 準同形で，更に全単射になること

例 3. $\varphi(4) = 2$, $\varphi(5) = 4$.

**演習 28**　$p$ を素数とするとき

$$\varphi(p^n) = p^n - p^{n-1}$$

を示せ.

$(m, n) = 1$ なる $n$ を固定する．$\bar{x}_1, \cdots, \bar{x}_{\varphi(m)}$ を $\mathbf{Z}/m\mathbf{Z}$ の既約剰余類全体とする．$\bar{n}\bar{x}_1, \cdots, \bar{n}\bar{x}_{\varphi(m)}$ も既約剰余類全体なので，集合としては一致している．したがって

$$\bar{x}_1 \cdots \bar{x}_{\varphi(m)} = \bar{n}\bar{x}_1 \cdots \bar{n}\bar{x}_{\varphi(m)} = \bar{n}^{\varphi(m)}\bar{x}_1 \cdots \bar{x}_{\varphi(m)}$$

$\bar{x}_1 \cdots \bar{x}_{\varphi(m)}$ は逆元をもつので $\bar{n}^{\varphi(m)} = \bar{1} \in \mathbf{Z}/m\mathbf{Z}$, つまり $(m, n) = 1$ なる任意の $n$ に対して

$$n^{\varphi(m)} \equiv 1 \mod m \tag{5.1}$$

$p$ を素数とすると，$\varphi(p) = p - 1$ なので任意の $x \not\equiv 0 \mod p$ に対して

$$x^{p-1} \equiv 1 \mod p$$

となり，フェルマーの小定理が得られる．これから任意の $x \in \mathbf{Z}$ に対して

$$x^p \equiv x \mod p \tag{5.2}$$

も分かる.

**命題 1（中国の剰余定理）**　互いに素な $m, m' \in \mathbf{N}$ を取る．任意の $a, a' \in \mathbf{Z}$ に対して

$$x \equiv a \mod m, \quad x \equiv a' \mod m'$$

を満たす $x$ が存在して $\mod mm'$ で一意.

**証明**　$m, m' \in \mathbf{N}$ は互いに素なので

$$nm + n'm' = 1$$

なる $n, n' \in \mathbf{Z}$ が存在する．つまり

$$n'm' \equiv 1 \mod m, \quad nm \equiv 1 \mod m'$$

なので，$x = an'm' + a'nm \in \mathbf{Z}$ の $\mod mm'$ の代表が求めるものである.

$x, x'$ が方程式を満たすと $x - x'$ は $\mod m$ でも $\mod m'$ でも 0 よって $x - x'$ は $\mod mm'$ でも 0 になり，一意性が分かる． □

**命題 2** 互いに素な $m, m' \in \mathbf{N}$ に対して

$$\varphi(mm') = \varphi(m)\varphi(m') \tag{5.3}$$

**証明** $k$ を $1, \cdots, mm' - 1$ の間にある自然数で $mm'$ と互いに素なものとする．すると

$$k \equiv j \mod m, \quad k \equiv j' \mod m'$$

なる $j \in \{1, \ldots m-1\}$, $j' \in \{1, \ldots m'-1\}$ が見つかる．逆に中国の剰余定理は $j$, $j'$ から $k$ が $\mod mm'$ で一意的に定まることを意味する．$k$ は $mm'$ と互いに素なので，ある $a$, $b \in \mathbf{Z}$ が存在して

$$ak + bmm' = 1$$

よって $k = j + cm (c \in \mathbf{Z})$ と置くと

$$1 = a(j + cm) + bmm' = aj + (ac + bm')m$$

となり $j$ は $m$ と互いに素になる．同様に $j'$ は $m'$ と互いに素で，これから示したい等式が分かる． □

例 4. $m = 3$, $m' = 5$ のとき $\varphi(15) = \varphi(3)\varphi(5) = 2 \cdot 4 = 8$

| mod 15 | ~~0~~ | 1 | 2 | ~~3~~ | 4 | ~~5~~ | ~~6~~ | 7 | 8 | ~~9~~ | ~~10~~ | 11 | ~~12~~ | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| mod 3 | 0 | 1 | 2 | 0 | 1 | 2 | 0 | 1 | 2 | 0 | 1 | 2 | 0 | 1 | 2 |
| mod 5 | 0 | 1 | 2 | 3 | 4 | 0 | 1 | 2 | 3 | 4 | 0 | 1 | 2 | 3 | 4 |

**表 5.5** $\times$ $\mod 15$, $\mod 3$, $\mod 5$

ここではこれ以上必要ないが，将来的には楕円曲線暗号などで必要になるであろう有限体について簡単にまとめておく．フェルマーの小定理は $(\mathbf{Z}/p\mathbf{Z})^\times = (\mathbf{Z}/p\mathbf{Z}) \setminus \{\bar{0}\}$ が積 $\cdot$ について巡回群であることを意味している．特に $\mathbf{Z}/p\mathbf{Z}$ が体であることも意味していている．このとき $\mathbf{F}_p$ と表すことが多い．これは標数 $p$ の有限体である．一般に $F$ を体とする．環の準同形 $f : \mathbf{Z} \ni n \mapsto n \times 1 \in F$ が単射で無いとき，$\mathbf{Z}$ は単項イデアル整域なので $\mathrm{Ker}\, f = (p)$ なる素数 $p$ がある．これを標数という．$f$ が単射のときは標数は 0 と言う．有限個の元からなる体が有限体で，定義からその標数はある素数 $p$ となる．このとき $f$ より

$\bar{f} : \mathbf{F}_p \to F$ は体の埋め込みである．つまり，$F$ は $\mathbf{F}_p$ の拡大体となる．このとき $\mathbf{F}_p$ を素体という．体の定義より $F$ は $\mathbf{F}_p$ 上のベクトル空間と見なせるが，仮定から有限次元ベクトル空間であり，この次元を $n$ と置くと $F$ は $q = p^n$ の元からなることが分かる．$\mathbf{F}_p$ 上のベクトル空間 $F$ の $\mathbf{F}_p$ 上の同型群がこの代数拡大（は $\mathbf{F}_p$ の正規分離拡大なので $F/\mathbf{F}_p$）のガロワ群となる．

**演習 29**　$n$ を 2 以上の自然数とすると $\mathbf{Z}/p^n\mathbf{Z}$ は $p^n$ 個の元からなる集合だが，体でないこと（したがって，これは上の有限体 $F$ とはなり得ないこと）を示せ．

**宿題 4**　標数 $p$ の有限体は $\mathbf{F}_p$ のガロワ拡大であることを示し，そのガロワ群を決定せよ（ヒント：(5.2) の $p$ を $q = p^n$ に代えた多項式でこの拡大を特徴付けよ．またそのガロワ群はフロベニウス写像で生成されることを示せ）．

### 5.1.2　公開鍵暗号—RSA

アリス (Alice, 後で A と略す) とボブ (Bob, 後で B と略す) が共通の秘密の鍵を共有していてその鍵を使ってアリスが文章 (平文) を暗号化した暗号文を，インターネット等を通じてボブに送りボブがその鍵を使って復号するような暗号を秘密鍵暗号（共有鍵方式と呼ばれることの方が多い）という．シーザー式暗号はその例である．送りたい文の A〜Z [8] を順に 0 から 25 に変換し，mod 26 で +3 した結果の 0〜25 を A〜Z に変換した文を送り，相手は逆算して暗号を特く．3 が共有鍵（秘密鍵）になる．詳しい説明は控えるが秘密鍵暗号には以下のような方式がある：

| 暗号 | 鍵の長さ | 発表年 | コメント |
|---|---|---|---|
| DES | 54bit | 1977 | 危殆化 |
| トリプル DES | $3 \times 54$bit | | 危殆化 |
| AES | 128/192/256bit | 2000 | [73] に詳しい説明 |

**表 5.6　秘密鍵暗号**

二人が知り合いで二人で合って秘密の鍵を共有することは簡単なように思えるが，インターネットでこれまで会ったことのない二人，例えば amazon とその新しい顧客が秘密の

8)　この話を初めて聞いたとき小文字の a〜z や句読点や空白がないのに違和感を感じたが，この時代は実際そうであったようで小文字や句読法が発明されたのは中世後期であったそうである．もし今でも英語の文が全て大文字で語の間に空白や句読点がないものであったら英語を読むのはいかに困難であったであろうか．

鍵を共有するにはどのようにすれば良いだろうか？このような問題を解決するために生まれたのが公開鍵暗号である．

1976 年デフィーとホフマンは公開鍵暗号のアイデアを発表したが，1978 年にリベスト・シャミール・アドルマンが発表した RSA アルゴリズムを解説したい：Bob に Alice が秘密情報を送りたいとする．Bob は同じぐらいの大きさのランダムな素数 $p$, $q$ を選び $m = pq$ を計算する．(5.3) より

$$\varphi(m) = \varphi(p)\varphi(q) = (p-1)(q-1) = pq - p - q + 1$$

である．$pq-p-q+1$ と互いに素な $0 < e < pq-p-q+1$ をランダムに選び，$\mod \varphi(m)$ で $e$ の逆元 $0 < f < \varphi(m)$ を互除法で計算する．つまり，

$$ef \equiv 1 \mod \varphi(m) \tag{5.4}$$

Bob は $[m, e]$ を公開鍵として公開し，$[m, f]$ を秘匿鍵として自分の所に秘密に保存しておく．Alice は Bob の公開鍵 $[m, e]$ を使って，自然数の秘密情報 $0 \leq C < m$ から

$$C^e \equiv D \mod m \quad 0 \leq D < m$$

を計算し Bob に $D$ を送る．Bob は秘匿鍵 $f$ を使って

$$D^f \equiv C \mod m \quad 0 \leq C < m$$

を得ることができる．実際，ある $k \in \mathbf{Z}$ が存在して (5.4) より

$$\begin{aligned} D^f &= C^{fe} \\ &= C^{k\varphi(m)+1} \\ &\equiv C \mod m \quad 0 \leq C < m \end{aligned}$$

となるからである．攻撃者 Eve が公開鍵 $[m, e]$ を知り Alice の $D$ を傍受したとしても，$m$ の因数分解を知らなければ $\varphi(m)$ が分からず，$\mod \varphi(m)$ での $e$ の逆 $f$ を求めることが出来ないので $C$ を復号出来ないわけである．この方法で Alice は Bob に例えば共通の秘密鍵を送り，共有することができる．公開暗号方式は計算が複雑で時間も掛かるので，その後は効率よく秘密鍵を使った暗号を使うことが可能になる．

**演習 30** $p = 3$, $q = 5$ のとき $ef \equiv 1 \mod \varphi(15)$ となる $e$, $f$ を探し，適当な $D \in \{1, 2, \ldots, 15\}$ を公開鍵で暗号化しそれを解読してください．

**演習 31** github の自分のアカウントに ssh で公開鍵を登録：

`% cd  # ホームディレクトリに移動`

`% ssh-keygen # RSA` **で秘密鍵と公開鍵の作成**

その後公開鍵を GitHub の自分のホームに登録してください．

**宿題 5**　$p = 11, q = 13$ で $ef \equiv 1 \mod \varphi(pq)$ となる $e, f$ を探し，適当な $D$ を公開鍵で暗号化しそれを解読するプログラムを作成してください．出来るならもっと大きな素数 $p, q$ にプログラムを拡張してください．

### 5.1.3　デジタル署名

物理的書類の真実性を担保するのにサインが有効であった．これを電子的な文書に実現するのが デジタル署名であり，RSA を使い以下のようにデジタル署名を作ることができる．Alice が公開鍵 $[m_A, e_A]$ と秘匿鍵 $[m_A, f_A]$ を作り公開鍵を公開しているとする．

Alice がデジタル署名したい平文を $M$ とし，Bob がそれを送りたい人間で Bob 以外の人間 Victor がそれを検証するとする．Alice はまず一方向ハッシュ関数で $M$ のハッシュ値を求め，それを秘匿鍵 $[m_A, f_A]$ で暗号化したものを $D$ とする．$(D, M)$ を Bob に送る．Bob 及び Victor は公開鍵 $[m_A, e_A]$ で $D$ を復号化し，$M$ のハッシュ値を求め同じかどうかを調べ，同じならばデジタル署名は正しいと判断できる．

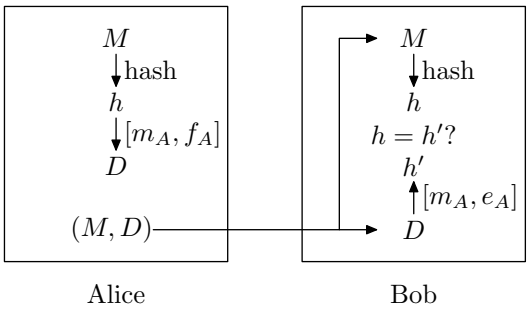



**図 5.1　デジタル署名**

さらに，Alice が公開鍵 $[m_A, e_A]$ と秘匿鍵 $[m_A, f_A]$ を作り公開鍵を公開していて，Bob も公開鍵 $[m_B, e_B]$ と秘匿鍵 $[m_B, f_B]$ を作り公開鍵を公開しているとする．Alice が Bob に短いメッセージ $M < \min(m_A, m_B)$ を送りたいとする．

$$M^{f_A} \equiv C \mod m_A \quad 0 \le C < m_A$$

$$C^{e_B} \equiv D \mod m_B \quad 0 \le D < m_B$$

として $D$ を Bob に送る．Bob は

$$D^{f_B} \equiv C \mod m_B \quad 0 \leq C < m_B$$

となるので

$$C^{e_A} \equiv M \mod m_A \quad 0 \leq M < m_A$$

となり，$M$ を復元できる．この方式もデジタル署名と呼ばれることがある．

### 5.1.4 認証局

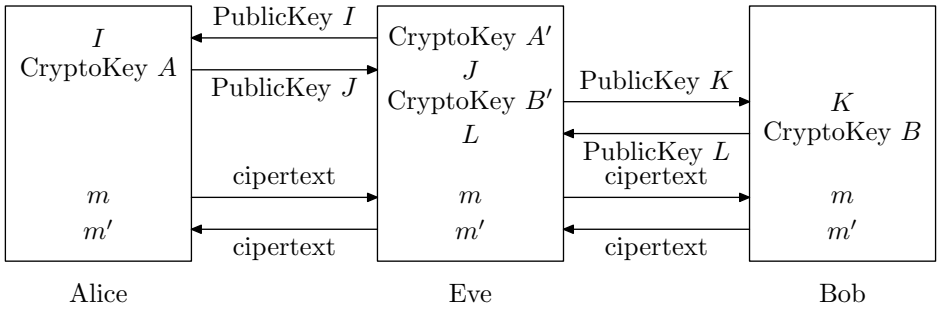



**図 5.2 中間者攻撃**

一つだけ問題なのは，公開暗号でもデジタル署名でも Alice が本当に Alice か Alice を騙った別人ではないか，Bob は本当に Bob なのか Bob を騙った別人ではないかを別の方法で確認しておく必要があることである．例えば，Eve が最初 Bob を騙って Alice と公開キーを共有し，Eve が Alice を騙って Bob と公開キーを共有すると Alice と Bob の間の通信が Eve を通して行われるようになり，二人の通信が筒抜けになるという中間者攻撃 (man-in-the-midle Attack) が知られている．

そのために使われるのが認証局 (Certificate Authority) である．まず，Alice と Bob は既にある認証局に信頼を確立しているとする．Bob から Alice への公開キーを送るとき，Bob から公開キーとその Bob によるデジタルサインを認証局へ送り，認証局で Bob から送られてきたことを確認する．その公開キーと認証局によるデジタルサインを Alice に送り，認証局から送られてきたことを認証し，Bob が本当に Bob であることを Alice に認証する (図 5.3)．秘密鍵はどこでもインターネットを通っていないことに注意する．Alice が Bob への公開キーを送るときも同様である．

ただし，Alice と Bob は既にある認証局に信頼を確立しているとは限らない．そこで，まずブラウザーが信頼できる認証局を幾つか初期設定で知っているようになっている．幾つかの認証局同士はお互いに公開キーと基本情報を認証したデジタル署名を共有してい

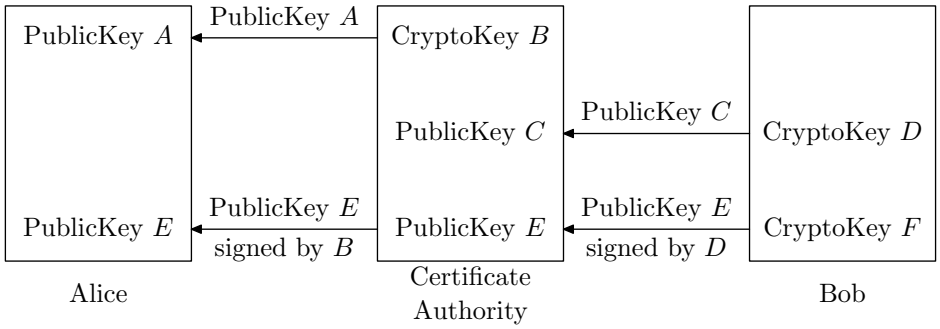



**図 5.3　Bob が Alice へ認証局を通して公開キーを送付**

て，それをたどって認証できる認証局のみを信頼し，その認証する相手を認証するという信用の輪を広げる仕組みになっている．

### 5.1.5　暗号を実装するプロトコル

暗号や電子署名は色々な方法で実装化されている：

- IPsec：IP パケット単位でのデータの改竄防止を行うプロトコル
- SSL =secure socket layer, TLS =transport layer security：暗号通信を行うためのプロトコル
- HTTPS, SMTPS, FTPS：元々のプロトコルのセキュリティを強化したプロトコル (図 5.4 矢印の部分をクリックして connection is secure を選択して，表れるウインドウの右下のアイコンをクリックした状態).
- SSH =secure shell：公開鍵暗号を使って，ネット上でログインしたり，ファイルを転送するプロトコル

また，それらの実装は日々問題点が表れ，それに対応する拡張が行われているので，CRYPTREC などで確認する必要がある．

## 5.2　セキュリティマネージメント

### 5.2.1　セキュリティの基本概念

コンピュータやネットを使った活動が安全かつ正確に出来るようにすることを情報セキュリティ (Information Security) といい，それへの対応を情報セキュリティマネージメント (Information Security Management) という．

情報セキュリティについての基本的な概念として以下のものがある：

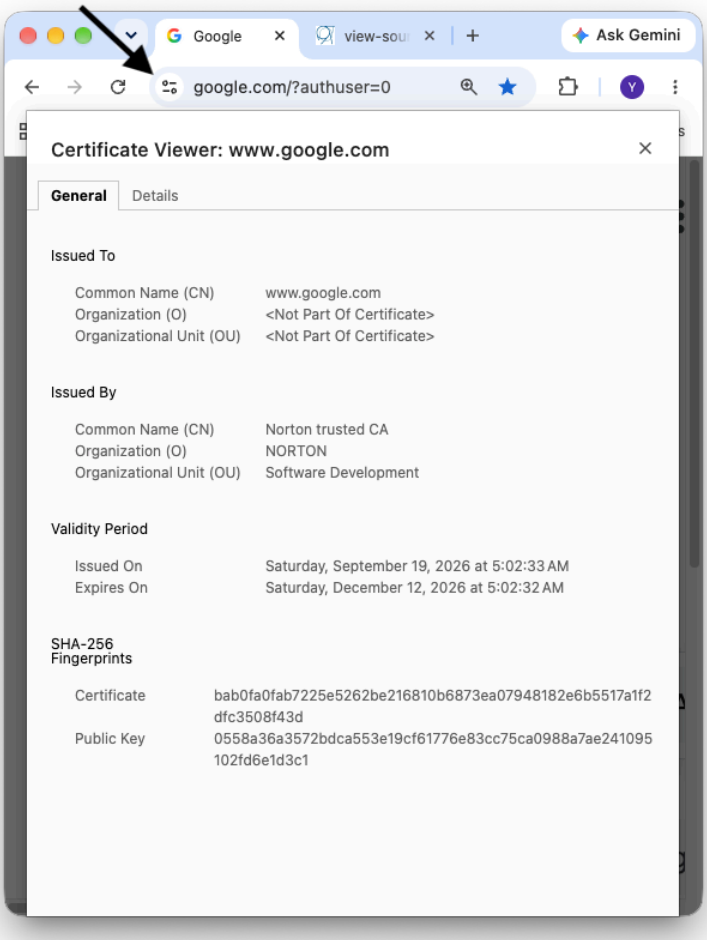



**図 5.4　ブラウザーでの認証証明**

- 情報セキュリティの CIA：基本の 3 つの柱
  - 機密性 (Confidentiality)：権限を持った人やデバイスのみが情報にアクセスできること
  - 統一性 (Integrity)：情報の改竄や破壊や消去が出来ない，起こらないこと
  - 有用性 (Availability)：情報にアクセスしたい時にその情報にアクセスできること
- 次の 4 つも合わせて情報セキュリティの 7 要素という：
  - 真正性 (Authenticity)：個人，組織，情報システム，情報が正当でなりすましがない状態.
  - 責任追従性 (Accountability)：情報を閲覧変更した個人，組織を特定，追跡できる状態.
  - 否認防止 (Nonrepudiation)：情報を閲覧変更したことを後から否認できないこと.

  - 信頼性 (Reliability)：情報システムが意図した通りに動くこと．
- 組織や個人に必要な価値のあるものを資産というが，特に情報システムや情報自体を情報資産という．
- 攻撃側と守備側の関係：
  - 脅威 (Threat)：情報システムや情報に危害を与えうる状況や事実
  - 脆弱性 (Vulnerability)：情報システムや情報に脅威が影響を与える部分があること．
  - リスク (Risk)：脅威が脆弱性のある情報システムや情報に影響を与える確率の度合い．

  リスクは大雑把に以下のように評価できる：

$$\begin{aligned}&\text{リスク}\\&=\text{情報資産の価値}\times\text{脅威の大きさ}\times\text{脆弱性の度合い}\end{aligned}$$

- $$\text{セキュリティシステムの強度}\leq\text{構成要因のもっとも弱い部分の強度}$$

- 脅威には三つの側面がある：
  - 物理的脅威
  - 人的脅威
  - 技術的脅威

  情報セキュリティのことを (ネットワークの) 技術的脅威のことにだけに限定するのは危険である．
- 攻撃の目的：
  - 愉快犯
  - スクリプトキディ
  - お金 RaaS
  - ハクティビズム：ハッカーの主義主張を広げるため
  - 貴重な個人や組織の情報の収集やダークウェブでの売買
  - 内部関係者の組織や個人への恨みへの復讐
  - サイバーテロリズム・サイバー戦争：

    ロシアのウクライナ侵攻や米軍・イスラエル軍のイラン戦争の始まる少し前，全土で例を見ないサイバー攻撃が起こり，社会の通信網を混乱させたり重要インフラを使用できなくした後に現実の戦争が起こった．
  - 敵国による世論操作・選挙介入：

民主国の選挙で陰謀論を広めて社会に混乱を起こしたり，選挙妨害などの選挙介入が行われ敵国に有利な指導者を生み出す工作，認知戦が行われている ([3], [50]).

### 5.2.2 技術的脅威

技術的脅威，特にネットワーク技術に関連した脅威であるサイバー攻撃について纏める．

- サイバーキルチェーン (Cyber Kill Chain)：キルチェーンは軍事用語で戦闘の進行段階を示すものだが，それをサイバー攻撃の分析に使ったのがサイバーキルチェーンである．次の様な攻撃の段階がある：
  1. 偵察 (Reconnaissance) ：標的やその脆弱性を探すこと．
     OSINT(Open Source Intelligence) によるデータ収集を含む．
  2. 武器化 (Weponization)：攻撃ツールを準備する段階
  3. 配達 (Delivery)：攻撃ツールを標的に届けること
  4. エクスプロイト (Exploit)：攻撃ツールを情報システムで実行すること
  5. インストール (Install)：永続的なアクセスを確保するためにマルウエアなどをインストールすること
  6. Command and controll(C&C)：C&C サーバーとの通信の確立
  7. 目的の実行：データの収集・破壊など脅威の目的を達成すること

  逆に言うと，もし途中の段階で阻止できれば被害を食い止めることが出来る．

具体的な攻撃方法を紹介する (がもちろん完全ではない．常に新しい攻撃法が開発されているので，まめに確認するようにしてほしい)：

- 一般的攻撃手法
  - 不正アクセス
  - クレデンシャルスタッフィング
  - 脆弱性の攻撃
  - ゼロデイ攻撃
  - マルウエア攻撃
  - ドライブバイダウンロード：ウェブなどでユーザーに意識させずにソフトウエアをダウンロードさせる攻撃
  - 標的型攻撃：攻撃相手を決めて徹底的に攻撃するやり方．
- パスワード攻撃：

  - ブルートフォース攻撃
  - リバースブルートフォース攻撃：パスワードを固定して利用者 id を変化させる攻撃
  - パスワードリスト攻撃：何処かで入手したパスワードのリストを使う攻撃
  - レインボーテーブル攻撃
  - スニッフィング：盗聴でパスワードを盗む攻撃
- マルウエア・不正プログラム：
  - ルートキット
  - バックドア
  - スパイウエア
  - ウイルス
  - ランサムウエア：身代金要求のための準備を助けるソフトを使った攻撃
  - ボット
  - トロイの木馬
  - ワーム
  - 偽セキュリティ警告：セキュリティ警告を画面に表示して，偽のサポートセンターに電話させたり，フィッシングサイトに導く攻撃.
- Web を使ったサイバー攻撃の具体例：
  - SQL インジェクション：Web ではログインのためのデータベースとして SQL を使うことが多く，適切な対策が取られていないと，そこに直接 SQL 文を書き込むことでログインしたり，利用者の情報を詐取されることがある.
  - OS コマンド・インジェクション：言語により Shell が呼び出せる関数を持つものがあり，そこのコマンドを直接入れて実行可能である.
  - ディレクトリ・トラバーサル：相対パスなどを使って公開されていないディレクトリに外部からアクセスし，情報を詐取すること.
  - バッファオバーフロー
  - セッション・ハイジャック：ウェブサービスでセッション id を発行するものがある．その管理がずさんだとセッション id を推測され，それを使って セッションの乗っ取りが行われる.
  - クロスサイトスクリプティング (XSS)：不正なスクリプトをクライアント側で行わせる攻撃
  - クロスサイトリクエストフォージェリ (CSRF)：不正なスクリプトをサーバ側で行わせる攻撃
  - HTTP ヘッダインジェクション：HTTP ヘッダの中に 不正なスクリプトを埋め

込むなどの攻撃
  - phishing フィッシング：信頼できる機関を装い偽の Web サイトに誘い出し個人情報などを集める中間者攻撃.
  - Click Fix：「私は人間です」のボタンなどを押させ，実際はコマンドラインのコマンドを実行する攻撃.
- Network を使ったサイバー攻撃の具体例：
  - サービス拒否攻撃 (DoS, DDoS =Distributed Denial of Service, ディードス)：多量のネットアクセスを起こさせサービスを麻痺させる攻撃．この攻撃はサービスを麻痺させるだけだが，裏で本当のサイバー攻撃が行われていることがある.
  - メールヘッダインジェクション
  - プロンプトインジェクション
  - FreeWiFi の使用に付け入る攻撃
  - リモートディスクトップへの攻撃
  - VPN への攻撃 (後で詳しく解説する)

### 5.2.3 技術的脅威への対応

技術的脅威における攻撃のそれぞれに対応の仕方があり (もちろん，新しい脅威には対応の方法が分からない場合もあり，その場合は脆弱性のあるサービスを無効化する必要がある)，それぞれに高度な専門性が必要になるが，ここでは一般的な最低限の対応について纏める.

- 認証 (authentification)：ある人が本人であることを確かめること
- 認可 (authorization)：どこまでアクセスを許可するかの決めること，その決まり

例 5. ホテルに予約した後，チェックインするとき予約した人間あることを確認することが認証であり，自分の泊まる部屋の鍵をうけとり，部屋の使用権を確保するのが認可に当たる．このとき鍵は自分の泊まる部屋のみにしか使えないので使用範囲を最小限に抑えていることがわかる.

認証の手段として

- 所持認証：持っているもので認証すること．例は自分の家の鍵
- 知識認証：知っている知識で認証すること．例はパスワードや暗唱番号
- 生体認証：体の情報を元に認証すること．例は指紋認証や顔認証

技術的脅威への対応

- 万が一その人の権限が攻撃者に奪われた場合も最小の被害で済むようにアクセス権は必要最小限に限る (最小アクセス権の原理) のが基本の方針
- パスワード管理：ただし後で指摘するように，以下を実現するのは普通の人間には極めて難しことである．
  - 予想が難しいパスワード
  - 定期的に変更
  - 使い回しはダメ
- 特に二つの認証方式を組み合わせたものを二要素認証という．キャッシュカードはその所持と暗証番号 PIN(Personal Identification Number) の知識を組み合わせた二要素認証の例である．最近ではホームページのサービスの登録の本人確認で，自分のアドレスを登録するとそのアドレスに数字をメールしてきて，それを https のホームページに貼り付けると本人確認が済むという二要素認証がよく使われている．三つ以上の認証方法を組み合わせた多要素認証が使われる場合もある．
- パスキー Passkey：各サーバーごとに最初にローカルで生成した公開鍵をサーバーに登録し，秘密鍵はログイン名とサーバーのアドレスと共に大切にローカルの端末に保存しておく．以降保存されたサーバーのアドレスを訪れると，サーバー側で毎回ランダムに生成される数字を公開鍵で暗号化した チャレンジというものを送り返してくるので，ローカルの秘密鍵で解読し，サーバーと照合し合致すればログイン出来る．秘密鍵を使うための本人確認はローカルの認証器 (所有認証) の生体認証やパスフレーズ (パスワードに近いが秘密鍵の解除のみに使われる，知識認証) などで行う．秘密鍵・生体認証・パスフレーズはログインするときインターネット上に一切送られていないこと，チャレンジは毎回廃棄されるので第三者に見られて構わないことに注意してもらいたい．また，フィッシングサイトを自分から訪れることもないし，フィッシングサイトにはアルゴリズムからもログイン出来ないことに注意してほしい．
- ファイヤーウォール (FireWall)：内部ネットワーク と WAN の間おいて通過できるパケットの IP アドレスとサービスの種類を制限する装置やアプリケーションを設置する．また，外部に公開したいメールサーバーや Web サーバが置かれる DMZ=DeMilitarized Zone が設置されることが多くなった (図 5.6).
- OS やアプリを常に最新のものに保つ．サポートが無くなった OS やアプリ，フレームワークは使用しない．
- ウイルス対策ソフトの導入．

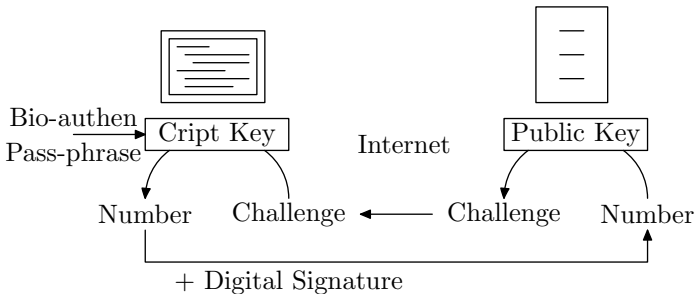



**図 5.5　パスキー**

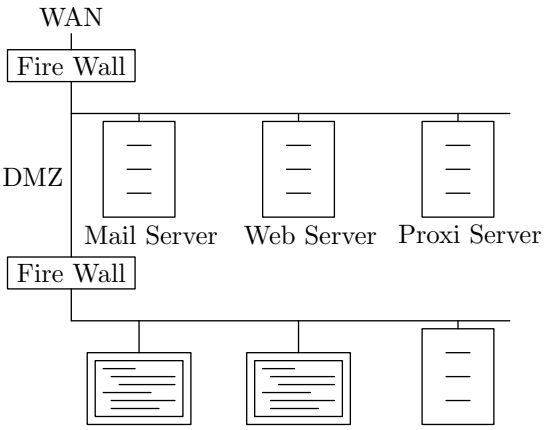



**図 5.6　ファイヤーウォール**

- FreeWifi の使用禁止.
- リモートディスクトップの使用禁止 (私の Mac だと System Setting/General/Sharing で Screen Sharing を off (図 5.7). 私は File Sharing と Remote Login は on にしているが，使っていない人，その意味が分からない人はこれらも off にした方が安全だろう).
- VPN(Virtual Private Network)：IPsec や SSL/TLS を使う SSL-VPN を使い通信を秘匿化して送る方法. LAN の外から LAN の中へ秘密にアクセスできるようになるのでセキュリティを保つためによく使われていた.

  しかし，ルート権限を持つもののアカウントが突破され遠隔から完全に乗っ取られたり，適切に管理されていない VPN 器機の脆弱性が狙われたりすることがある[9)]ことが分かって，最近では使うことを規制するようになった.

9)　警察庁「令和 8 年上半期におけるサイバー空間をめぐる脅威の情勢等について」によると，最近のランサムウエアのインシデントの半数以上で VPN の脆弱性が使われていた.

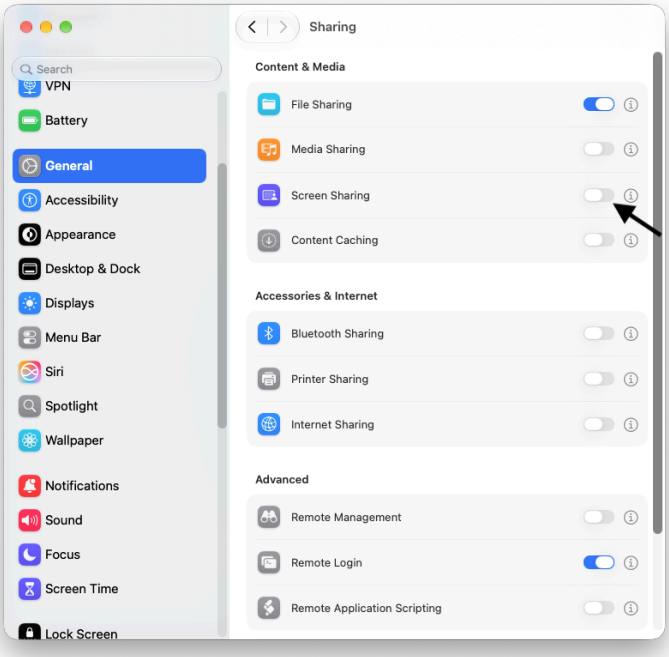



図 5.7　リモートディスクトップ (Screen Sharing) の使用禁止

### 5.2.4　人間的・物理的脅威と対応

セキュリティでは技術的側面だけでなく，人間的・物理的側面も重要になる．例えばシステム管理者に「難しいパスワードを定期的に変更し，パスワードの使い回しはするな」と言われていても面倒くさくてその指導に従っていない人の方が多いと推測される（皆さんの中にはこんな酷いパスワードを使っている人はいないと思うが…）．しかし，一般にセキュリティを強くすると使い勝手が不便になることも多い．

本気で攻撃を仕掛けてくる相手 (国) は物理的・人間的なセキュリティの脅威も突いてくる．特に色々な組織の中枢に近いものはそこに近づく前からセキュリティ全般に注意を払う訓練が必要である．また，フィッシングはネットワークセキュリティと人間的なセキュリティの二つの側面を持っているように，この三つは独立している訳では無いが，便宜上三つに分けて解説する．

物理的なセキュリティ ：

- 自分の部屋を出るときには必ず施錠する．
- 自分の部屋の外では鍵やカードや認証器は肌身離さず持っている．

- 他人がいる可能性のある場所でデジタル器機を離れるときは画面をロックして，パスワード・生体認証がなければ使えないようにする．
- 機密情報の入った USB メモリーは極力持ち歩かない．保管場所には施錠する．
  その組織のロゴなどの付いた USB メモリーを駐車場などにわざと落として，それを拾わせて内部のパソコンにウイルスを感染させる攻撃もある．
- 次のような古典的な攻撃にも気を付ける：
  - ショルダーハッキング：肩越しに暗証番号を盗み見
  - トラッシング：ゴミ箱あさり
  - スキミング：クレジットカード等の情報の電子的な盗み取り
  - ピギーバック (piggyback), 共連れ：鍵のない人が鍵を持った人と鍵が無いと入れないところに一緒に入ること
    ただし，目の前に鍵がなくて困っている人がいたら邪険にすることも出来ないので，声を掛けて顔を確認して入ってもらうことにしても良いだろう．
- 自分や知人の写真・PID・マイナンバー・暗証番号・生体認証情報をみだりに公表しない．
- 写真の日付・GPS 情報などは不用意に公表しない．また映り込みから場所を特定されることがあるので，それにも十分に気を付ける．ネットで「特定屋」と検索すると数多く見つかる．
- SNS の個人情報はダダ漏れなので個人情報はあげない．
  例えば旅行中の写真をその時点でインターネットにアップすると，その時家にいないことを泥棒に知らせてしまう危険がある．
- 紛失防止タグを使った家の特定：ストーカーなども使っている．
- スマホをオンにしての盗聴が可能になっている．秘密の会議の時はスマホを預けるなどのセキュリティ対策が必要になっている．会議室の最大限のセキュリティを保つには電磁シールドの敷設も必要だろう．
- ランサムウエア攻撃でバックアップも同時に暗号化されてしまいデータを復元できないことが頻繁に起こっている．ネットに繋がっていないデバイスへの定期的なバックアップをお奨めする．
- 政治家や企業経営者はパワハラ・セクハラ・不倫・週刊誌の取材・税務調査・スパイに気を付ける．
- 
  - 某国は闇バイトによる (空港近くでドローンを飛ばして飛行機の運航を妨害するなどの) 使い捨て工作を行っていると聞く．
  - Google アースやストリートビューの情報から物理的な攻撃を考える攻撃者もいる．例えば，駐車場に止まった業者のトラックの情報を使ってその施設への侵入

を試みるという攻撃が知られている．
セキュリティ責任者はそのような観点にも注意が必要．

**演習 32**

- スマホの写真の位置情報・時間情報を確認・削除する方法を調べてください．他人と写真を共有するときはこのようなメタ情報を消すことをお奨めする．
- また，スマホやパソコンは貴方の個人的な情報を常に収集している．(私の iPhone だと) Setting/Privacy and Security のアプリとアップルのアクセス権を必要最小限に設定してください．
- Google などは Cookie を使って皆さんの検索履歴などから情報を分析し，インターネットでオークションに掛け，巨万の富を得ている．自分が使っている Browser 毎に Cookie を消す方法を調べてください．定期的に消すことをお奨めする．
- Google ストリートビューで自分の家・実家がどう写っているか確かめてください．危険性が確認できた場合対策を取ってください．

ソーシャルエンジニアリング：人間の心理的弱点を突く攻撃

- 不審な直接のコンタクト・電話・メールに気を付ける．
- 不審でないと思われる直接のコンタクト・電話・メールにもソーシャルエンジニアリングでないかと気を付ける．
- Web ページのハイパーリンクの下線や QR コードは見た目だけからは何処へのリンクか分からないので，軽率にそのリンクに飛ばず，リンク先を確認してから訪問する．
- AI によるソーシャルエンジニアリングの高速化
- 次のような情報収集が行われている：
  - フィッシングなどネットを使った違法な情報収集
  - 個人情報を調べるための景品付きアンケートなどの合法な情報収集 ([3])
  - 格安のスマートテレビ (Android TV) に搭載された ACR (=Automated Content Recognition 自動コンテンツ認識) による情報漏洩
  - イランへの攻撃では，モサド・アマン (8200 部隊) が監視カメラをハッキングしたり，攻撃先に秘密カメラを仕込んだりしていたことが明らかになった．
  - 一部の国は国家情報法を持ち，海外の自国民を監視するため外国に秘密警察を持っている．他国民を監視していない保証もない．
- エプスタイン事件からも分かるようにハニートラップも現実に行われている．裏に他

国の諜報機関がいたとも言われている．某大統領はその国に弱みを握られているという噂もある．

そのような罠に気を付ける．

- 最近の事例：
  - 個人の生成 AI への機密情報の提示，漏洩
  - BeReal などからの機密情報の提示，漏洩
  - 認知戦・情報戦：最近日本でも知事選や総選挙で広く攻撃が行われていたようである．

**演習 33**　Web ページのハイパーリンクの下線や QR コードで文字でリンクを表示する方法を調べてください．これからは毎回クリックする前にチェックすることをお奨めする．

[8] には Kali Linux やネットを使ったソーシャルエンジニアリングの具体的手法の解説があるので是非参考にしてほしい．ただし，これは攻撃のマニュアルともなるのでそこには十分注してほしい．

**演習 34**　Have I Been Pwned を使ってメールアドレスに付随した password が Dark Web に漏洩していないかチェックしてください．もし漏洩が確認できた場合対策を取ってください．まめにチェックすることをお奨めする．

# 第 6 章　情報・社会・AI・健康

## 6.1　情報とは

Data(データ) は数字や文字や図形などで表されたもの，Information (情報) はデータに意味を持たせたもの，Knowledge(知識) は情報を解釈して体系化したもの，Wisdom(知恵) は知識を体得して人生に役立つものとしたものである．DIKW ピラミッドとはそれらを以下のように図式化したものである：

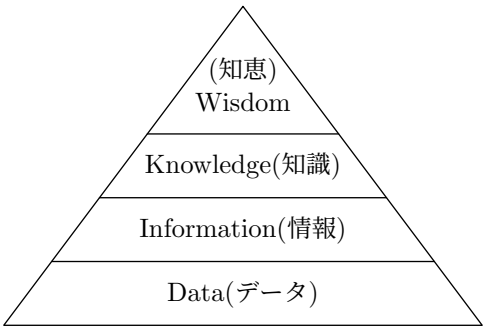



**図 6.1　DIKW ピラミッド**

例えば生成 AI にレポートを丸投げして内容を確認せず提出するのは，情報のみ得ることに対応する．出てきたレポートを自分なりに解釈し自分の言葉で喋れるようにするのは知識の習得にあたる．生成 AI と壁打ちしながら自分に欠けていた視点からの考察を深め，自分だけでは書けなかったレベルのレポートを AI と共同で作成するのは知恵を得る使い方である．

## 6.2　情報社会と法律

情報社会と情報セキュリティーに関係する日本の法律：

- 知的財産権
- 著作権
- 個人情報保護法

- 不正アクセス禁止法
- サイバーセキュリティ基本法
- 電気通信事業法
- 電子署名及び認証業務に関する法律
- 電波法
- 特定電子メールの送信の適正化等に関する法律
- 有線電気通信法

## 6.3 AI の発達の歴史

- ロジックセオリスト：1955 年から 1956 年にかけてアレン・ニューウェル，ハーバート・サイモン，J・C・ショーが開発した，論理推論を真似するよう設計された世界初のプログラム．
- イライザ ELIZA：1964 年ワイゼンバウムが開発した対話システム．アルゴリズムは簡単だったが，これに夢中になる人が続出した．
- エキスパートシステム
- neural network ニューラルネットワーク，machine learning 機械学習，deep learning 深層学習などのモデルが研究されるようになった．
- ディープブルー： 1997 年チェスの世界チャンピオンカスパロフに勝った
- ワトソン： 2011 年クイズ番組ジョパディで人間に勝利
- AlphaGo 2016 年碁で名人に勝利，GAN =Generative Adversarial Network：敵対的生成ネットワークが使われた．
- LLM =large language model 大規模言語モデルが発達し Transformer が開発され，GPT =Generative Pre-Trained Transformer が開発された．2022 年それを対話的に使う ChatGPT が発表され世界に生成 AI ブームをもたらし，株も AI バブルの様相を示し現代に至る．

## 6.4 情報社会と心と体の健康

最近，体やメンタルの不調を訴える人・社会的な問題を起こす人が増加している．

### 6.4.1　データ

1. 2023 年のネット利用時間は高校生が 6 時間 14 分，中学生が 4 時間 42 分，小学生（10 歳以上）が 3 時間 46 分だっただった[1)].
2. 2024 年の裸眼の視力が 1.0 未満の割合は高校生で 71 %，中学生で 60 %，小学生で 36 %だった[2)].
3. 2021 年度中等度以上のうつ症状がある子どもの割合が高校生 30 %，中学生 24 %であると発表された[3)].
4. 小中高校生の不登校の人数が 2023 年には過去最多の 34 万人を越えた[4)].
5. 2024 年オーストラリアでは 16 歳未満の SNS を禁止する法律を定めた.
6. 警視庁が 2025 年に公表したアンケート調査によると国内のオンラインカジノ利用経験者は約 337 万人，そのうち約 200 万人はギャンブル依存症を自覚していた.
7. トクリュウ全体の年間摘発者数うち「闇バイト」応募・関与者の人数は 2023 年 10,105 人中約 3,925 人，2024 年 12,178 人中 4,759 人だった.

### 6.4.2　対策の前提となる考え方

1. 科学的な（実験や統計や論理や数式で白黒付けられるような）反証可能 ([38]) な議論・対話可能な議論を行う.

   具体的対処の有効性は体の状態・症状・心の状態の改善度から評価する．特に，排便の状態，肌の状態や体の痛み，疲労感・睡眠不足感・気分をモニターする.

   自分のことは自分では分かりづらい．例えば目の 10cm 前でスマホを見ている人がいたら自分のスマホを目から離す行動をとるなど，他人を詳細に観察しそれが自分に当てはまらないか思い返してみるのは良いことである.
2. 進化生物学：ヒトが農耕を始めてから未だ 400 世代程しかたっていないので，それ以来の生活の変化に我々の体は適応していない．ヒトが適応している 1・2 万年前の生理学的環境・状態と近い（逆に人工的な環境の）生活を送る必要がある ([37], [39]).
   (a) ヒトは狩猟採集生活に適応しているが，運動しない生活，座り続ける生活，夜寝ない生活，長時間近くを見続ける生活，イヤホンを使い続ける生活には適応していない.

---

1) こども家庭庁 2023 年度のインターネット利用環境実態調査
2) 文部科学省 2024 年度学校保健統計
3) 国立成育医療研究センター「コロナ×こどもアンケート」第 4 回調査報告
4) 文部科学省 令和 5 年度児童生徒の問題行動・不登校等生徒指導上の諸課題に関する調査

(b) ヒトは狩猟採集生活の雑食の生活に適応しているが，多様な食物を摂らない生活，超加工食品には適応していない．

(c) ヒトはスマホを使い続ける生活には適応していない ([11], [16], [32], [34], [47]).

3. 動物は環境に合わせて行動を変えるように進化してきた．これを制御しているのが脳の報酬系[5]から放出されるドーパミン[6]であり，ドキドキわくわくする感情を引き起こし，我々が新しい行動を行ったり新しい情報を得るのを促進するように働いている．ドーパミンはアドレナリンに代謝され攻撃性を増す効果もある．

   スマホ（のアルゴリズム）はドーパミンを継続的に大量に放出させ，使用者を依存症にするように作られており，それを作っている会社はそこから巨万の富を得ている．ただし，ドーパミン自体は善でも悪でもない．ドーパミンを上手くコントロールして，なりたい自分に近づいてほしい ([40]).

4. 我々のデジタルの行動データはブラウザーの Cookie などに蓄積されていて，それが売買されたり，アルゴリズムがフィルターバブル[7] ([55]) ，SNS がエコーチェンバー現象[8]を引き起こしている．更に，最近ではこれらの現象に加えてアテンションエコノミーの拡大[9] や諸外国からの情報戦・認知戦により誤情報・偽情報が生み出され，それらが生成 AI により著しく増幅されている．それらにさらし続けられることで，それらの中に所謂「真実」を見つけ陰謀論に絡め取られる人々が増えている ([46], [50]). 陰謀論に惹かれることもドーパミンの放出と関連している．

5. それらによりトランプ大統領のようなポピュリズム政治家が世界中に台頭している．

---

5） 1954 年オールズらにより最初に発見されたときは快楽中枢と呼ばれていたが，後に快楽ではなく，新しい行動や情報への報酬を与えていることが明かになり，報酬系と呼ばれるようになった．

6） 現在では脳内物質で説明されているが，古くからスキナーによる小動物を使った実験から現象としては知られていた．そのため，スマホをスキナー箱という研究者もいる．この心理学の分野を行動分析学 [93] という．

7） 「インターネットの検索サイトが提供するアルゴリズムが，各ユーザーが見たくないような情報を遮断する機能」（フィルター）のせいで，まるで「泡」（バブル）の中に包まれたように，自分が見たい情報しか見えなくなること．(Wikipedia 2024/04/07, 18:34)

8） 自分と似た意見や思想を持った人々の集まる空間（電子掲示板や SNS など）内でコミュニケーションが繰り返され，自分の意見や思想が肯定されることによって，それらが世の中一般においても正しく，間違いないものであると信じ込んでしまう現象．(Wikipedia 2024/04/07, 18:35)

9） 情報の優劣よりも「人々の関心・注目」という希少性こそが経済的価値を持つようになり、それ自体が重要視・目的化・資源化・交換財化されるようになるという実態を指摘した概念．(Wikipedia 2025/03/21, 8:12)

### 6.4.3　デジタルデトックス

1. 生活改善：現在の生活はデジタルを除いても昔はなかった危険に囲まれている．もちろんスマホ＞ストレートネック＞肩こりのように直接の関連がある場合もあるので，どちらにしてもここから改善する必要がある．
   (a) バランスのとれた（多様性の高い）質の高い食事を適量，よく噛んで食べる．
      i. 野菜・果物・豆類・ナッツ・魚・肉・タマゴ・乳製品・発酵食品・ビタミン・ミネラルをバランスよく摂る．
      ii. コンビニで売っているような食品・ファストフードなどの超加工食品を食べない．これらにもドーパミンを放出させる働きがあり強い依存性を持っている．例：野菜ジュース・フルーツジュース・清涼飲料・カップ麺・ポテトチップス・揚げもの・加工肉・コンビニ弁当・サンドイッチ・ハンバーガー

         ただし，時たま友達や家族と一緒にこのようなものを食べるのは特に問題ない．逆に，他人に自分と同じ基準での食生活を求めるのは現実的でなく，場合によっては人間関係をこわすのでお奨めしない．
      iii. 毎日朝食をとる．

         例：プチトマト・スプラウト・ゆで卵・納豆・豆腐・バナナ・ミカン・ヨーグルト・ゴマ・お酢・アマニ油・スープ・緑茶・紅茶・コーヒーの中から何点かを適宜組み変えて摂る[10)]．
   (b) バランスのとれた質の高い運動を適量，正しい姿勢で行う．

      注意点：口呼吸の矯正，猫背・巻き肩・ストレートネックの矯正，肩甲骨の可動域の確保，反り腰の矯正，体の左右差の矯正，アキレス腱の柔軟性・アーチの保全・足指可動域の確保．

      i. 毎日 1, 2 時間ほど日光を浴び遠方を見ながらのウォーキング・自転車・スポーツを行う．
      ii. 長時間椅子に座り続けない．階段を積極的に使う．
      iii. 隙間時間はスマホは見ず，片足立ち・発声練習・ストレッチ・足指のストレッチなどを行う．
   (c) 質の高い睡眠と休養を適量，規則正しくとる．

10)　基本パッケージ化されていて冷蔵庫を出して直ぐに食べられるもので (i) の要素を多数含むように選んでいる．

i. 毎日 8 時間以上の規則正しい睡眠.

ii. 毎日朝日を浴びる.

(d) バランスのとれた質の高い，社会活動を含む高度な知的活動を適量行う.

(e) 昔（殆ど）存在しなかった化学物質への暴露を避ける：

果糖ブドウ糖液糖・砂糖・人工甘味料・大量のサラダ油 ([14])・トランス脂肪酸・ラクトアイス・合成洗剤 ([69])・PFAS・マイクロプラスチック・サプリ・抗生物質 ([4])・薬など

(f) 我々の世界認知がアルゴリズムや金儲けや敵国や生成 AI から影響を受けていることを認識し，自分の認知が歪まされていないか常に検討することが必要である.

これらはできる限り続けることをお奨めする.

2. デバイスとの付き合い方：

(a) 様々な通知機能は絶対外せないものを除いて停止.

(b) メール，SNS，ニュースの確認は一日数回に限定．週末は見ないと決めて知人に知らせるのもいい.

(c) 集中しないといけないときはデジタルデバイスの電源を落とす.

(d) スマホを含めデバイスを見るときはなるべく大きな外付けモニターを使い，表示フォントも大きなものを使う．モニターはなるべく遠くに置き，目に過度な負担をかけないようにする．iPhone では Setting /Screen Time/Screen Distance で画面との距離の警告を設定できる．また，イヤホンやヘッドホンではなく外付けのスピーカーを使い，他人にうるさくない音量に保つ．悪い姿勢を続けないように気を付ける.

目も耳も体も（頭脳も）一生もの，大切に扱ってください.

(e) まめに Cookie のリセットなどを行ってフィルターバブルを防ぐことをお奨めする．詳しいやり方はブラウザー毎に検索するように.

(f) プログラミングでデジタルデトックス：プログラミングやデバッキング，文書をまとめたりプレゼンの準備をすること ([58]) はアナログな作業．コードを書く前に，アナログでどれだけ綿密に計画を立てられるかでプロジェクトとその保守拡張の成功が決まる．データ構造とアルゴリズムをよく検討し，必要に応じて手で図を作成し，場合によっては数学的な性質を証明する必要がある．また，コーディングではパソコンでの作業が増えてスマートホンの小さな画面から離れられる.

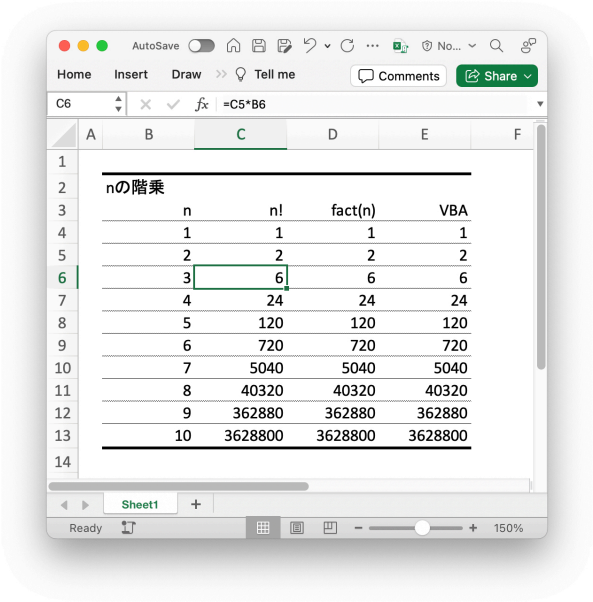
私の経験からのアドバイス：プログラミングに集中するとデジタルに距離が置けるようになる.

# 付録 A　Excel の関数と VBA

## A.1　Excel の関数と VBA

### A.1.1　Excel の関数でのプログラミング

Excel の関数によるプログラミングではそれなりに色々なことが出来るし，仕事ではデファクトスタンダードなので使わざるをえない．幾つかの方法で階乗の計算をしてみる．




| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | | | | | | |
| 2 | | nの階乗 | | | | |
| 3 | | n | n! | fact(n) | VBA | |
| 4 | | 1 | 1 | 1 | 1 | |
| 5 | | 2 | 2 | 2 | 2 | |
| 6 | | 3 | 6 | 6 | 6 | |
| 7 | | 4 | 24 | 24 | 24 | |
| 8 | | 5 | 120 | 120 | 120 | |
| 9 | | 6 | 720 | 720 | 720 | |
| 10 | | 7 | 5040 | 5040 | 5040 | |
| 11 | | 8 | 40320 | 40320 | 40320 | |
| 12 | | 9 | 362880 | 362880 | 362880 | |
| 13 | | 10 | 3628800 | 3628800 | 3628800 | |
| 14 | | | | | | |

**図 A.1　階乗計算**

図 A.1 の 2 行目はセルの間の掛け算を使った関数計算である．セルあるいは数式バーに数字を普通に入力し，リターンキーを押すと通常の設定だとセルの右寄りに表示される．数式バーでは文字列は ' の後に続け，リターンキーを押すと通常の設定だと左寄りに表示されるが，ここでは右寄りになるように表示している．数式バーでの数式を入力するにはキーボードから = を入力する．セルの値は B5 のように参照され，セルをクリックすると = の後に自動で入力される．上の場合セル C6 には =C5*B6 が入力されていて，そ

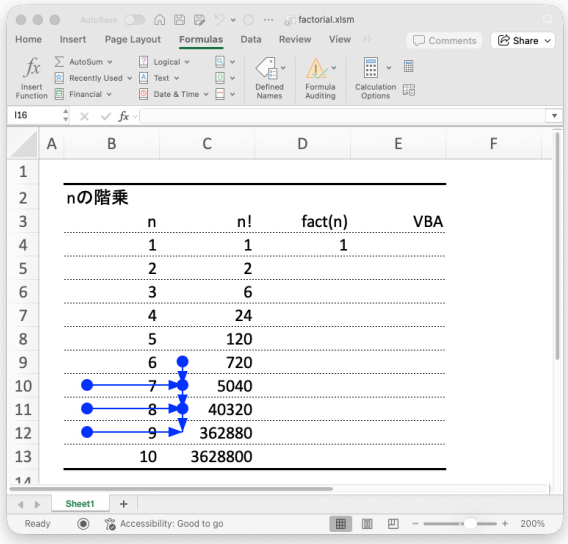



**図 A.2　Trace Precident の (何度かの) 実行**

こでリターンキーを押すと計算した値がセルに現れる．ただし，* は積を表す記号でキーボードから正しい位置に入力する必要がある．

一個づつ入力するのは大変で間違いが起こりやすいので以下のようにする：C5 に正しい式を書いて，ここを選択し，選択の部分を少し動かすと十字の記号があらわれるので，そのまま，下までドラッグすると正しい式がコピーされる．これは相対参照という機能であるが参照先を動かしたくないときは=$C$5*B6 のような絶対参照を使う必要がある．ここはよく間違えて微妙に狙ったものと異なる結果になり，慣れない人は途方に暮れるところでなので注意してほしい．Formula のタブに Trace Precident のボタンがあるのであるセルがどこを参照しているかを矢印で確認できる (図 A.2)．我々の例は小さすぎてあまり役立たないが，複雑な参照では視覚的に確認が出来て便利である．

3 行目では繰り込み関数 Fact(n) を使って計算している．計算的にはどの方法でやっても同じであるが，やりたいことが既に繰り込み関数で実現されていることも多いので必要に応じて Help やインターネットの検索等で探してほしい．

### A.1.2　VBA でのプログラミング

次に 4 行目で行っている VBA を使った計算を説明する．VBA とは Visual Basic for Applications の略で Excel・Word のスクリプト言語で，Basic から派生したものである．Office に標準搭載されているので，自分で開発環境を導入する必要が無いのは初学者にとっては魅力的である．

Excel のメニューバーの Excel メニューから Preferences を選び，Preferences のパネルで

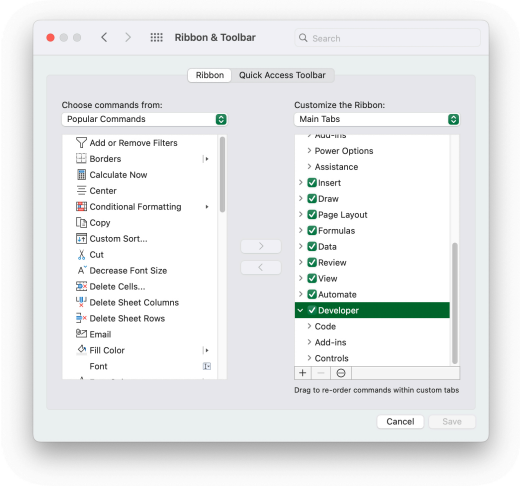



**図 A.3　Excel の設定画面**

Ribbon & Toolbar ＞ Ribbon ＞ Main Tabs で Developer をチェック．閉じてファイルを見ると Developer タブが見えるようになる：

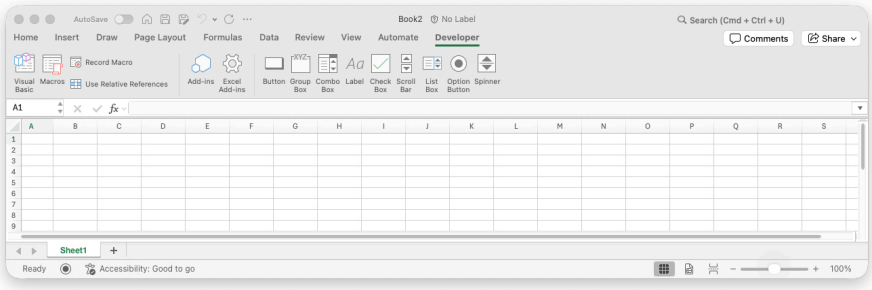



**図 A.4　Developer のタブを押した後**

Developer タブを選び左側の Visual Basic のボタンをクリックすると VBA の開発環境が開く．左側の上の窓がプロジェクトエクスプローラー，その下がプロパティウインドウ，左側の上の窓がコードウインドウだが最初は表示されていない (図 A.5)．その下がミデエットウインドウであり，ここに出力することでプログラムの確認が出来る．

メニューバーの insert の Module を選ぶと，プロジェクトエクスプローラーに Module1 が追加され，コードウインドウが使えるようになる (図 A.6)．

次に開くとき VBA を有効にするためには fuge.xlsm という拡張子で保存する必要ある．また，そのファイルを開くときは VBA を使用して良いかの確認アラートがでるので，自分で作ったファイルの場合 Enable Macro のボタンを押してほしい．ただし，他人の作った.xlsm ファイルのときは十分注意が必要である．

先程の例で VBA のコードをあげる (図 A.7)．

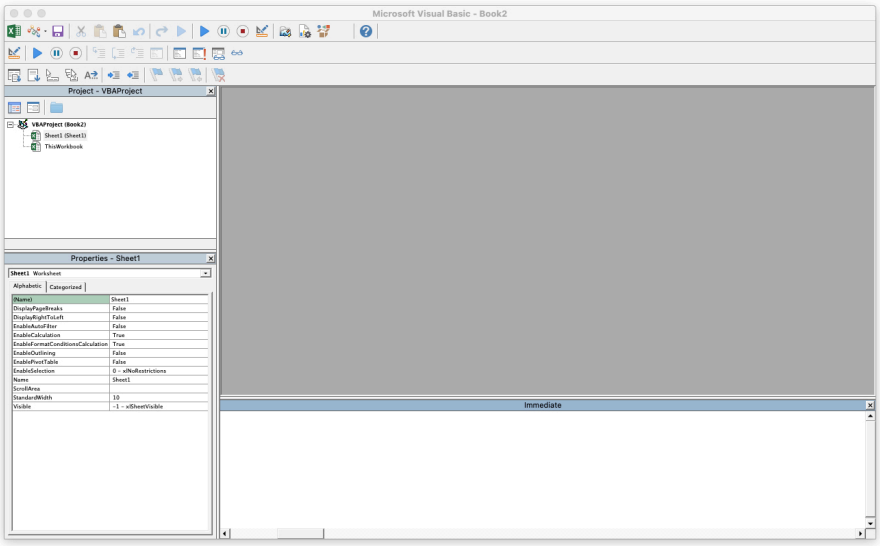



**図 A.5　Visual Basic のボタンを押した後**

まず，変数の宣言を強制する設定にすることが推奨されているので，メニューバーの Excel のメニュー Preferences を開き，editer の option の二つ目の部分にチェックを入れておく．この設定が有効になっていると Option Explicit と示されている．

プログラムに戻る．’ がコメントの記号で，この部分はプログラムに何の影響も与えないが，将来の自分・他人が理解し易いようにコメントを書くように努めてほしい．

文の後に ; は書かない．

Sub のあとに関数の名前を書く，この場合は factorial() で必ず () が必要で，(var1, var2) のようにして変数を入れることで関数への変数の受け渡しが出来る．次の行から End Sub の間に定義を書く．これらは自動補完されるので正しい入力をすれば改行で自動に現れる．Dim が変数の宣言で As の後に型の宣言を行っている．ここでは long は長い整数を表す．maxN は定数のつもりで a, n が計算に使う補助的な変数である．

```
For n = 1 To maxN
    a = a * n
    Cells(n + S, E).Value = a ’ex C(1+S,E) = E4 cell
Next n
```

が VBA の for 文で n = 1 から maxN まで a に n を掛けた a * n を a に書き換えそれを Cells(n + S, E) に書いている．ただし，S = 3，E = 5 である．この表現 Cells(n + S, E) は行列と同じで (n + S, E) 成分，つまり，n+3 行 5 列の成分を表す．.Value はこのセルのプロパティ Value , つまり，値を表している．様々な method （Obj.hoge() のようなもの）も用意されているがオブジェクト指向のプログラミングをしているというより，Excel の

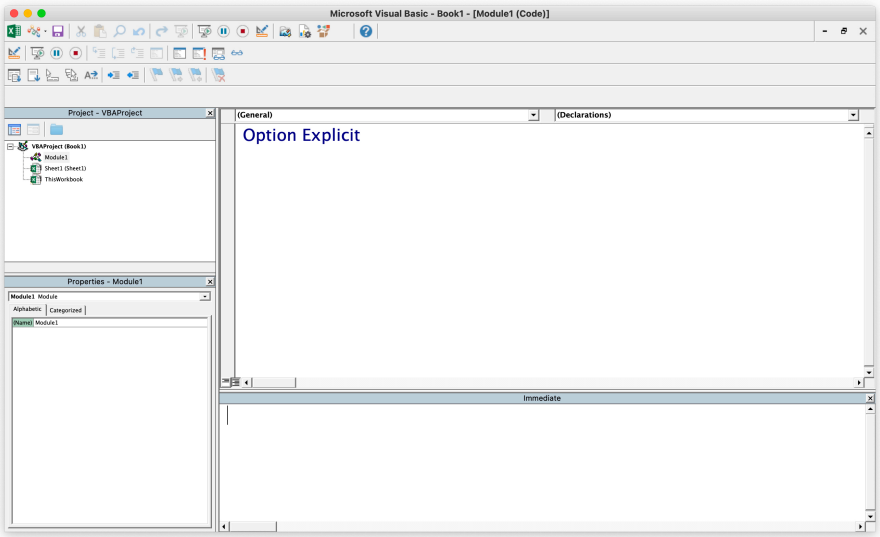



**図 A.6　Module の追加後**

book がオブジェクトの塊となっているので，VBA からそれを操作しているという方がもっともらしい．

プログラムを実行するにはメニューバーの Run のメニュー Run Sub/UseForm を選択すると実行され，シートの E 行に結果を書き出しているはずである．あと最低必要な文は if 文で，例えば

```
Sub myfun()
    If Cells(1, 1).Value < 60 Then
            Cells(1, 2).Value = "F"
    ElseIf Cells(1, 1).Value < 70 Then
        Cells(1, 2).Value = "C"
    ElseIf Cells(1, 1).Value < 80 Then
        Cells(1, 2).Value = "B"
    ElseIf Cells(1, 1).Value < 90 Then
        Cells(1, 2).Value = "A"
    Else
        Cells(1, 2).Value = "S"
    End If
End Sub
```

のようにすると Cells(1, 1) の値に応じて Cells(1, 2) に成績を記入することが出来る．

他にも色々な機能があるが，取りあえず今説明したことで VBA を動かすことが出来る

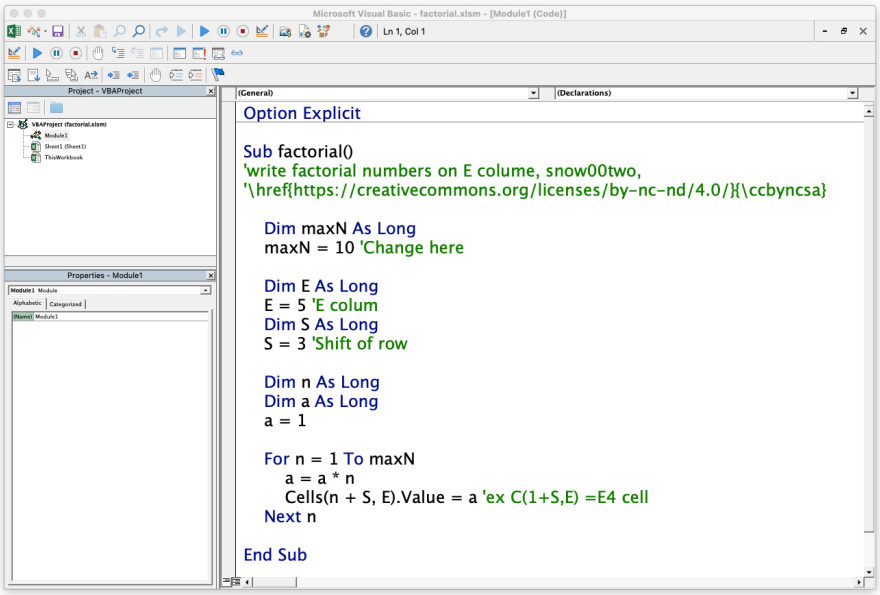



**図 A.7　factorial() のコード**

であろう．外部ファイルとの入出力も可能なので，Excel の関数では実現できないようなプログラムも作ることが出来る．上でも注意したがウイルスのような破壊的なプログラムを書くことも可能であるので注意が必要である．色々調べてほしい．

**演習 35**　Excel の参照機能を使って Fibonacci 数列を書き出せ (図 A.9)．また，同じことを VBA を使って書き出せ．余裕のある人は Excel のシートから幾つ出力するか制御出来るようにシステムを変更せよ．

**演習 36**　Excel の参照機能を使って Pascal の三角形を書き出せ (図 A.10)．また，同じことを VBA を使って書き出せ．余裕のある人は Excel のシートから幾つ出力するか制御出来るようにシステムを変更せよ．

**宿題 6**　ここまで作ってきたプログラムを生成 AI に Python や JapaScript (+node.js ) のプログラムに書き換えてもらい，必要に応じて訂正しそれを実行せよ．

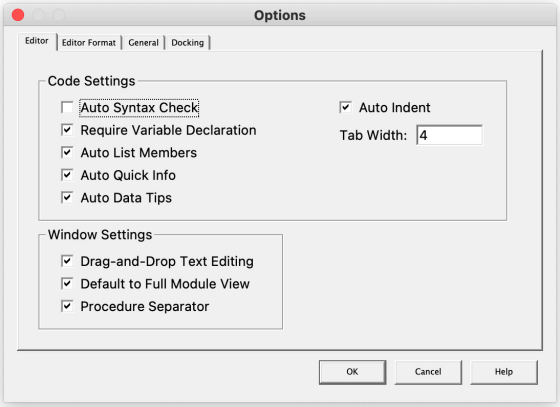



**図 A.8　VBA の設定画面**

### A.1.3　関数によるデータ分析

Excel 関数のもう少し詳しい使い方とデータ分析，グラフなどによる可視化について解説する．先程も説明したが，まずセルを選ぶと数式バーに入力出来るようになる．=を入力すると数式を入力出来るようになる．ここで他のセルをクリックするとそのセルの内容を参照できることになるし，関数を書くとその関数が使えるようになる．

図 A.11 の成績集計用の Excel シートを用いて簡単なデータ分析について解説する：連続したセルの指定は C2:C10 や C 列を表す C:C で行う．例えば C4 から C15 までの平均を求めるには C16 に

```
=AVERAGE(C4:C15)
```

と書けばいい．あるいは関数を書いて C4 から C15 までのセルをシフトキーを押しながら選ぶと自動で挿入される．

ヒストグラムを書くために H4 に

```
=FREQUENCY(C4:C15,F4:F15)
```

と入力してリターンキーを押すと H4 から H15 に頻度が計算される．この内容からグラフを作るとヒストグラムが出来る．ただし，これにはそれなりの設定が必要である (図 A.12, [79] 12 章参照)．H4 から H15 までの総和をもとめるには H16 に

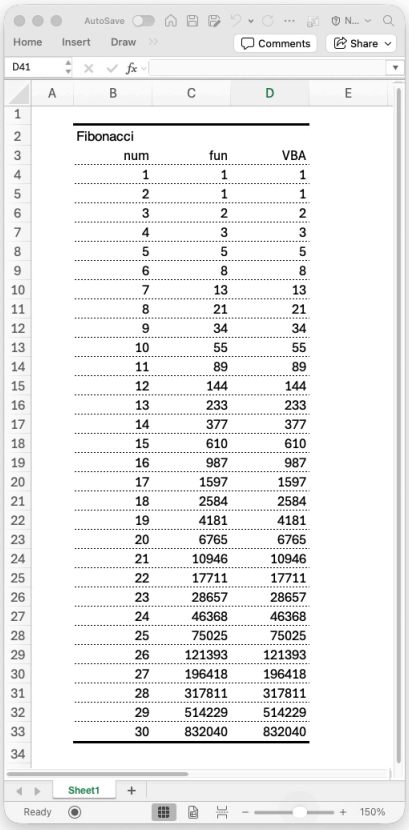
```
=SUM(H4:H15)
```

と書く．これは集計が確実に行われているかのチェックにも成る．

条件分岐は D4 の内容のように

```
=IF(C4>=$K$4,$J$4,IF(C4>=$K$5,$J$5,
IF(C4>=$K$6,$J$6,IF(C4>=$K$7,$J$7,$J$8))))
```

とすれば良い．その後十字の記号を下までドラッグすると正しい式がコピーされる．絶対



| Fibonacci | | |
|---|---|---|
| num | fun | VBA |
| 1 | 1 | 1 |
| 2 | 1 | 1 |
| 3 | 2 | 2 |
| 4 | 3 | 3 |
| 5 | 5 | 5 |
| 6 | 8 | 8 |
| 7 | 13 | 13 |
| 8 | 21 | 21 |
| 9 | 34 | 34 |
| 10 | 55 | 55 |
| 11 | 89 | 89 |
| 12 | 144 | 144 |
| 13 | 233 | 233 |
| 14 | 377 | 377 |
| 15 | 610 | 610 |
| 16 | 987 | 987 |
| 17 | 1597 | 1597 |
| 18 | 2584 | 2584 |
| 19 | 4181 | 4181 |
| 20 | 6765 | 6765 |
| 21 | 10946 | 10946 |
| 22 | 17711 | 17711 |
| 23 | 28657 | 28657 |
| 24 | 46368 | 46368 |
| 25 | 75025 | 75025 |
| 26 | 121393 | 121393 |
| 27 | 196418 | 196418 |
| 28 | 317811 | 317811 |
| 29 | 514229 | 514229 |
| 30 | 832040 | 832040 |

**図 A.9　Fibonacci 数列**

参照と相対参照の使い方に注意してほしい．

L4 のように

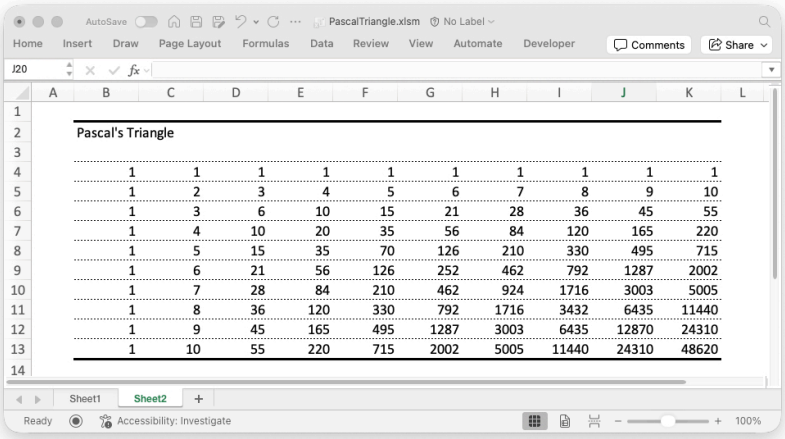
```
=COUNTIF($D$4:$D$15,J4)
```

とすると集計がされる．これも十字の記号を下までドラッグ．集計はヒストグラムにまとめてある (図 A.11 参)．

あと，スタイルの問題だが手入力部分の数字のフォントは青で，いつもの固定の部分と自動計算のところは黒になるように字の色が設定してある．ここでは個別の成績部分と最終成績の閾値の部分がそれに当たる．

### A.1.4　歴史と参考文献

IBM PC 市場では Lotus 1-2-3 が覇権を握っていたが，1984 年の Macintosh の発売に合わせて Microsoft は Macintosh 版 Excel を開発し 1985 年に発売を開始した．その後 1985 年 Microsoft は Windows 1.0 を開発し，Macintosh 版 Excel を Windows 版に移植し 1987 年に発売を開始した ([23] の 16 章にはこれらのことと S. Jobs と B. Gate の関係につい




| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | | | | | | | | | |
| 2 | Pascal's Triangle | | | | | | | | | |
| 3 | | | | | | | | | | |
| 4 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 5 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 6 | 1 | 3 | 6 | 10 | 15 | 21 | 28 | 36 | 45 | 55 |
| 7 | 1 | 4 | 10 | 20 | 35 | 56 | 84 | 120 | 165 | 220 |
| 8 | 1 | 5 | 15 | 35 | 70 | 126 | 210 | 330 | 495 | 715 |
| 9 | 1 | 6 | 21 | 56 | 126 | 252 | 462 | 792 | 1287 | 2002 |
| 10 | 1 | 7 | 28 | 84 | 210 | 462 | 924 | 1716 | 3003 | 5005 |
| 11 | 1 | 8 | 36 | 120 | 330 | 792 | 1716 | 3432 | 6435 | 11440 |
| 12 | 1 | 9 | 45 | 165 | 495 | 1287 | 3003 | 6435 | 12870 | 24310 |
| 13 | 1 | 10 | 55 | 220 | 715 | 2002 | 5005 | 11440 | 24310 | 48620 |
| 14 | | | | | | | | | | |

**図 A.10　参照機能を使った Pascal's triangle**

ての記述がある)．VBA の起源はもともとは 1964 年に J. Kemeny と T. Kurtz によって発表された BASIC =Beginner's All-purpose Symbolic Instruction Code であるが，1991 年に Microsoft がオブジェクト指向の考えを取り入れた Visual Basic 1.0 を発表し，1994 年に VBA =Visual Basic for Application を開発し Excel 5.0 に搭載されたのが始まりである ([71])．

参考文献について：[73] は Excel の関数でどのようなことが出来るかを分かり易く解説したもの．[35] はこの資料であげた図 A.1, A.9, A.10 のような見栄えの Excel を作るにはどうすればいいかを解説したもの．[36] は VBA について非常に短いが分かり易く解説したもの．[65] は VBA について非常に詳しく解説したものである．Excel の機能は膨大なので，辞書として [78], [79], [80] も必要となるだろう．

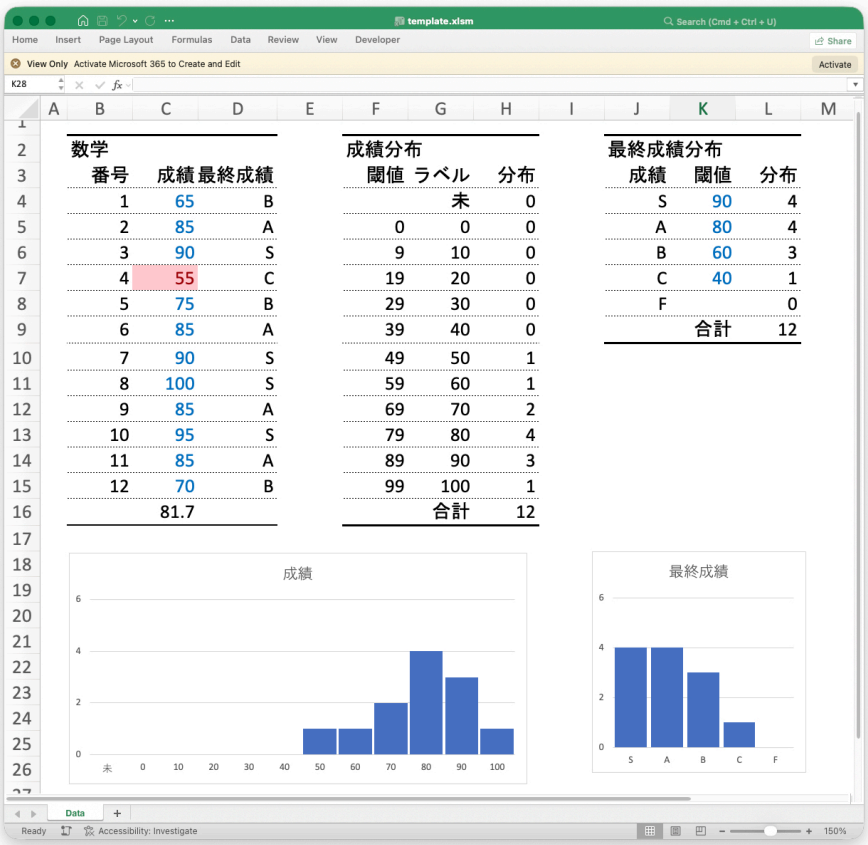



**図 A.11　データとヒストグラム**

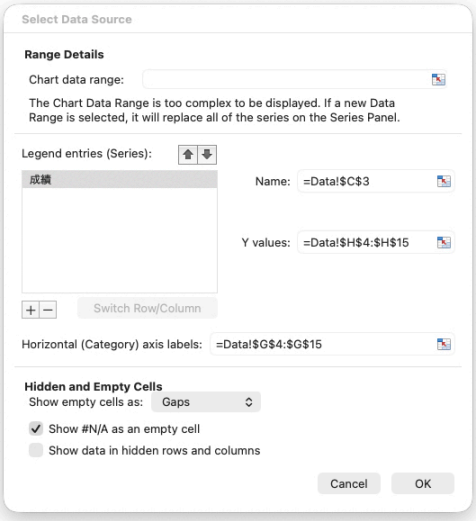



**図 A.12　Graph のデータ**

# 付録 B　TeX と Markdown

## B.1　TEX

### B.1.1　TEX のインストール

TEX とは:

- TEX: フリーで使える数式を含む文書作成言語 (compiler).
  D. Knuth により開発された.
- LATEX: L. Lamport が TEX を使いやすくしたマクロパッケージ．数式・相互参照・表・作図・プレゼン・hyperlink などが使える
- 高校の先生・高校生でも使ってる人がいるらしい.

英語表示の Mac の場合のインストールパッケージ:

- MacTeX: mac 用 installing package （以下を全て含んでいる）
- TeXLive: TEX の本体, mac だと /usr/local/texlive にインストールされる.
- TeXShop: TEX の mac 用の editor （開発環境で, VSCode などでも使用可能）
- beamer: プレゼンのための LATEX の documentclass
- METAPOST: TEX の描画ソフト (font 作成ソフト METAFONT([33]) から派生した描画言語)

**演習 37**　LATEX のインストール（Mac の英語表示のみ）:

1. google で MacTeX を検索
2. MacTeX をダウンロード
3. MacTeX.pkg をダブルクリックし，インストーラーの指示に従いインストール.
4. texshop で 設定が必要かも知れない．詳細は texwiki を参照

texlive のファイル群は/usr/local/texlive という不可視フォルダーにインストールされて

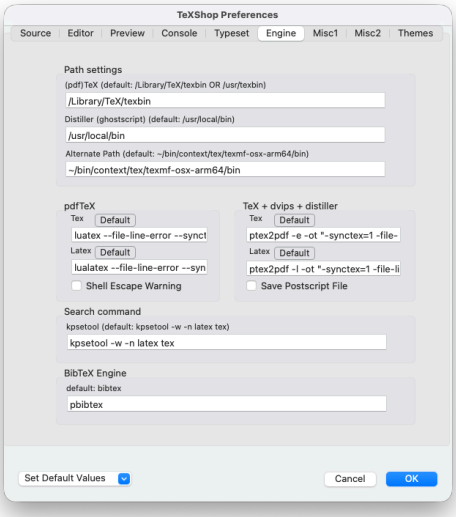



**図 B.1　texshop の設定**

いる．コマンドラインユーザーインターフェイスでは簡単にアクセスできるが，（Mac の）グラフィカルユーザーインターフェースでは Finder のメニューバーの go のメニューの Go to Folder を選ぶとフォルダーの選択窓が開くので/usr/local/texlive と打ち込むと移動できる．

Windows などの場合には [86] などを参考にしてインストールして下さい．

### B.1.2　TEX のタイプセット

次のソースファイル

```
\documentclass[11pt]{article}
\usepackage{graphicx, xcolor}
\usepackage{latexsym}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{amsthm}
\renewcommand{\figurename}{図}
\renewcommand{\proofname}{証明}
\renewcommand{\tablename}{表}
\DeclareMathOperator{\const}{const}
\title{Hello world \TeX !}
\begin{document}
```

```
\maketitle
```

**因数分解** `$x^3-1=(x-1)(x^2+x+1)$`.

**不定積分**

```
\[
   \int \sin x\, dx =-\cos x,\quad
   \int \frac1xdx=\log |x|
\]
```

**行列**

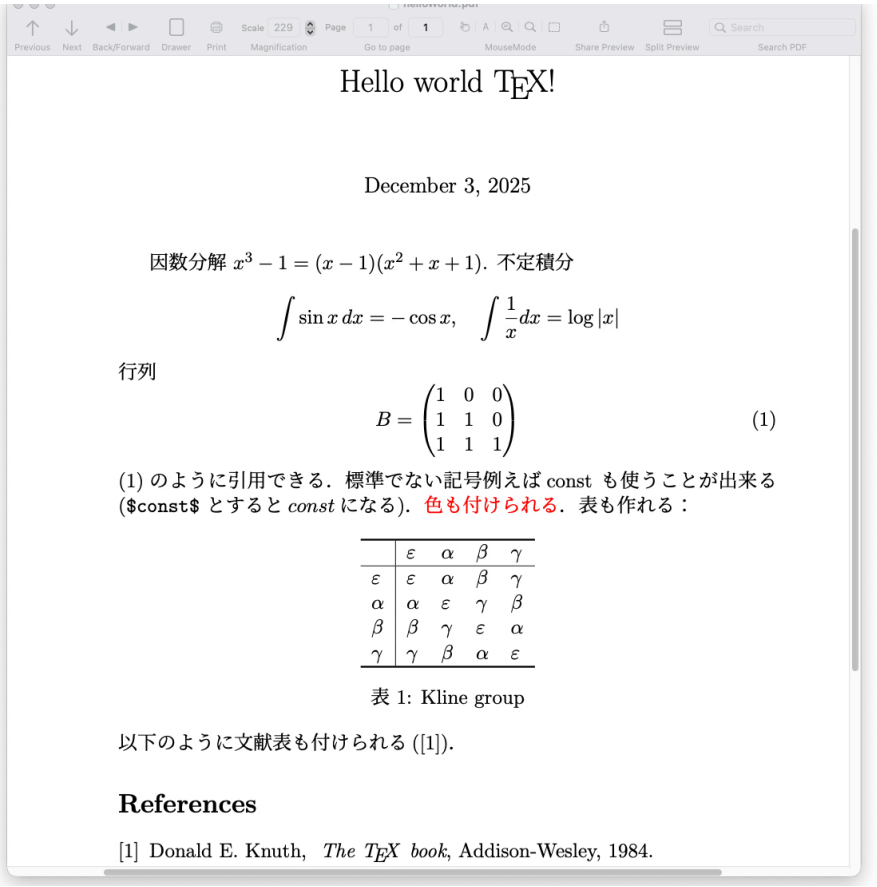
```
\begin{equation}\label{eqMat}
   B
   =
   \begin{pmatrix}
      1&0&0\\
      1&1&0\\
      1&1&1
   \end{pmatrix}
\end{equation}
```

`\eqref{eqMat}`**のように引用できる．標準でない記号例えば**`$\const$`
**も使うことが出来る** (`\verb/$const$/` **とすると**`$const$`**になる)．**

`\textcolor{red}{`**色も付けられる**`}`**．表も作れる**:

```
\begin{table}[thb]
   \centering
   \begin{tabular}{l|llll}\noalign{\hrule height 1pt}
      &$\varepsilon$&$\alpha$&$\beta$&$\gamma$ \\
      \hline
      $\varepsilon$&$\varepsilon$&$\alpha$&$\beta$&$\gamma$ \\
      $\alpha$& $\alpha$&$\varepsilon$ &$\gamma$&$\beta$ \\
      $\beta$& $\beta$&$\gamma$&$\varepsilon$&$\alpha$ \\
      $\gamma$& $\gamma$ &$\beta$&  $\alpha$&$\varepsilon$\\
     \noalign{\hrule height 1pt}
   \end{tabular}\caption{Kline group}
\end{table}
```

```
\noindent
以下のように文献表も付けられる (\cite{K}).
\begin{thebibliography}{9}
\bibitem{K} Donald E. Knuth, \textit{ The \TeX\, book},
   Addison-Wesley, 1984.
\end{thebibliography}
\end{document}
```



Hello world TEX!

December 3, 2025

因数分解 $x^3 - 1 = (x-1)(x^2+x+1)$. 不定積分

$$\int \sin x\, dx = -\cos x, \quad \int \frac{1}{x} dx = \log|x|$$

行列

$$B = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix} \tag{1}$$

(1) のように引用できる．標準でない記号例えば const も使うことが出来る (**$const$** とすると *const* になる)．色も付けられる．表も作れる：

| | $\varepsilon$ | $\alpha$ | $\beta$ | $\gamma$ |
|---|---|---|---|---|
| $\varepsilon$ | $\varepsilon$ | $\alpha$ | $\beta$ | $\gamma$ |
| $\alpha$ | $\alpha$ | $\varepsilon$ | $\gamma$ | $\beta$ |
| $\beta$ | $\beta$ | $\gamma$ | $\varepsilon$ | $\alpha$ |
| $\gamma$ | $\gamma$ | $\beta$ | $\alpha$ | $\varepsilon$ |

表 1: Kline group

以下のように文献表も付けられる ([1]).

**References**

[1] Donald E. Knuth, *The TEX book*, Addison-Wesley, 1984.

**図 B.2　Hello world TEX!**

を helloWorld.tex[1)] として保存し（読み込みたい画像とかのファイルがあればそのディレクトリに置き），helloWorld.tex を TeXShop.app で開き， typeset をクリックするか， cmd T とタイプするとコンパイルできる (図 B.2).

1)　どんなコンピュータ言語でも最初に書くプログラムの名前を helloWorld のようなものにするのがプログラマーの伝統である．また，どうでも良い名前の場合 hoge とか fuga のような名前を使うのも伝統である．

**演習 38**　上のソースファイルをコンパイルして pdf を表示してください.

`\begin{document}`
の前を preamble (序文) という．取りあえず preamble はコピーして使い，簡単な数式を [84] などを調べながら書いてみることをお奨めする．ひとことコメントすると
`\DeclareMathOperator{\const}{const}`
が標準にない記号 const を定義しているところである．あと，TEX の機能と云うより TeXShop ( Editor ) の機能だが,

- cmd キーを押しながらクリックすると対応する pdf の場所とソースファイルの場所を行き来することができる.
- コンパイルしてエラーが出て止まったとき，console の上部の Goto Error をクリックすると，ソースファイルの該当部分に飛ぶことができる.
- ソースファイルで Esc を押すと補間機能を使うことが出来る．例えば`\begin{eq` まで打って Esc を押すと

  ```
  \begin{equation}
  •
  \end{equation}•
  ```

  と補完される.

また，emath という大熊一弘氏の開発した日本の小中高数学のプリント作成用の latax の style file があり，教科書の数式に近い出力を得ることが出来る．ここからファイルを取り寄せ,
/usr/local/texlive/texmf-local/tex/latex/local/にコピーし, terminal から
`%` sudo texhash
として path を通す． ただし`%` はプロンプトで，タイプしていけない．preamble に
`\usepackage{emath}`
を追加すると使えるようになる．詳しくは emath のマニュアルを参照.

### B.1.3 TEX の図式作成ソフト

$$\begin{array}{ccc} S^2(1) & \longrightarrow & \mathbf{R}^3 \setminus \{\mathbf{0}\} \\ {\scriptstyle \mu'}\downarrow & & \downarrow{\scriptstyle \mu} \\ S^2(1)/\sim & \xrightarrow[\pi]{} & \mathbf{P}^2 \end{array}$$

のような図式を描く, texlive に含まれるソフトの一つが XY-pic である．まず，

```
\input xy
\xyoption{all}
```

を preamble に付け加え，本文中に

```
\[
   \xymatrix{
      S^2(1) \ar[d]_{\mu'} \ar[r]
       & \mathbf R^3\setminus \{\mathbf 0\} \ar[d]^{\mu} \\
     S^2(1)/ \sim  \ar[r]_{\pi} & \mathbf P^2
   }
\]
```

を入れると，上の図式を描くことができる．`\xymatrix{...}` の中でマトリックスを描く要領で成分の数式の記号を並べている．`&`は列の分割を表し，`\\`は改行である． `\ar[d]` は下への矢印， `\ar[r]` は右への矢印を描く指令で，それに続く `_{\mu'}` は下付き，`^{\mu}`は上付きの矢印の修飾を描く命令である．

### B.1.4 TEX の描画ソフト

METAPOST のコンパイルは，ソースファイル figure.mp を texshop で開き，ファイルのウインドウの上部にあるファイルタイプのセレクターを LaTeX から MetaPost に変更し，タイプセットする (図 B.3) と，figure.1, figure.2 のような postscript ファイルが生成される．あるいは，ファイルのあるディレクトリで terminal から

```
% mpost figure.mp
```

でコンパイル出来る．

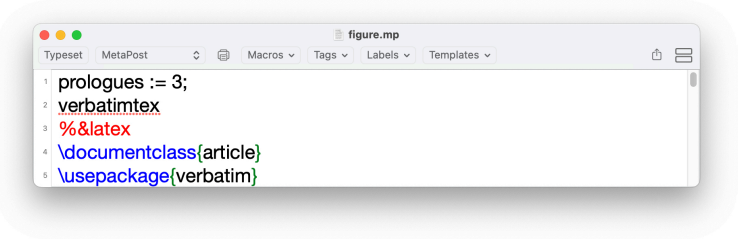



図 B.3　METAPOST のコンパイル

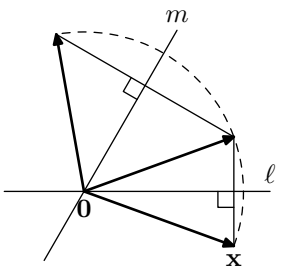



図 B.4　METAPOST の描画例

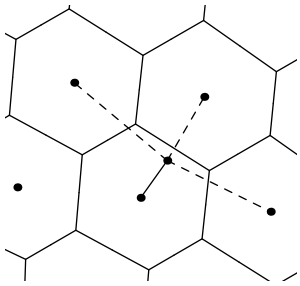


図 B.5　METAPOST の描画例

のような絵を描く METAPOST のソースファイルの例を次にあげる:

```
prologues := 3;
verbatimtex
%&latex
\documentclass{article}
\usepackage{verbatim}
\usepackage{latexsym}
\usepackage{amssymb}
\usepackage{amsmath}
\begin{document}
etex;

numeric a, b;
a=10pt;
b=6pt;

beginfig(1);
   z0=(5a,3a);
```

```
z1=z0-(3a,0);
z2=z0+(7a,0);
draw z1--z2; %line \ell
label.top(btex $\ell$ etex,z2);
z5=z0+(7a*cosd(60), 7a*sind(60));
z6=z0-(3a*cosd(60), 3a*sind(60));
draw z5--z6;%line m
label.top(btex $m$ etex,z5);

for j=-2 upto 10:
   z[100+j]=z0+6a*(cosd(10j),sind(10j));
endfor;

draw
   for i=-2 upto  9:
      z[100+i]..
   endfor
   z110 dashed evenly;

z150=whatever[z0,z2]=whatever[z98,z102];
z160=whatever[z0,z5]=whatever[z102,z110];
z151=z150-(b,0);
z152=z151-(0,b);
z153=z152+(b,0);
z161=z160-(b*cosd(60),b*sind(60));
z162=z161+(b*cosd(150),b*sind(150));
z163=z162+(b*cosd(60),b*sind(60));

draw z98--z102--z110;
draw z151--z152--z153;
draw z161--z162--z163;

label.bot(btex $\mathbf 0$ etex,z0);
label.bot(btex $\mathbf x$ etex,z98);
```

```
   pickup pencircle scaled 1pt;
   drawarrow z0--z98;
   drawarrow z0--z102;
   drawarrow z0--z110;
endfig;

beginfig(2);
   def voro(expr p, q, r) =
      begingroup
          save a, b, c, u, v, w, e, f, g, h;
          pair a, b, c, u, v, w;
          numeric e, f, g, h;
          a = 0.5[q,r];
          b = 0.5[r,p];
          c = 0.5[p,q];
          (e,f)= r-q ;
          (g,h) = p- r;
          u = (f, -e);
          v = (h, -g);
          w = whatever[a + u, a - u] = whatever[b + v, b - v] ;
          draw a--w--b;
          draw w--c;
      endgroup
   enddef;

   z0=(-5a,-5a);
   for i=1 upto 6:
      for j=1 upto 6:
       z[10i +j]= z0 + (5i*a, 0) +5j*(a/2, sqrt(3)*a/2)
          +(a*cosd(30(i+j)+i*j/2), a*sind(30(i+j) + i*j/2) );
      endfor;
   endfor;
```

```
    z100 = z32+ (a,1.4a);
    dotlabel(btex etex, z100);
    draw z100--z32;
    draw z100--z33 dashed evenly;
    draw z100--z23 dashed evenly;
    draw z100--z42 dashed evenly;

    for i=1 upto 6:
       for j=1 upto 4:
        dotlabel(btex etex, z[10i +j]);
       endfor;
    endfor;
    for i=1 upto 5:
       for j=1 upto 3:
        voro(z[10i +j], z[10(i+1)+j], z[10i +j+1]);
        voro(z[10i +j+1], z[10(i+1) +j+1], z[10(i+1)+j]);
       endfor;
    endfor;
endfig;

verbatimtex
\end{document}
etex;
end
```

を figure.mp と言う名前で保存し，コンパイルすると figure.1, figure.2 というファイルが作られる．METAPOST のプログラミング言語としての仕様 (詳しくは [85]):

- beginfig(1); から endfig; までが figure.1 の描画内容である．figure.2 も同様．これ以外の最初の部分と最初の部分はコピペすることをお奨めする．
- 文末には ; が必要．% の後がコメント文
- 点を表すのに, z10 のような z に数字を並べた表記を使う．インデックスを変数として扱うときは z[i] のような記法を使う．
- 点 z1 から z2 への直線を書くのは

```
draw z1--z2;
```

点 z1 から z2 への矢印を書くのは

```
drawarrow z1--z2;
```

z1, z2, z3 を補間するベジェ曲線は

```
draw z1..z2..z3;
```

- METAPOST 特有の機能として

```
z150=whatever[z0,z2]=whatever[z98,z102];
```

のように直線と直線の交点を求めることが出来る．様々な初等関数も用意されている．関数は `sind(10)` のように呼び出される．
- for 文で点を定義したり，曲線を描くことも出来る．
- 図への数式や文字のラベルでは LaTeX と同じものを使える．例えば

```
label.bot(btex $\mathbf 0$ etex,z0);
```

は点 z0 の真下の部分に **0** ( `$\mathbf 0$` ) を書くことを命令するコマンドである．
- beginfig(2) に関数の使い方の例をあげたのでぜひ読み解いてほしい．この例では def から enddef の中身 volo が三つの入力変数をもつ関数である．= の後がその定義である．定義の中で使う変数をまず保存して，その後変数の宣言を書く．次に二点の中点をもとめ，二点の作る辺に直交するベクトルをもとめ，その中点に直交する線分をもとめている．もう一つの二点についても同様に中点に直交する線分をもとめ，それらの交点として三点に外接する円の中心をもとめている．そこから各中点まで線分を 3 本書いている．

  あとは正 6 角形格子の点を少し振動させ，それらの点についてのボロノイ分割を volo() を使って書いている．
- ここではこれ以上述べないがもちろん if 文も使える．

**演習 39**　上の METAPOST のソースファイルを figure.mp と言う名前で保存し，コンパイ

ルし，上で作った tex のファイルに取込み pdf に表示してください．

TikZ と言う style file もあり，それを使うと直接 LaTeX の中で METAPOST に似たコマンドで描画できる．また metafun という METAPOST を現代風に拡張した style file もある．

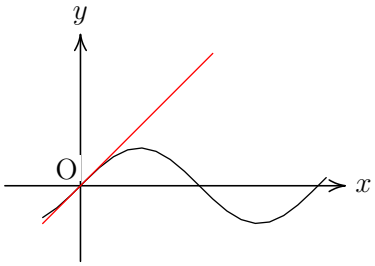
MePoTeX という（ここから入手可能 ）METAPOST を拡張する style file もあり，(emath と同じようにインストールし path を通せば） LaTeX の中で使える． preamble に

```
\usepackage{MePoTeX}
```

を追加し，本文に

```
\begin{MPpic}<5mm>(7|2,4|2)
\mptXaxis[l]<1mm,0mm>{$x$}
\mptYaxis[b]<0mm,1mm>{$y$}
\sendMP{
   xdraw() kansu(sin(t))
   (-1,6.5,20);
   xdraw(red) kansu(t)
   (-1,3.5,20);
}
\mptLabel{origin}[tr]
<0mm,4mm>{O}
\end{MPpic}
```

を挿入すると




のような図が描ける．

**宿題 7**　自分のセミナーの内容を説明する LaTeX のファイルを作成し，コンパイルせよ（目標は下の作業を含めて pdf で 5〜10 ページほど）．それに表・図式・参考文献を付け加えよ．METAPOST で図を描き，それをその説明の LaTeX の pdf の中に表示せよ．

## B.2 Markdown

TEX とは直接関係はないが，最近よく使われる Markdown 言語についても紹介したい．これは Web page を編集するための言語 html (=hyper text markup language) を簡易的に表すために，つまり，markdown を目指して，2004 年ジョン・グルーバー (J. Gruber) によって開発された言語である．VSCode で IntroToMarkdown.md というファイルを作り，以下のソースをコピーし，cmd+shift+V とタイプすると（またはウインドウの右上にあるノートに虫眼鏡のアイコンをクリックすると）図 B.6, B.7 のような html のプレビュー画面が現れる．

```
<!-- Introduction to markdown created by snow00two -->

# Introduction to Markdown

## Markdown

- list1
- list2
  - sublist1
  - sublist2
- list3

 1. list1
 2. list2
 3. list3

Separator:

---

*italic* ,  **bold** , ***italic bold***

> quotation
 >> second quotation
```

````
Programming Language ‘JavaScript‘ list:

‘‘‘javascript
function hoge () {
  console.log("problem");
}
‘‘‘

Table:

| Data | A | B |
| ---: |: ---: |: ---: |
| 0 | 1 | 2 |

Mathematics:
$\sqrt2$, $\pi$, $\frac1{x + 1}$

Hyperlink to: [amazon](https://www.amazon.co.jp/amazonprime)

Image file: ![image file](dodecahedron.jpg)
````

文法についてはこのファイルを見れば直ぐに分かるだろう．最近の技術系ホームページ (例えば GitHub) は Markdown で作られていることが多い．また，生成 AI のファインチューニングという訓練で Markdown が使われていることはよく知られていて，ChatGPT へのプロンプトの入力も Markdown を使うことが推薦されている ([42], [15]). 最後に markdown から LaTeX を作ることも出来る（とは言っても数式等は LaTeX で作る必要がある).

**演習 40**　1. VSCode で上のソースファイルを参考にして，1〜2 ページ程度で markdown を説明する markdown の文書を作ってプレビューして下さい.

2. そのソースファイルを，生成 AI を使って latex のソースファイルに変換してください (その中でコマンドの実行を頼まれるかも知れません).
3. その過程で生成 AI が何をしたのか分析してください.

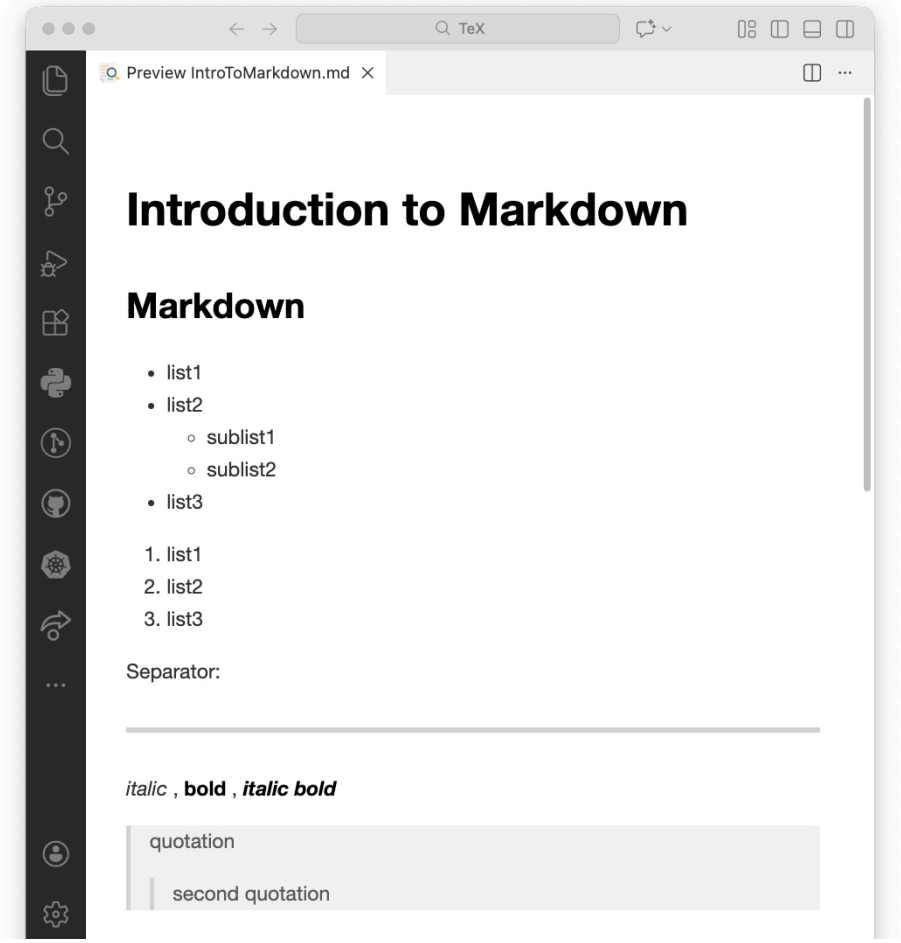



**図 B.6　VSCode での IntroToMarkdown.md のプレビュー 1**

4. その分析に基づき，自分一人でもう一度その latex の書類を markdown に変換して下さい.

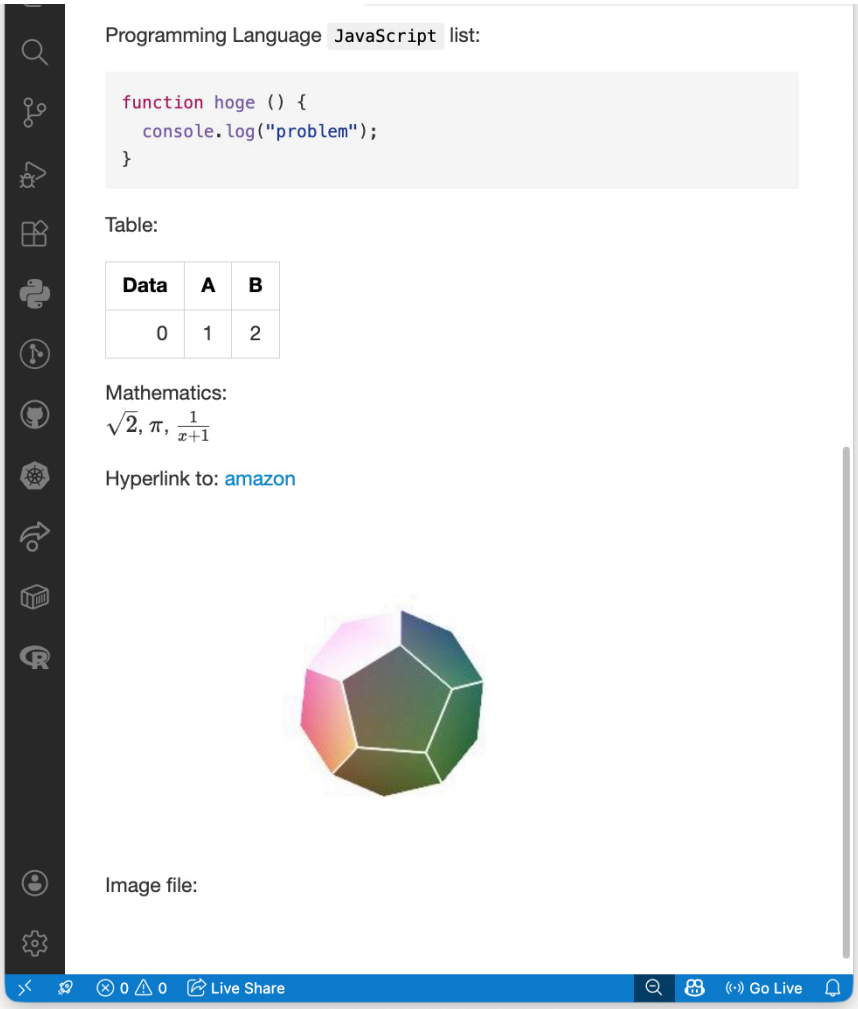



**図 B.7　VSCode での IntroToMarkdown.md のプレビュー 2**

# 付録 C　JavaScript と p5.js

私のポートフォリオに私が JavaScript と p5.js で作ったアプリが置いてある．その幾つかについて説明する．

## C.1　p5.js の 2D アプリ

### C.1.1　p5.js

p5.js は Processing という Java 用の描画プログラミング言語を Javascript に移植したものである．以下のウェブ上のエディターで実行可能である．

https://editor.p5js.org （以下この色はハイパーリンク）

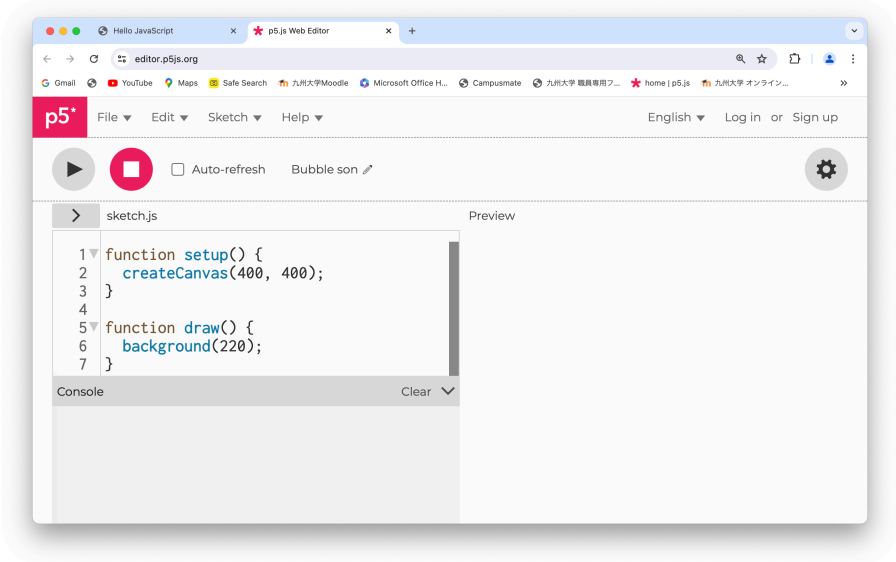



**図 C.1　ウェブ上の p5.js エディター**

公開した sketch.js を sketch.js の部分にコピーし，（ファイル octahedron.stl をアップロードし）左上の三角のボタンを押すと右画面に描画が始まる．もちろん隣りの四角のボタンで止まる．ただし .js は JavaScript のファイルであることを示す拡張子（ extension ）である．

また以下のようにすると自分のパソコン上で動かせる．p5.js というライブラリーをダウンロードすると p5 というディレクトリが作られる．公開された sketch.js をそのディレクトリーの sketch.js と置き換え，フリーのエディター VSCode Visual Studio Code でそのディレクトリ p5 を開き，Live Server の拡張を VSCode にインストールして，Go Live のボタンを押すと Chrome で再生を始める．

### C.1.2　一般の JavaScript の動かし方

JavaScript の処理の仕方は幾つかある．JavaScript はブラウザーに組み込まれているので，まず

```
<!DOCTYPE html>
<html>
   <head>
       <meta charset="UTF-8" />
       <title>Hello JavaScript</title>
   </head>
   <body>
      <script type="text/javascript" src="test.js"></script>
   </body>
</html>
```

のような html のファイルを hoge.html として保存し，次に

```
window.alert('hello world!');
```

という行のみからなるファイルを test.js として hoge.html と同じディレクトリに保存する．ブラウザー Chrome のメニューバーの File のメニュー Open Files... から hoge.html を開くと

**図 C.2**

図 C.2 のような結果が得られる．あるいは

```
console.log('hello world!');
```

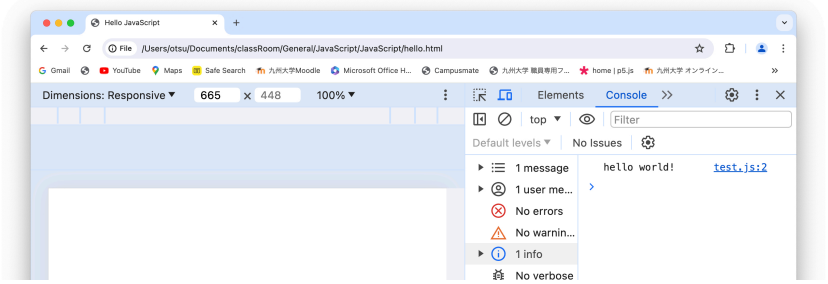



**図 C.3**

を test.js として保存し，また hoge.html を開くと File のメニュー View から Developer の Developer Tools を確認すると図 C.3 のような結果が得られる．

他にも Terminal で test.js のあるディレクトリに移動し

```
% osascript test.js
```

で Terminal に結果が表示される．ただし，これは Mac のみで，逆に言うと Mac のスクリプト言語として JavaScript を使うことが出来る．一般的には Node.js をダウンロードしインストールすると

```
% node test.js
```

で Terminal に結果が表示される．したがって Mac を直接コントロールする必要が無いときは osascript より node.js を使うことをお奨めしたい．

### C.1.3 JavaScript の言語仕様

- JavaScript には変数の型はなく，文の終りで ; が必要
- let は変数の宣言，以前は var も使われていた．
- const は定数の宣言
- //はコメント文でこれ以降の文は無視される．
  /* */はブロックコメント文で途中の文は無視される．

よく使う制御構文：for 文の例：

```
let I = 5;
for (let i=0; i < I; i++) {
   console.log(i);
}
```

if 文の例：

```
let m = 80;
if ( m<60 ){
   console.log('F');
} else if ( m<70  ) {
   console.log('C');
} else if ( m<80  ) {
   console.log('B');
} else if ( m<90  ) {
   console.log('A');
} else {
   console.log('S');
}
```

function hoge(var1,...,var3) は関数 hoge の宣言で，続く {} の中に関数の定義を書く．var1,...,var3 が関数の変数で定義の中で使える．関数の中で定義された変数はローカル変数で，関数外で使われる変数はユニバーサルな変数になる．

### C.1.4　2D アプリのソース

以下が Sierpinski gasket を描くアプリのソースファイル sketch.js の内容である．

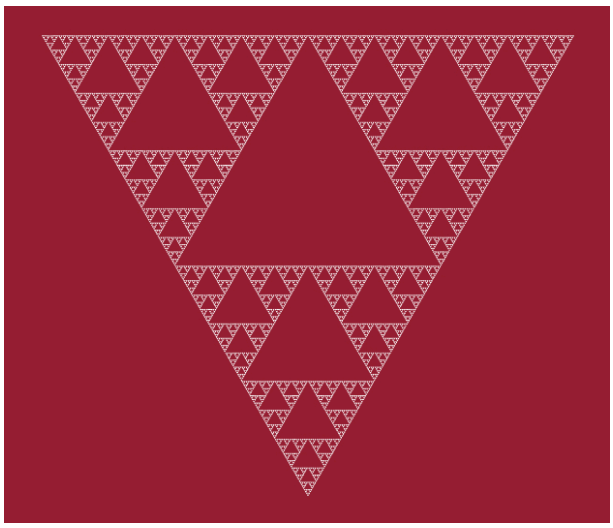


**図 C.4　Sierpinski gasket**

```
/* Sierpinski gasket etc., abbreviated version
 * 2026/06/22, by snow00two
```

```
 * \href{https://creativecommons.org/licenses/by-nc-nd/4.0/}{\ccbyncsa}
 */
/* canvas setting */
const WIDTH_CANVAS = 720 * 3/2  ; //=1080
const HEIGHT_CANVAS = 405 * 3/2 ; //=607.5
const CENTER_X = WIDTH_CANVAS/2 ;
const CENTER_Y = HEIGHT_CANVAS/2 ;

/* color setting */
const DARK_COLOR = [150,30,50];
const WHITE_COLOR = 255;
let colorChoice ;
let choiceC ;/* variable for color scheme */
let backColor ;
let frontColor ;

/* Sierpinski gasket setting, abbr. 3 */
const BASE_SEGMENT_3 = 500;
let height3;
let radius3;
let shift3Y;
let fixed3X;
let fixed3Y;

function setup() {
  /* Sierpinski gasket additional setting */
  height3 = sqrt(3)* BASE_SEGMENT_3 / 2;/* sqrt(3) is defined here */
  radius3 = 2 * height3 /3 ;/* the radius of circumscribed circle  */
  fixed3X = ( WIDTH_CANVAS - BASE_SEGMENT_3 ) / 2 ;
  fixed3Y = ( HEIGHT_CANVAS - height3  ) / 2 ;
  shift3Y= ( HEIGHT_CANVAS - height3 ) / 3 ;

  colorChoice = createSelect();/* object setting color scheme */
  colorChoice.option('normal color','normal');
```

```
  colorChoice.option('reverse color','reverse');
  colorChoice.selected('normal');
  colorChoice.position(30, 8);
  colorChoice.changed( resetBackground ) ;

  frameRate(0.6);

  /* create canvas */
  createCanvas(WIDTH_CANVAS, HEIGHT_CANVAS);
  background(DARK_COLOR);
}

let n = 0; //clock parameter
/* main function controlling this application */
function draw() {
  let choiceC = colorChoice.value();
  if (choiceC =='normal' ){
    backColor = DARK_COLOR;
    frontColor = WHITE_COLOR;
  } else {
    backColor = WHITE_COLOR;
    frontColor = DARK_COLOR;
  }
  background(backColor);

  translate(CENTER_X, CENTER_Y - shift3Y);
  fill(frontColor);
  iterateTriangleS(n);

  let I = 9 ; //Number of iterations
  if( n < I ){
    n++ ;
  } else {
    background(backColor);
```

```
    n = 0 ;
  }
}

/* recursive function creating Sierpinski gasket */
function iterateTriangleS(num){
  push();
    if (num == 0){
      fill(frontColor);
      noStroke();
      triangle(-BASE_SEGMENT_3 / 2, -radius3 /2,
      BASE_SEGMENT_3 /2,  -radius3 /2, 0, radius3 );
  //Here we erase the center reverse triangle
    } else {
      scale( 1 / 2 );
      fill(frontColor);
      noStroke();
      for (let l=0;l < 3;l++){
        translate(0, radius3 );
        iterateTriangleS(num - 1);
        translate(0, -radius3 );
        rotate(2*PI/3);
      }
    }
  pop();
}

function resetBackground () {
  background(backColor);
  n = 0;
}
```

### C.1.5　p5.js の 2D モードの説明とソースファイルの解説

p5.js の 2 次元モードでは $y$ 軸の方向が通常とは逆で下向きになっている．これはテレビなどの表示では，一次元で送られてくるデータを図 C.5 のように上から下に折り返しながら表示していたからである．したがって，円周上で角度が増加するプログラムでは時計回りに回転する．もちろん，反時計回りにするためにはパラメータの符号を代えれば良い．

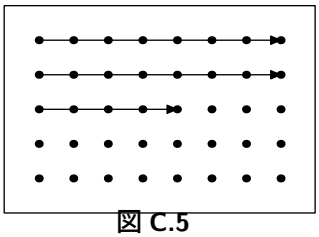


図 C.5

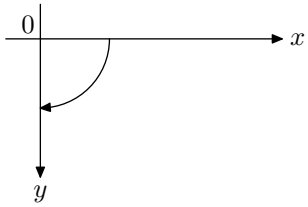



図 C.6

関数 setup() の中でウィンドウ等を初期化している．createCanvas( widthCanvas, heightCanvas) で描画領域を指定している．関数 draw() が呼び出されると変数 n を 1 フレーム毎に 1 づつ増やしながら，その都度フレームを描画し続ける．いじってみるとだんだん理解が深まると思う．本質的な部分は帰納関数 iterateTriangleS() である．

p5.js 特有の関数を説明したい：

- translate$(a, b)$; $(a, b)$ だけ平行移動し，その点を $(0, 0)$ として描画
- rotate(theta); 角度 theta だけ直交座標系を回転し，それを新たな直交座標系として描画
- triangle$(a, b, a', b', a''b'')$; $(a, b)$, $(a', b')$, $(a'', b'')$ を頂点とする三角形を描画
- background($[a, b, c]$); 画面を RGB $[a, b, c]$ で埋める．ただし $a, b, c$ は 0 から $255 = 2^8 - 1$ の間の整数．
- fill($[a, b, c]$); 次の図形を RGB$[a, b, c]$ で埋める．
- push(), pop() はその間に変えられた設定を元に戻すためのスタックに書き込むための関数

関数 iterateTriangleS(num) がこのプログラムの本体である．push() でそれまで呼び込まれた設定を保存し，pop() で元の設定に戻っている．$y$ 軸が逆だと分かりずらいので，$y$ 軸を通常のようにとり，回転方向も反時計回りだと思って正しくプログラムすると（図 C.7），出力は $y$ 軸が逆になって描画される．三角形の重心が原点にあるとする．num $= 0$

のとき fill(frontColor) で次の図を frontColor で満たすことを， noStroke() で輪郭を描かないことを指示し，

```
triangle(-baseSegment3 / 2, -radius3 /2,
   baseSegment3 /2,  -radius3 /2, 0, radius3 );
```

で，辺の長さが baseSegment3 でその底辺が $x$ 軸に平行な正三角形を書いている．このとき, radius3 は正三角形の外接円の半径である．

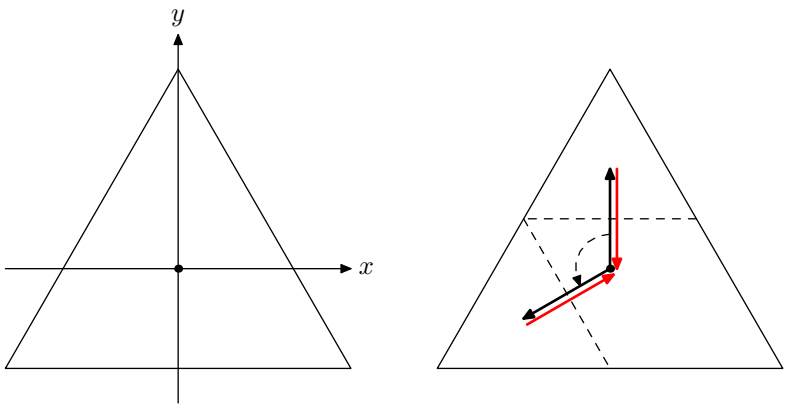



**図 C.7**

num $= 1$ のときは scale 1/2 にして $(0, 0)$ から (0,radius3) 進むので実際は $(0, 0)$ から (0,radius3/2) 進んで三角形を描く．その後 $(0, -\text{radius3})$ 進み $(0, 0)$ に戻り $2\pi/3$ 回転する．これを三回繰り返し，3 つの三角形を描く．

num$> 1$ のときはこれを繰り返すことで Sierpinski gasket を書くことが出来る．

**宿題 8**　Sierpinski Carpet(図 C.8) を描くようにプログラムを変更せよ．

**宿題 9**　Koch flake(図 C.9) を描くようにプログラムを変更せよ．

## C.2　p5.js の 3D-アプリ

### C.2.1　3D アプリのソース

3 次元の描画を行うことも出来る．createCanvas を

createCanvas(widthCanvas, heightCanvas, WEBGL);

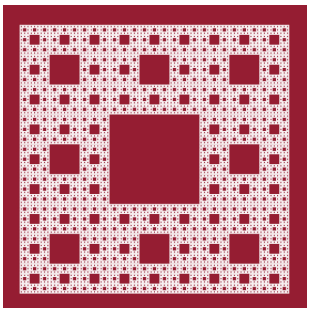


**図 C.8　S. Carpet**

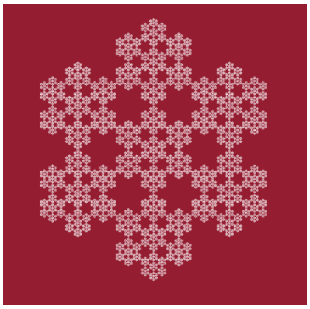


**図 C.9　Koch flake**

に替えれば良い．ただし，WEBGL とはブラウザ上で CG を描画するための API である．以下がソースファイル sketch1.js の本質的な内容である．

```
/* Cubes landing before the Great Pyramids abbreviated version
 *  2026/06/17, by snow00two,
 * \href{https://creativecommons.org/licenses/by-nc-nd/4.0/}{\ccbyncsa}
 */
const WIDTH_CANVAS = 1080  ; // 720x3/2
const HEIGHT_CANVAS = 607.5 ; // 405x3/2
const BACK_COLOR = [150, 200, 250] ;//[200, 220, 200]

let octahedron ;
function preload(){
  octahedron = loadModel('octahedron.stl');
  //https://www.printables.com/model/411768-octahedron-freecad/files
}

function setup(){//canvas : window of the projection coordinates
  createCanvas(WIDTH_CANVAS, HEIGHT_CANVAS, WEBGL);
  background(BACK_COLOR);
}

let i = 0;//basic parameter for frames

function draw(){
```

```
let I = 9425; //change the priod condition to 3x2Pi/0.002=9424.7...

if (i<I) {
} else {
  i = 0;// reset basic parameter
}

// define the camera coordinates from a camera on the helicopter
// of the following coordinates in the world coordinates :
let cameraX = cos(0.5 + i * 0.002)*250 ;
let cameraY = sin(0.5 + i * 0.002)*250;
let cameraZ = 200+sin(0.5 + i * 0.002)*30;
camera(cameraX,cameraY, cameraZ, 100, 0, 100, 0, 0, -1);
// the output images are in the projection coordinates.

//The world coordinates :
background(BACK_COLOR);
directionalLight(250, 200, 200, 0, -1, -1); //light from the sun
fill(250, 200, 200);
plane(10000,10000);

stroke(0);
strokeWeight(2);
fill(150,150, 150);
pyramid(3);

translate(-250,0,0);
pyramid(3);

translate(-250,0,0);
pyramid(1.5);

ambientMaterial(125);
ambientLight(100,100,100);
```

```
  shininess(100.0);

  translate(700, 200, max(500 - i * 0.3, 67) );
  sCube(5);

  let M = 25;
  for(let m = 0 ; m < M ; m++){
    translate(140,140, max(650 + 150*m - i * 0.3, 0));
    sCube(5);
  }
  i++;
}

// The modeling coordinates :
function pyramid(scaleN){
  push();
    rotateZ(QUARTER_PI);
    scale(scaleN);
    model(octahedron);
  pop();
}

function sCube(scaleN){
  push();
    rotateZ(QUARTER_PI);
    scale(scaleN);
    box(25);
  pop();
}
```

### C.2.2 p5.js の 3D モードの説明とソースファイルの解説

関数 preload() でディレクトリにある図形 octahedron.stl を読み込み，関数 setup() の中でウィンドウ等を初期化する．関数 draw() が呼び出されると変数 i を 1 フレーム毎に 1 づつ増やしながら，その都度フレームを描画し続ける．いじってみるとだんだん理解が深

まると思う.

p5.js 特有の組込関数の説明：

- translate(a, b, c); $(a, b, c)$ だけ平行移動し，その点を $(0, 0, 0)$ として描画
- rotateZ(theta); $z$ 軸を中心に角度 theta だけ直交座標系を回転し，それを新たな直交座標系として描画．rotateX, rotateY も同様.
- scale(a) 座標系を a 倍する
- strokeWeight(a) 太さ a で描く
- background(a,b,c); 背後の画面を RGB [a,b,c] で埋める．ただし $a, b, c$ は 0 から $255 = 2^8 - 1$ の間の整数.
- directionalLight(a, b, c, d,e,f) RGB[a, b, c] の光で向き (d,e,f) $-1 \leq d, e, f \leq 1$ で光を照らす
- ambientLight(a, b, c) RGB[a, b, c] の光で背景となる光を照らす
- ambientMaterial(a, b, c) 次の図形の壁の色を RGB [a,b,c] で設定
- fill(a,b,c) 次の図形を RGB [a,b,c] で埋める.
- plane(a,b), box(c) などは 3 次元の組込立体図形
- camera(a, b, c, a', b', c') カメラの位置 $(a, b, c)$ 対象の中心 $(a', b', c')$ に向けてウィンドウを描画
- push(), pop() はその間に変えられた設定を元に戻すためのスタックに書き込むための関数
- model(modelName) modelName を描く

**宿題 10**　GitHub のアカウントがあれば GitHub の Pages という機能を使うと自分のホームページが作れます．私のポートフォリオもそのようにして作ったものです．やり方を調べて自分のホームページを作って，自分の作った作品を公開してください.

# 付録 D　Shell と Shell Script 入門

## D.1　標準入出力とパイプ

最初難しく感じるのが標準入力・標準出力・標準エラー出力とファイルディスクリプタであると思うので解説する．

```
% echo "example text"
example text
```

この例では”example text”標準入力で次の行が標準出力である．

```
% echo "example text" > text
%
```

この例では標準出力がファイル text に出力され，そのせいでターミナルには出力されない．>をリダイレクトという．確かめてみると

```
% cat text
example text
```

となり，ファイル text の内容が標準入力となり，次の行が標準出力である．

```
% cat test
cat: test: No such file or directory
```

この例では間違えた名前を打ったので，そのようなファイルはないと標準エラー出力に表示された．つまり，標準エラー出力と標準出力はファオルトではターミナルの次の行である．

次にパイプについて説明する．

```
% cat text | wc
       1       2      13
```

この|がパイプである．前半のコマンド cat text の標準出力が直接 wc の標準入力に送られその標準出力が次の行に出力されている．

入出力の管理はファイルディスクリプターによって制御されている．これは比較的小さな非負整数で，標準入力は 0, 標準出力は 1, 標準エラー出力 2 となっていいる．新しいデバイスを開いたりすると追加の非負整数が割り振られる．これを使って Shell は細かな入出力の管理をしている．

## D.2 Shell Script

Shell の操作をまとめて Script を作ることが出来る．これを Shell Script という．まず

```
#!/bin/sh

echo "Hello World!"
```

という内容を hello という名前で保存する．そのディレクトリで

```
% ls -l
total 8
-rw-r--r--  1 snow00two  staff  33 Apr 30 13:28 hello
# このままだと実行可能でない
%  chmod +x hello   # 実行可能に変換
%  ./hello
Hello World!
```

**演習 41**　コマンドラインを使い，ディレクトリーを移動したり，ディレクトリーやファイルの作成や削除を行ってください．

| | |
|---|---|
| pwd | 現在のディレクトリー表示 |
| ls | 現在のディレクトリーの中のファイル表示 |
| cd dir1 | ディレクトリー dir1 へ移動 |
| cd | Home ディレクトリーへ移動 |
| cp file1 file2 | file1 を file2 にコピー |
| mv file1 file2 | file1 の名前を file2 に変更 |
| rm file1 | file1 の削除 |
| touch file1 | file1 の変更時刻の更新，file1 が無ければ file1 を作成 |
| mkdir dir1 | ディレクトリー dir1 を作る |
| rmdir dir1 | ディレクトリー dir1 を削除 |
| ps | 現在のプロセスを表示 |
| kill pid | プロセス id (自然数)pid のプロセスの終了 |
| which cmd | cmd が使えればそのコマンドの path を表示 |
| who | 誰が今ログインしてるかを表示 |
| who am i | 自分がどの username でログインしているか表示 |
| date | 現在の日時を表示 |
| cal | 現在の月のカレンダーを表示 |
| chmod +x file1 | file1 をコマンドとして実行可能にする |
| ./file1 | 現在のディレクトリの file1 をコマンドとして実行 |
| chmod -x file1 | file1 をコマンドとして実行不可能にする |
| echo ”text” | 文字列 text を表示 |
| cat file1 file2 | file1, file2 の内容を連結 (concatenate) して表示 |
| pr file1 | file1 の内容をフォーマットして表示 |
| tail -n file1 | file1 の最後の $n$ 行を表示 |
| head -n file1 | file1 の最初の $n$ 行を表示 |
| more file1 | file1 の内容を確認しながら表示 |
| ed file1 | ed で file1 を編集 |
| vim file1 | vim で file1 を編集 |
| code file1 | VSCode で file1 を編集 |
| open -a apli file1 | アプリ apli で file1 を編集 |
| sh | subshell を開く |

**表 D.1　コマンドの基本**

| | |
|---|---|
| wc file1 | file1 の行数，語数，文字数を表示 |
| cmp file1 file2 | file1 と file2 が違えば最初の違う行を表示 |
| diff file1 file2 | file1 と file2 がどのように違うかを表示 |
| grep hoge file1 | file1 の hoge を含む行を表示 |
| sort file1 | file1 の行毎にソートする |
| man cmd1 | cmd1 のマニュアル表示 |
| history | これまで使ったコマンドのリスト |
| printenv | 全ての環境変数の表示 |
| echo $ENV | 環境変数 ENV(が定義されていれば) 表示 |
| su hoge | hoge という username のユーザに変わる |
| su | root に変わる．Mac では廃止 |
| sudo cmd1 | スーパーユーザー権限で cmd1 を実行 |

**表 D.2　コマンドの基本 2**

| | |
|---|---|
| control D | end of file |
| control S | log out |
| tab | filename 補間 |
| * | 任意 filename |
| *hoge | 最後が hoge の任意 filename |
| !! | 直前のコマンドをもう一度 |
| !cmd1 | cmd1 で始まる一番新しいコマンドをもう一度 |
| cmd1; cmd2 | コマンド cmd1 と cmd2 の並列 |
| cmd1\| cmd2 | pipe: コマンド cmd1 の標準出力を cmd2 の標準入力へ |
| > file1 | file1 へ redirect |
| >> file1 | file1 へ追加 |
| < file1 | file1 から redirect |
| . | current directory |
| .. | parent directory |

**表 D.3　terminal と shell のメタキャラクター**

| | |
|---|---|
| control A | 行の先頭へ |
| control B | 一文字前へ |
| control E | 行の最後へ |
| control F | 一文字後ろへ |
| control N | 次の行へ |
| control P | 前の行へ |
| control D | 右を消去 |
| control K | カーソルの後を消去 |
| control H | 左を消去 |
| control V | 一画面移動 |
| control Y | 再入力 |

**表 D.4　Emacs 由来**

# 付録 E　Git と GitHub

## E.1　バージョン管理の必要性

バージョン管理とはあるプロジェクトのファイルやディレクトリーをいつでも元の状態に戻れるように状態を保ちながら編集し続けることをいう．例えば事務的な Excel 書類 hoge.xlsm を

`hoge7_4.xlsm`

のような形で最終変更日を付けて保存したりしたことがあると思う．しかし，複数の人が同じファイルを扱う場合（自分一人の場合でも），2 つのファイル

`hoge7_6.xlsm`

`hoge7_7.xlsm`

が `hoge2026_7_4.xlsm` からどの順番に生成されたか分からなくなったような経験のある人も多いだろう．

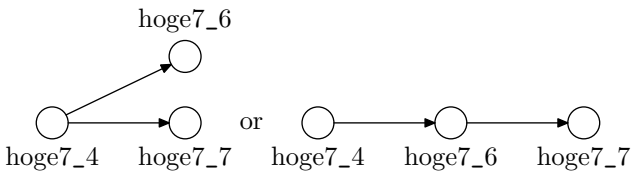



**図 E.1　どっち？**

誰が何時手を入れて，それがどのように統合されたのか履歴と責任を明らかにし，そのサポートを行うのがバージョン管理ソフトの仕事である．もちろん，使う側にもその使い方の理解が欠かせない．Git はそのようなバージョン管理をするためのツールであり，最近ではバージョン管理のデファクトスタンダードである．

Git ではブランチという枝の様な構造を使ってバージョン管理を行っている．新しいことを付け加えるときはまず幹に当たる main ブランチから新しいブランチを切ってそこで作業し，一段落付いたら main ブランチに統合する，あるいは他の人間にレビューしてもらってその後統合するという方針で作業を行う．

GitHub というクラウドサーバーに適宜ファイルを保存し，ネット上の仲間と共同でバージョン管理をするのが今の開発の主流である．

## E.2　バージョン管理ツールの歴史

- SCCS 1972 年 IBM/System 370 向けに開発された．ローカル型.
- RCS 1982 年，ローカル型.
- CVS 1986 年，クライアントサーバー型.
- Subversion 2000 年，クライアントサーバー型.
- Bitkeeper 2000 年，分散型．商用ソフトで Linux の開発でフリーで使われていた.
- Git 2005 年，分散型．突然 Bitkeeper が使えなくなったリーナス・トーバルズが開発を始めた.

## E.3　Git のインストール

Mac や Linux だと既に git がインストールされてるので，ターミナルで

```
% git --version
git version 2.54.0
```

のようにすれば確認出来る．ただし% はプロンプトで入力する必要は無い．必要に応じて次の行に出力も書いておく．Windows だとインストールが必要.

Git の初期化：まず Git に自分のことを教える必要がある.

```
% git config --global user.name "hoge"
% git config --global user.mail "hoge@example.com"
```

後の GitHub の登録後に本人確認のメールとか届くので，本物のメールアドレスを登録すること．細かい設定も似た感じで出来る．例えば先程言った幹に当たるブランチの名前を main にするには

```
% git config --global init.defaultBranch main
```

とすれば良い（以前は master という名前だったが奴隷制を思い出させるということで main のような名前で呼ばれることが多くなった）．なお

```
% git config --list
```

でちゃんと設定できたか確認出来る.

## E.4 Git の最低限の基本操作

Git ではまずはコマンドラインを使って操作し，使いながら Git におけるバージョン管理の概念・使用法の理解を深めていくことをお奨めする．分かってきたらグラフィカルな方法に移って構わない．

- Git はディレクトリ単位でバージョン管理するので，まずディレクトリを初期化する必要がある

```
% mkdir hogehoge
% cd hogehoge
% git init
```

新しいファイルを作って最初のコミットを行う．最初は説明を省くが後で説明を加える．

```
% echo "Hello Git!" > textA.txt

% git add textA.txt

% git commit -m "first commit"
```

- git の状態を表示

```
% git status
On branch main
nothing to commit, working tree clean
```

- 新しいことをするには，main ブランチでは作業を行わず，例えば branchA というブランチを作り，そこに移って作業する．

```
% git branch branchA
% git switch branchA
```

この段階で書類を VSCode などで編集すると，このブランチ branchA の上で編集されるだけで，main ブランチの内容はもとのままに止まっているが，現在の編集内容も保存されていない．

例えば適当にファイルを変更して

```
% echo "add a comment" >>  textA.txt
```

- 編集内容を保存するため，ファイル textA.txt をステージング

```
% git add textA.txt
```

し，問題なければコミットして作業中の内容を保存.

```
% git commit -m "edit textA"
```

"edit textA" がそのコミットのコメントで，後で確認出来るように必ず付け加えることになっている．もし git commit のみだとエディターが自動的に立ち上がって，入力を要求される.

以後同じことを繰り返す.

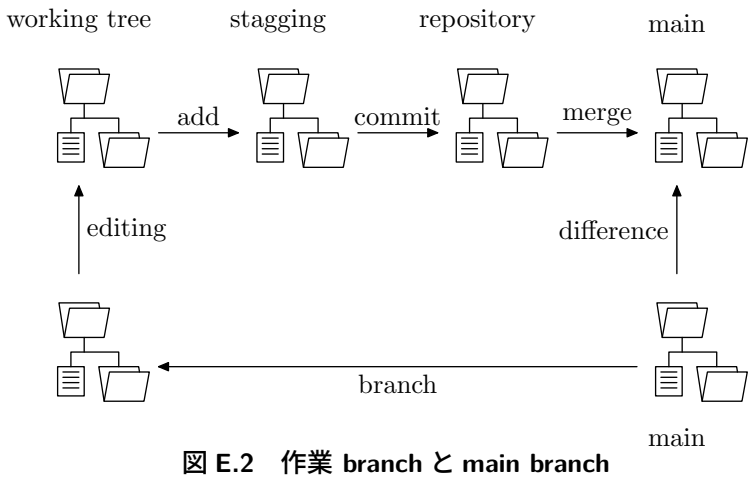



**図 E.2　作業 branch と main branch**

- 幾つものファイルを同時に編集しても構わないし，何度かブランチ branchA 上でコミットを続けることも出来るので適宜部分的に保存しながら作業できる.
- branchA ブランチの変更した内容を main ブランチに付け加えることをマージというが，特に一人で開発しているときはブランチ一個で作業しストレートフォワードマージといい，何も手を入れずそのまま main にマージすることが多いだろう.

やり方を説明すると，一回メインブランチに移動して

```
% git switch main
% git merge branchA
```

大事なことなので強調しておくと，Git では main で編集作業をすることは推奨されていない．新たな編集作業は新たなブランチで行い，切りの良いとき main に統合

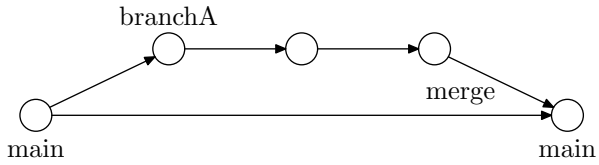



**図 E.3　branchA と Main branch とコミット状況**

することを繰り返すことが奨励されている.

**演習 42**　以上のことをターミナルで行なってください.

## E.5　Git のやや詳しい操作

以下もう少し詳しいやり方を説明する：

- ブランチの関係を見る

```
% git log
```

情報が多すぎるので各項目を一行で表す

```
% git log --oneline
011784d (HEAD -> main, origin/main, testSite) add site data
c2b3d4f first commit
```

の方が見やすいかもしれない．最初の数字がコミットハッシュ (の一部) で，この数字でコミットを制御している．関係をグラフで表すことも出来る

```
% git log --graph --oneline
```

- リポジトリーの全てのブランチ名の表示

```
% git branch
```

- コッミットハッシュ c1f4522 のブランチに移動

```
% git switch --detach c1f4522
```

- ブランチ<branchName>の削除

```
% git branch -d <branchName>
```

適宜不必要なブランチを消して整理することをお奨めする.

- ステージングの取消し

```
% git rm --cached <fileName>
```

- 全ての変更ファイルをステージング：

```
% git add -A
```

- カレントディレクトリの全ての変更ファイルをステージング：

```
% git add .
```

- コミットしたコメントを間違えたときの変更の仕方：

```
% git commit --amend -m “amend text”
```

- 最後のマージとの変更点表示

```
% git diff
```

- <fileName>の変更点表示

```
% git diff -- <fileName>
```

- 途中を消すには

```
% git merge --squash <branchName>
```

- merge で衝突が起こった場合の解消

```
% git merge --abort
```

で一回中止し，編集して衝突を除く.

- マージの他に rebase というのもある

```
% git rebase
```

- main で間違えて作業したときの対応：まず変更点を保存して

```
% git stash
```

ブランチを切って

```
% git switch -c branch
```

その branch に変更点をアップ

```
% git stash pop
```

まだ他にも沢山のコマンドがあるので，参考文献などで調べてください．

## E.6 GitHub

ファイルサーバー GitHub を使うと Git の有用性が広がる．ファイルのクラウドへの保存とネット上でのファイルの共有とかが重要である．個人ではフリーで使用できる．

- GitHub を訪れアカウントを作成
- GitHub の自分のページにサインイン
- GitHub の自分のアカウントに ssh で公開鍵を保存：まず自分のパソコンのログインディレクトリーで以下のように公開鍵を作成する．

  ```
  % cd  # ホームディレクトリに移動
  % ssh-keygen # RSA で秘密鍵と公開鍵の作成
  ```

  途中でパスフレーズを 2 回聞かれるので（空白でも作れるが）絶対入力するように．上手くいくと

  ```
  % ls -a
  ...
  .ssh
  ...
  ```

  上のように鍵が作られている．その後, .ssh のなかの公開鍵を GitHub の自分のホームに登録.

  1. GitHub の Home の一番右上のアイコンのボタンをクリック (図 E.4 だと矢印の部分，ファビコンと言うらしい).
  2. 左の Setting のボタンをクリック
  3. SSH and GPG Key のボタンをクリック
  4. SSH の New Key のボタンをクリック
  5. Title と Key を入力
- GitHub で Repository の New ボタンをクリック Repository Name に (適当な名前)fuga

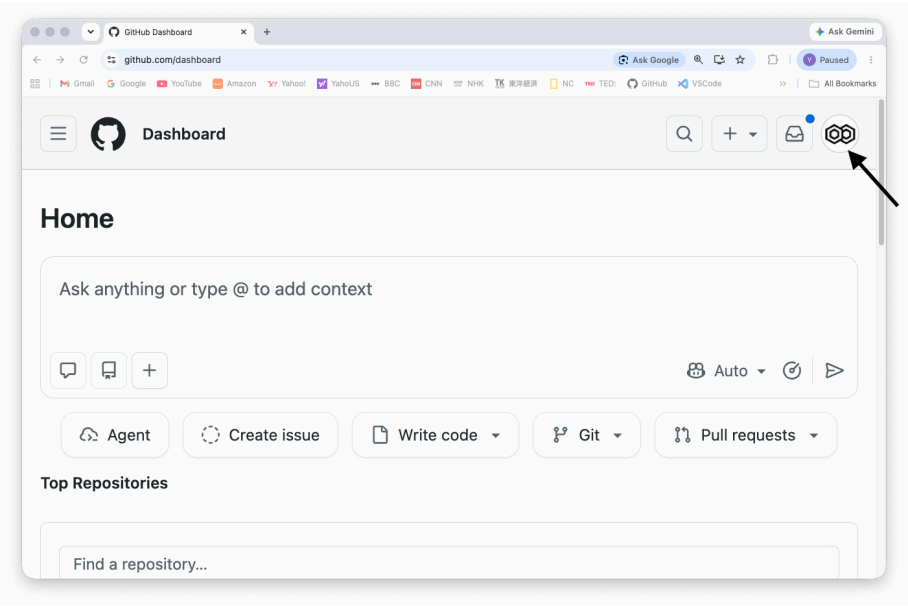



**図 E.4　GitHub の Home**

を記入．他は省略して Create Repository のボタンをクリックするとリポジトリー が作成され，そこには以下のような指示がある：

```
…or create a new repository on the command line

echo "# fuga" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:snow00two/fuga.git
git push -u origin main
```

これに従ってコマンドを実行する．以下やや詳しく解説：

ローカルでディレクトリ fuga を作成し，そこに移動し最低限必要な README.md を作成

```
% mkdir fuga
% cd fuga
% echo "# fuga" >> README.md
```

そのディレクトリの初期化：

```
% git init
```

README.md をステージングしてコミット

```
% git add README.md
```

```
% git commit -m "first commit"
```

Main ブランチを切り設定

```
% git branch -M main
% git remote add origin git@github.com:snow00two/fuga.git
```

最初のプッシュ：

```
% git push -u origin main
```

ここでパスフレーズを聞かれて，正しければ GitHub にデータが送られる．

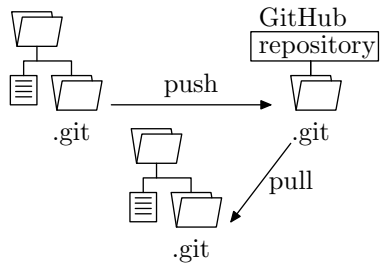



**図 E.5　GitHub, push, pull**

- 以後 main ブランチの新しいデータを GitHub のにアップするのは

```
% git push
```

で良い．
- GitHub からデータを取ってくるには

```
% git pull
```

これは以下と同じ

```
% git fetch
% git merge
```

一つづつ確認して作業を行いたい場合はこちらをお奨めする．

なお最初に GitHub からデータを取ってくるには，例えば

```
% mkdir fractalApp
% cd fractalApp
% git clone https://github.com/snow00two/fractalApp.git
```

のようにすれば良い．

**演習 43**　以上のことを GitHub とターミナルで行なってください．

## E.7　VSCode と Git/GitHub

Git/GitHub と VSCode は相性が良いので，VSCode で Git/GitHub を使うことをお奨めする．

- VSCode で Git で管理されているディレクトリーを開くと自動で Git の管理を感知してターミナルはそのディレクトリーをカレントディレクトリーとして開いてくれる．また．以前使ったコマンドの History の復元を行ってくれ，コマンドラインから Git や GitHub を管理し易い環境になっている．ファイルの編集自体は VSCode の編集画面を使えば良い．
- VSCode の設定を GitHub で同期するように設定できる．
- GitHub の Student Developer Pack[1] に入っていると，VScode の拡張機能タブで「GitHub Copilot」を検索してインストールすると，使用料無しで VScode から生成 AI の GitHub Copilot が使えるようになり，生成 AI にコーディングの補助を受けながらコーディングすることが出来て便利である．

## E.8　GitHub Pages

GitHub では Pages というサービスを使って自分のホームページを世界中に公開できる．

- § E.6 で作ったローカルの Repository fuga に docs というディレクトリを作り，その中にホームページとして実現したい html, css, js のファイルを準備する．markdown でも十分で

1)　2026/4 から新規受付を停止していた．いろいろ変わるので詳しいことはネットで確かめてください．

```
% git branch pages
% git switch pages
% mkdir src docs
% cd src
% echo "# This is my test site." > index.md
% pandoc -f markdown -t html -s -o ../docs/index.html index.md
% git add -A
% git commit -m "add site data"
% git switch main
% git merge pages
% git push
```

- GitHub のホームページの Repository fuga の上の段の Setting(見えないときは More の下に隠れている) を選び，左のコラムの Pages を選択し，セレクターから Deploy from a branch を選びその下の Branch でセレクターから main と /docs 選ぶ.
- 暫くすると Setting で Pages の上の部分に

  Your site is live at https://snow00two.github.io/fuga/

  のようなアドレスが現れると思う．そこをクリックするとあなたのホームページが表示される (図 E.6)．markdown については latex と markdown を参照.

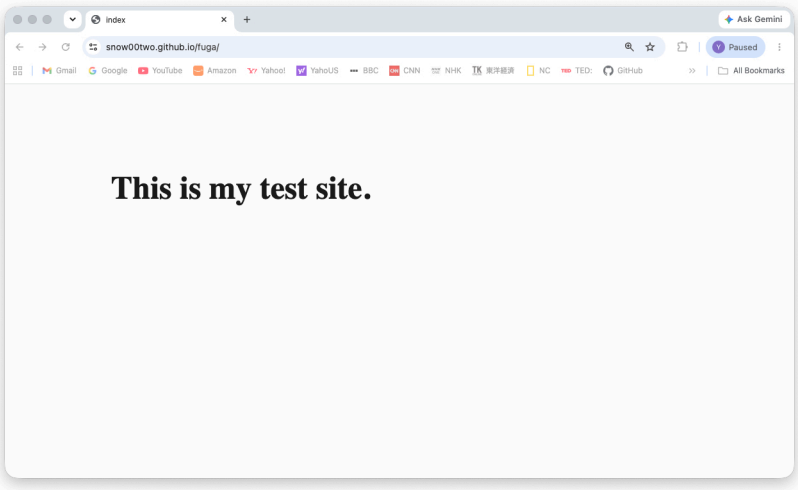



**図 E.6　作られたホームページ**

ただもう少し凝ったホームページを作りたい場合これでは不十分なので，[93] や動画などを参考にして HTML や CSS やデザインの基礎を学んで欲しい.

**演習 44**　以上のことを GitHub で行なってください.

**宿題 11**　皆さんの作ったアプリや動画や論文を世界に向けて発信するウェブサイトを作って公開し，そのアドレスを私の所に送って下さい.

## E.9　この先に

多人数での開発のためには Pull request と Issue が必要になるが，これについては参考文献などを調べてほしい．また，マージするときに衝突が起こる場合があるのでその場合は手動でマージする必要がある．そのやり方も参考文献などを調べてほしい.

また，Actions という GitHub の自動化のサービスもあるので調べて活用してほしい.

# 参考文献


[1] Bool, Laws of thought, 1848

[2] ボズウェル, フィッチャー, リーダブルコード

[3] ジェイミー・バートレット, 操られる民主主義: デジタル・テクノロジーはいかにして社会を破壊するか, 草思社, 2018

[4] ブレーザー, 失われてゆく我々の内なる細菌, みすず書店, 2015.

[5] チャルディーニ：影響力の武器

[6] Feynman, Lectures on computation, Perseus, 1996, 日本語版：ファインマン計算機科学, 岩波書店, 1999.

[7] 藤原進之介, 学校で習っていなくても読んで理解できる藤原進之介のゼロから始める情報 I, KADOKAWA, 2023.

[8] Joe Gray, Jin Maeda, 人を動かすハッカーの技術：ソーシャルエンジニアリングの実践と防御

[9] ゴールドスタイン, 復刊計算機の歴史, 共立, 2016.

[10] 堀田 良之, 加群十話: 代数学入門 (すうがくぶっくす 3), 朝倉書店, 1988.

[11] アンデシュ・ハンセン, スマホ脳, 新潮新書, 2020.

[12] ユバル・ノラ・ハラリ, サピエンス全史

[13] NISC, インターネット安全・安心ハンドブック

[14] 林 裕之, 「隠れ油」という大問題, 三五館, 2017.

[15] 平田 賀一, IT エンジニアのための Markdown 実践入門 —— 生成 AI 時代の”伝わる”書き方, 技術評論社, 2026.

[16] 石井光太, ルポ スマホ育児が子どもを壊す, 新潮社, 2024.

[17] Apple Computer, Apple Human Interface Guidelines: The Apple Desktop Interface, Addison-Wesley, 1987, 日本語版, 1989.

[18] 渡辺さき［令和 8 年度］IT パスポート 超効率の教科書＋よく出る問題集, インプレス, 2026.

[19] 情報処理推進機構, 安全なウェブサイトの作り方

[20] 情報処理推進機構, サイバーセキュリティ経営ガイドライン

[21] 情報処理推進機構, 中小企業の情報セキュリティガイドライン

[22] 一 般社団法人 JPCERT コーディネーションセンター, インシデントハンドリングマニュアル

[23] アイザックソン, スティーブ・ジョブス I, II, 講談社, 2011.

[24] カーニハン, 教養としてのコンピューターサイエンス講義 第二版, 日経 BP, 2020.
[25] 木村秀紀, Fourier-Laplace 解析, 岩波書店, 1993.
[26] 小学館 DVDBook, 歌舞伎のいき, 第 2 巻「時代物・荒事編」, 2008.
[27] カーニハン, カーニハンの UNIX 回顧録, 丸善, 2025.
[28] Kernighan, Pike, Unix programming environment, Pearson, 1984 (訳 UNIX プログラミング環境, KADOKAWA, 2017).
[29] Kernighan, Ritchie, The C programming Language–2nd edition, Prentice Hall,1989. (訳 プログラミング言語 C 第 2 版 ANSI 規格準拠, 共立出版, 1989.)
[30] アラン・ケイ, アラン・ケイ, ASCII, 1992.
[31] Koblitz, A course in Number theory and Cryptography, GTM, ダウンロード
[32] 小森 裕介, [ 改訂新版] プロになるための Web 技術入門, 技術評論社, 2024.
[33] 川本 晃司, スマホ失明, かんき出版, 2022.
[34] 川島 隆太, 榊 浩平, スマホはどこまで脳を壊すか, 朝日新書, 2023.
[35] 熊野 整, 外資系投資銀行の Excel 仕事術, ダイヤモンド社, 2015.
[36] 工藤 喜美枝, 入門!Excel VBA クイックリファレンス 改訂版, ムイスリ出版, 2018.
[37] ヴァイバー・クリガン=リード, サピエンス異変——新たな時代「人新世」の衝撃, 飛鳥新社, 2018.
[38] カール・ポッパー, 科学的発見の論理 上下, 恒星社厚生閣, 1971.
[39] ダニエル・リーバーマン, 人体六〇〇万年史上・下——科学が明かす進化・健康・疾病, 早川書房, 2015.
[40] ダニエル・リーバーマン, マイケル・ロング , もっと! : 愛と創造，支配と進歩をもたらすドーパミンの最新脳科学, インターシフト (合同出版), 2020.
[41] ティム・バーナーズ=リー, Web の創成 —World Wide Web はいかにして生まれどこに向かうのか, 毎日コミュニケーションズ, 2001.
[42] Berryman, Zigler, LLM のプロンプトエンジニアリング, オライリージャパン, 2025.
[43] Minsky, Computation: Finite and Infinite Machines, 1967.
[44] 三浦伸夫, 数学の歴史, 放送大学教育振興会, 2019.
[45] マロット, 行動分析学入門, 産業図書, 1998.
[46] 物江 潤, デマ・陰謀論・カルト (新潮新書) , 新潮社, 2022.
[47] 物江 潤, デジタル教育という幻想 GIGA スクール構想の過ち, 平凡社, 2023.
[48] 松田晃一, p5.js プログラミングガイド 改訂版, 2021, カットシステム
[49] 一般社団法人 日本シーサート協議会シーサート人材ワーキンググループ, 改訂新版 セキュリティエンジニアの教科書 一般社団法人 日本シーサート協議会シーサート人材ワーキンググループ

[50] 長迫, 小谷, 大澤, SNS 時代の戦略兵器 陰謀論 民主主義をむしばむ認知戦の脅威, ウェッジ, 2025.
[51] NISC, インターネット安全・安心ハンドブックダウンロード
[52] 小川和久, 総理、国防も安全も穴だらけ！国民を守れない国・ニッポン
[53] H.G. Orwell, Nineteen Eighty-Four, 1949.
[54] 奥村 晴彦, 森本 尚之, ［改訂第 5 版］基礎からわかる情報リテラシー, 技術評論社, 2023.
[55] イーライ・パリサー, 閉じこもるインターネット――グーグル・パーソナライズ・民主主義 , 早川書房, 2012. 改題：フィルターバブル――インターネットが隠していること (ハヤカワ文庫 NF), 早川書房, 2016.
[56] キャサリン・プライス, スマホ断ち 30 日でスマホ依存から抜け出す方法, 角川新書, 2024.
[57] ラインゴールド, 思考のための道具, パーソナルメディア, 1987.
[58] ガー・レイノルズ, プレゼンテーション Zen 第 3 版, 丸善出版, 2021.
[59] 坂村健, 高等学校 情報 I , 数研出版, 2021.
[60] Silverman, Tate, Rational Points on Elliptic Curves, Springer, 2015
[61] 瀬戸, 斎藤, 徹底攻略情報セキュリティマネージメント, インプレス
[62] 高木貞治, 初等整数論講義, 共立出版, 1971.
[63] 達人プログラマー
[64] 高橋 佑磨, 片山 なつ, 伝わるデザインの基本 増補改訂 3 版よい資料を作るためのレイアウトのルール, 技術評論社, 2021.
[65] 田中靖浩, 会計の世界史, 日本経済新聞出版, 2018.
[66] 井上 直也, 村山 公保, マスタリング TCP/IP―入門編―(第 6 版), オーム社, 2019.
[67] アンドルー・ホッジス, エニグマ アラン・チューリング伝 上下, 勁草書房, 2015. 映画化：イミテーション・ゲーム エニグマと天才数学者の秘密, モルテン・ティルドゥム監督, 2014.
[68] A. Turing, On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society 42, 1936.
[69] 宇津木龍一, シャンプーをやめると髪が増える, KADOKAWA, 2013.
[70] 内田 広由紀, 7 日間でマスターする 配色基礎講座 (デザイン ビギナー シリーズ), 2000
[71] クジラ飛行机, プログラミング言語大全, 技術評論社, 2025.
[72] Robin Williams, ノンデザイナーズ・デザインブック [第 4 版], マイナビ出版, 2016
[73] 矢野 啓介, [改訂新版］プログラマのための文字コード技術入門, 技術評論社, 2018.

[74] 山森丈範，[改訂第 4 版] シェルスクリプト 基本レファレンス, 技術評論社, 2024.
[75] 英文詳説世界史, 山川出版, 2019.
[76] 山口泰, 東京大学教養学部テキスト情報 第三版, 東京大学出版会, 2025.
[77] 吉田 拳, たった 1 日で即戦力になる Excel の教科書, 技術評論社, 2014.
[78] 国本 温子, 緑川 吉行, できる大事典 Excel VBA 2019/2016/2013& Microsoft 365, インプレス, 2021.
[79] 尾崎裕子, 日花弘子, できる大事典 Excel 2016, インプレス, 2021.
[80] 羽山博, 吉川明広, できる大事典 Excel 関数 2016/2013/2010, インプレス, 2017.
[81] シェルスクリプト基本レファレンス
[82] 結城浩, 暗号技術入門 第 3 版, SB クリエイティブ, 2015.
[83] 山田祥寛, 改訂 3 版 JavaScript 本格入門, 2023, 技術評論社
[84] TeXShop のメニューバーの Help メニューにある説明文書と動画
[85] /usr/local/texlive/2024/texmf-dist/doc/latex の中のドキュメントファイル: User's guide for amsmath package, Using the amsthm package, XY-pic User's Guide, XY-pic Reference Manual, The METAPOST system, A users manual for METAPOST など
[86] 奥村晴彦, 黒木裕介, ［改訂第 9 版］LaTeX 美文書作成入門, 技術評論社, 2023.
[87] The LaTeX Companions Third Revised Boxed Set, Addison-Wesley, 2007.
[88] Donald E. Knuth, The METAFONTbook, Addison-Wesley,1986.
[89] サル先生の Git 入門
[90] 高橋 あおい, つくって、壊して、直して学ぶ Git & GitHub 入門, 翔泳社, 2025.
[91] 渡辺宙志, ゼロから学ぶ Git/GitHub, 講談社, 2024.
[92] ポンストール, ロリガー, 実用 Git 第 3 版, オライリー, 2024.
[93] Mana, 1 冊ですべて身につく HTML & CSS と Web デザイン入門講座［第 2 版］, SB クリエイティブ, 2024.

# 索引

16 進法, 11
2 進法, 11

CSS, 58

DHCP, 52
DNS, 53

HTML, 55
HTTP, 55
HTTPS, 58

IP address, 52

JavaScript, 58

RGB, 18
RSA, 71

TCP, 53

UDP, 54
URL, 55

VPN, 81

Wellknown port, 54
WWW, 55

アプリケーション層, 54
アルゴリズム, 29
インターネット, 48
オイラー関数, 67
オートマトン, 38
カラーコード, 18
クエリ, 58
クッキー, 59
クライアント, 55
サイバーキルチェーン, 77
サンプリング, 20
サンプリング周波数, 20
サンプリング定理, 21
サーバー, 55
スキーム, 55
ソーシャルエンジニアリング, 84
タグ, 56
チャレンジ, 80
チューリングマシン, 40
テキスト, 12
デジタル署名, 72
ドメインネーム, 52
ハイパーテキスト, 55
バイト, 12
バイナリー, 12
パスキー, 80
パーセントエンコーディング, 59
ビット, 11
ファイヤーウォール, 80
フェルマーの小定理, 68
フリップフロップ回路, 34
ブラウザー, 55
プロトコル, 51
ヘッド, 40
ホフマン法, 28
ポート, 53
ユニバーサルチューリングマシン, 43
ランレングス法, 27
リスク, 76

ルーティング, 53
レインボーテーブル, 26

一方向ハッシュ関数, 25
不可逆圧縮, 27
中間者攻撃, 73
二要素認証, 80

公開鍵暗号, 71
単位元, 65
危殆化, 26
可換群, 65
可逆的圧縮, 27
圧縮率, 27
多要素認証, 80
巡回群, 66

情報セキュリティ, 74
情報セキュリティマネージメント, 74
情報資産, 76
拡張子, 5
文字列, 17
既約剰余類, 67
最小アクセス権の原理, 80
有用性, 75
構造化データ, 23
標数, 69
機密性, 75

環, 65
相対 path, 5
秘密鍵暗号, 70
符号化, 12
素体, 70
統一性, 75

絶対 path, 4
群, 65

脅威, 76
脆弱性, 76
認可, 79
認証, 79
認証局, 73
辞書攻撃, 26

逆元, 65
量子化ビット数, 21